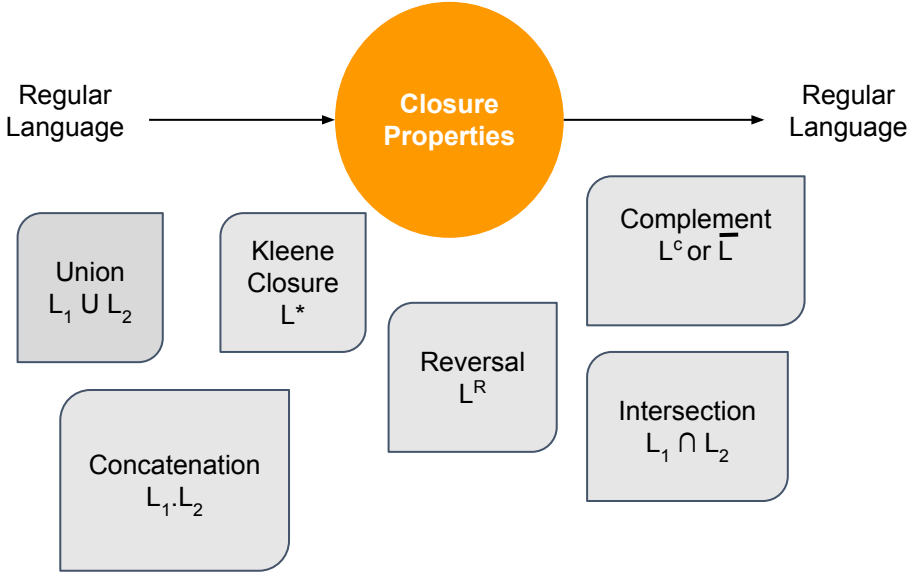
Automata Formal Languages & Logic

Preet Kanwal
Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

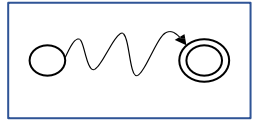
Preet Kanwal
Department of Computer Science Engineering


Regular Language
Closure Properties
Regular Language
Union
$L_1 \cup L_2$
Kleene Closure
L^*
Complement
L^c or $\overline{L}$
Reversal
L^R
Concatenation
$L_1.L_2$
Intersection
$L_1 \cap L_2$

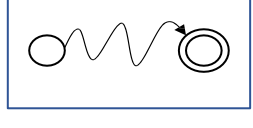
PES UNIVERSITY ONLINE

Union
$L_1 \cup L_2$

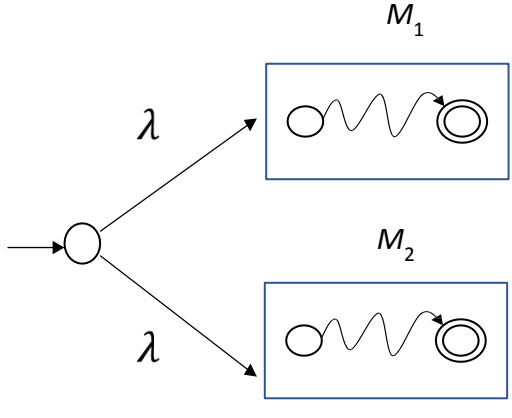
M_1



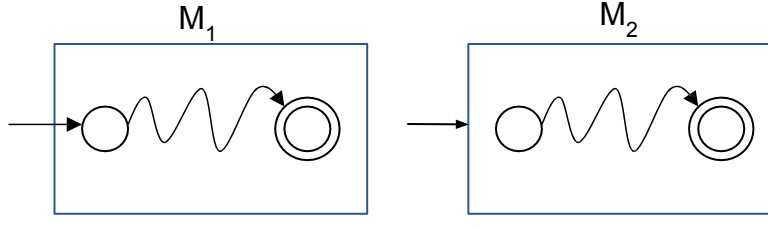
M_2



Union
$L_1 \cup L_2$

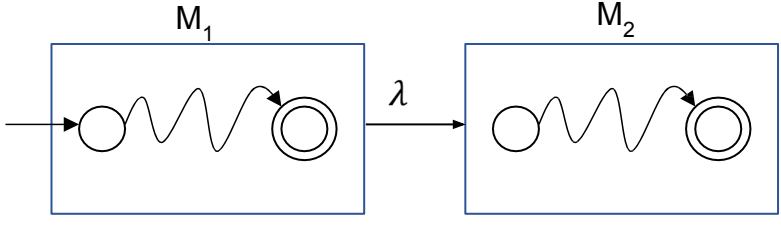


Concatenation
$L_1.L_2$

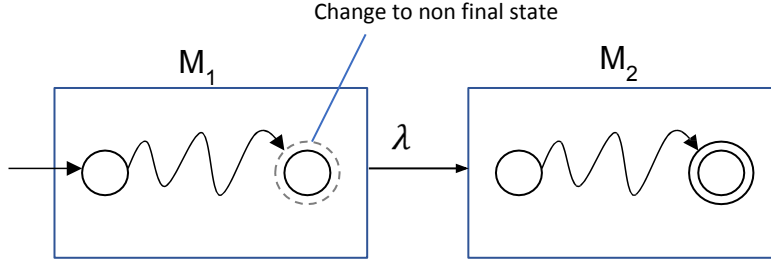


PES UNIVERSITY ONLINE

Concatenation
$L_1.L_2$

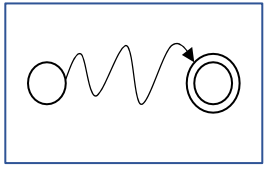


Concatenation
$L_1.L_2$



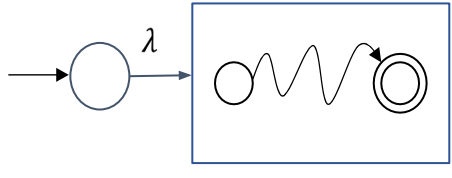
M

Kleene Closure L*



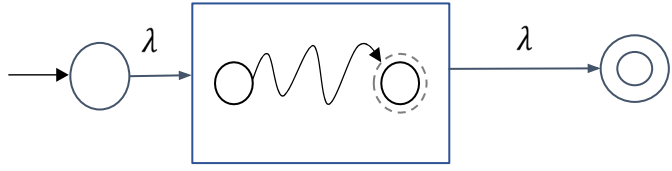
M

Kleene Closure L*

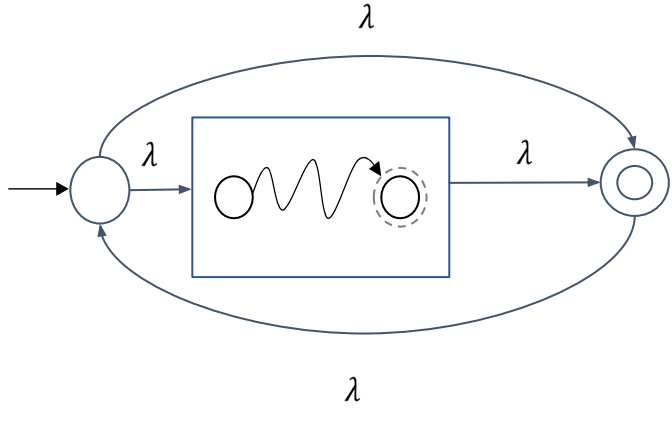


M

Kleene Closure L*

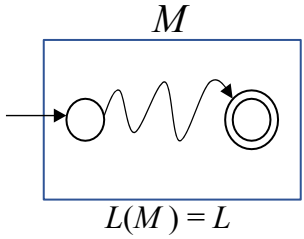


Kleene Closure L*



PES UNIVERSITY ONLINE

Reversal
L^R

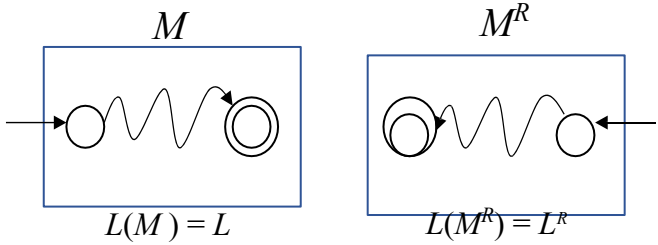


How to reverse of a Regular Language :

- Make Start State as Final State
- Final State as Start State
- Reverse all the transitions
- If there is more than one final state in the machine M , a new start can be introduced with lambda transitions leading from it to these multiple final states.

PES UNIVERSITY ONLINE

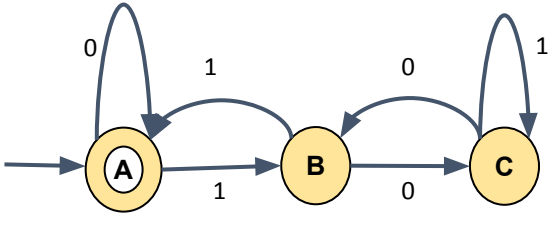
Reversal
L^R



How to reverse of a Regular Language :

- Make Start State as Final State
- Final State as Start State
- Reverse all the transitions
- If there is more than one final state in the machine M , a new start can be introduced with lambda transitions leading from it to these multiple final states.

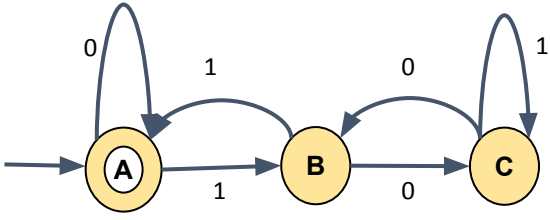
Unit 2 - Example where L and L^R is a same language



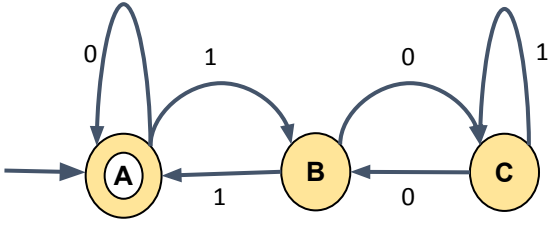
L = L(M)

= Binary Strings divisible by 3

Unit 2 - Example where L and L^R is a same language

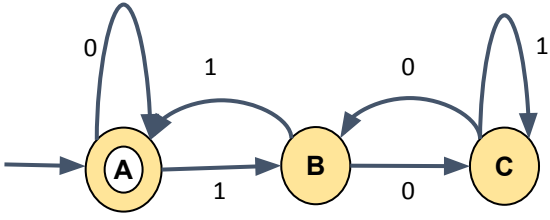


L = L(M)
= Binary Strings divisible by 3



$L^R = L(M^R)$
= Binary Strings divisible by 3

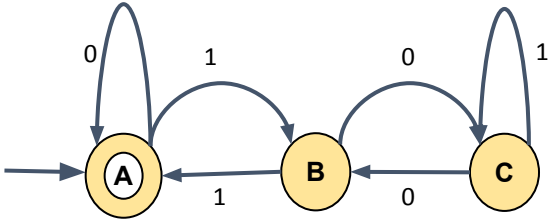
Unit 2 - Example where L and L^R is a same language



L = L(M)
= Binary Strings divisible by 3

Each String is a palindrome

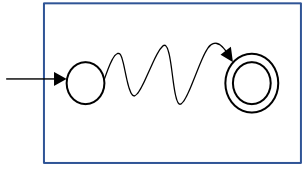
$L^R = L(M^R)$
= Binary Strings divisible by 3



PES UNIVERSITY ONLINE

Complement
L^c or $\overline{L}$

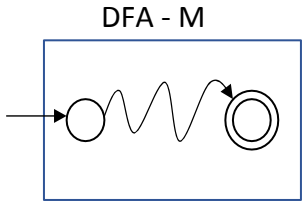
DFA - M

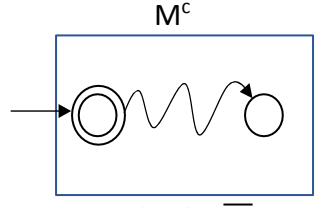


How to Complement a Regular Language :

- Construct a DFA.
- Toggle the final and non-final states.
- Works only for the DFAs and not for the NFAs

Complement
L^c or $\overline{L}$



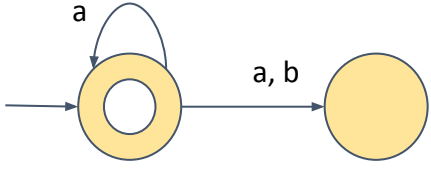


$$L(M^c) = \overline{L}$$

How to Compliment a Regular Language :

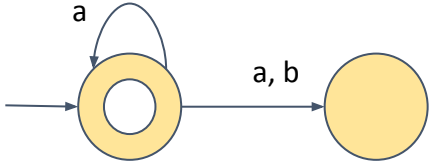
- **Construct a DFA.**
- **Toggle the final and non-final states.**
- **Works only for the DFAs and not for the NFAs**

Complement of a NFA may not be complement of the regular language

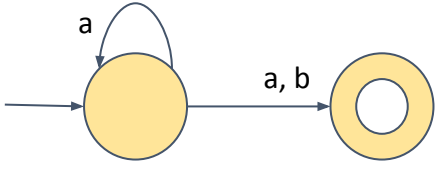


L = { λ, a, aa, aaa .. }

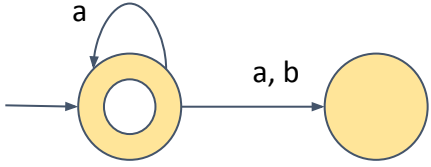
Complement of a NFA may not be complement of the regular language



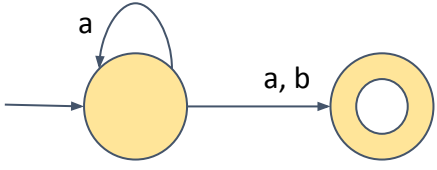
L = { λ, a, aa, aaa .. }



Complement of a NFA may not be complement of the regular language

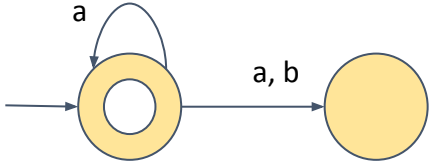


L = { λ, a, aa, aaa .. }



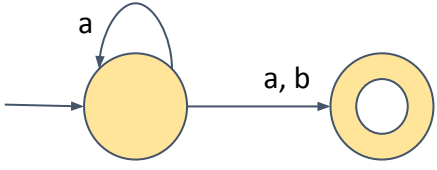
L = a*(a+b)

Complement of a NFA may not be complement of the regular language



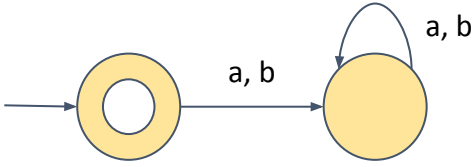
L = { λ, a, aa, aaa .. }

Complement of NFA is a not a complement of the language



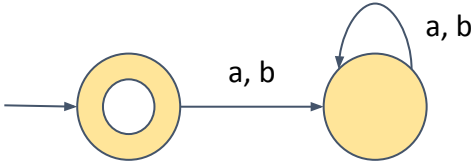
L = a*(a+b)

Complement of a NFA may not be complement of the regular language

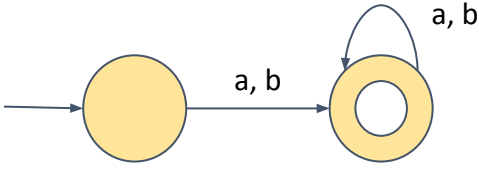


L = { λ }

Complement of a NFA may not be complement of the regular language



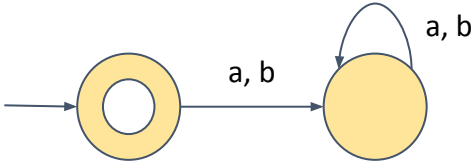
$L = \{ \lambda \}$



$L = (a+b)^+$

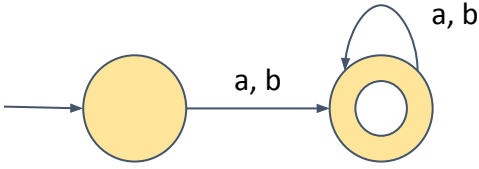
PES UNIVERSITY ONLINE

Complement of a NFA may not be complement of the regular language



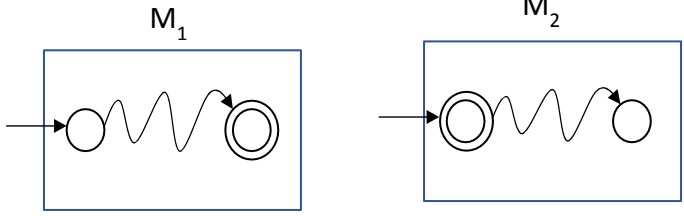
$L = \{ \lambda \}$

Complement of NFA is a complement of the language



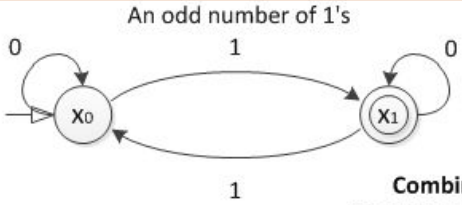
$L = (a+b)^+$

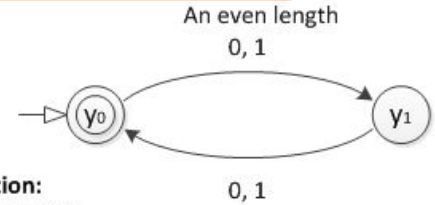
Intersection
$L_1 \cap L_2$



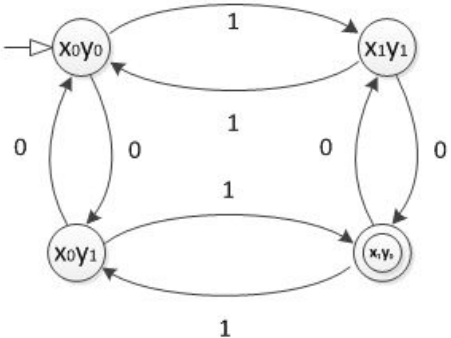
Two ways to find Intersection of two Regular Languages :

1. **Take Cartesian production of the states of the two FA:**
   - **#of States in new FA will be m x n where,**
     m - # states in M_1 and n - # states in M_2
   - **Start state of new FA will be the pair :**
     {Start State of M_1 , Start State of M_2)}
   - **Final state of new FA will be the pair :**
     {Final State of M_1 , Final State of M_2)}
2. **Use De Morgan's Law**





Combination:
An odd number of 1's
& an even length



DeMorgan's Law: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$

	L_1 , L_2	regular, regular
⇒	$\overline{L_1}$, $\overline{L_2}$	regular, regular
⇒	$\overline{L_1} \cup \overline{L_2}$	regular
⇒	$\overline{\overline{L_1} \cup \overline{L_2}}$	regular
⇒	$L_1 \cap L_2$	regular

PES UNIVERSITY ONLINE

Answer the following :

1. if $L_1 = \{a^n b\}$ and $L_2 = \{ ba\}$ then prove that $L_1 \cup L_2$ is regular
2. if $L_1 = \{a^n b\}$ and $L_2 = \{ ba\}$ then prove that $L_1 . L_2$ is regular
3. if $L = \{a^n b\}$ then prove that L^* is regular
4. if $L = \{a^n b\}$ then prove that L^R is regular
5. if $L = \{a^n b\}$ then prove that L^c is regular
6. Let L1 = {w | w contains '11' as a substring} and
   L2 = {w | w contains '00' as a substring} then prove that L1 ∩ L2 is regular. That is come up with an automaton accepting the strings that contain both '11' and '00' as a substring.

PES UNIVERSITY ONLINE

Can you answer the following about Regular language:

1. Membership question : Does String w $\in$ L?
2. Testing Emptiness : Does L = { }?
3. Does L = Σ^* ??
4. Is a given language finite or infinite?
5. Given a regular language L_1 and L_2 can we check if $L_1 = L_2$?

PES UNIVERSITY ONLINE

Can you answer the following about Regular language:

1. Membership question : Does String w $\in$ L?
   Construct a FA for L and check whether w lands in a final state.
2. Testing Emptiness : Does L = { }?
3. Does L = Σ^* ??
4. Is a given language finite or infinite?
5. Given a regular language L_1 and L_2 can we check if $L_1 = L_2$?

PES UNIVERSITY ONLINE

Can you answer the following about Regular language:

1. Membership question : Does String w $\in$ L?
   Construct a FA for L and check whether w lands in a final state.
2. Testing Emptiness : Does L = { }?
   Yes if there is no path from start state to final state.
3. Does L = Σ^* ??
4. Is a given language finite or infinite?
5. Given a regular language L_1 and L_2 can we check if $L_1 = L_2$?

PES UNIVERSITY ONLINE

Can you answer the following about Regular language:

1. Membership question : Does String w $\in$ L?
   Construct a FA for L and check whether w lands in a final state.
2. Testing Emptiness : Does L = { }?
   Yes if there is no path from start state to final state.
3. Does L = Σ^* ??
   Check if complement of L accepts nothing.
4. Is a given language finite or infinite?
5. Given a regular language L_1 and L_2 can we check if $L_1 = L_2$?

PES UNIVERSITY ONLINE

Can you answer the following about Regular language:

1. Membership question : Does String w $\in$ L?

   Construct a FA and check whether w lands in a final state.

2. Testing Emptiness : Does L = { }?

   Yes if there is no path from start state to final state.

3. Does L = Σ^* ??

   Check if complement of L accepts nothing.

4. Is a given language finite or infinite?

   Construct if FA for L contains a loop. If yes, then the lang is infinite.

5. Given a regular language L_1 and L_2 can we check if $L_1 = L_2$?

PES UNIVERSITY ONLINE

Can you answer the following about Regular language:

1. Membership question : Does String w $\in$ L?

   Construct a FA and check whether w lands in a final state.

2. Testing Emptiness : Does L = { }?

   Yes if there is no path from start state to final state.

3. Does L = Σ^* ??

   Check if complement of L accepts nothing.

4. Is a given language finite or infinite?

   Construct if FA for L contains a loop. If yes, then the lang is infinite.

5. Given a regular language L_1 and L_2 can we check if $L_1 = L_2$?

   If minimal DFA for both L1 and L2 is same the languages are equal.

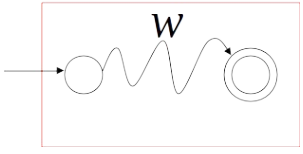
THANK YOU

Preet Kanwal
Department of Computer Science & Engineering
preetkanwal@pes.edu
+91 80 6666 3333 Extn 724

Decidable properties of regular language

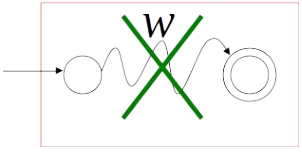
1. Membership Question

DFA



$w \in L$

DFA



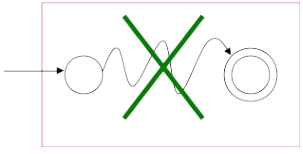
$w \notin L$

2. Testing Emptiness

DFA

$L \neq \emptyset$

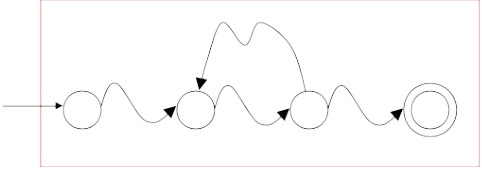
DFA



$L = \emptyset$

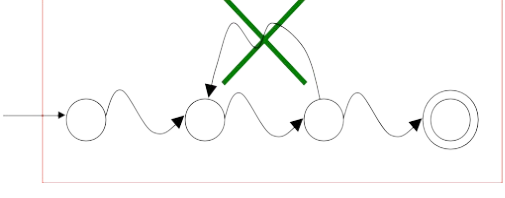
3.Is a given regular language finite or Infinite?

DFA



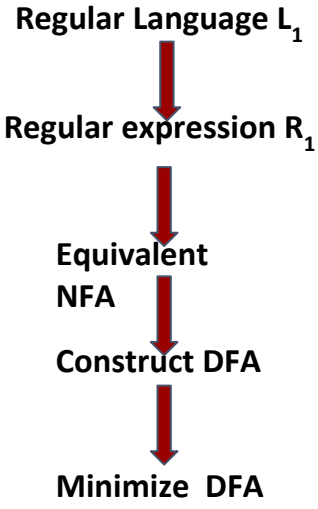
L is infinite

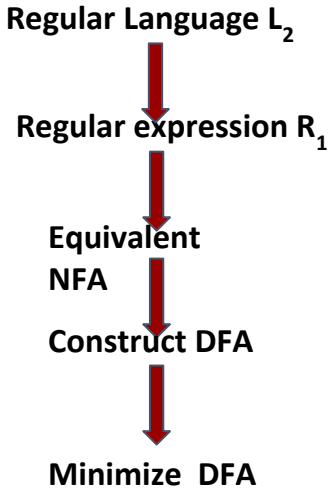
DFA



L is finite

4.Given a regular language L_1 and L_2 how can we check if $L_1 = L_2$?





RE (A) =RE (B) iff the minimized DFA of both the expression are same as the minimized DFA is unique

THANK YOU

Preet Kanwal
Department of Computer Science & Engineering
preetkanwal@pes.edu
+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

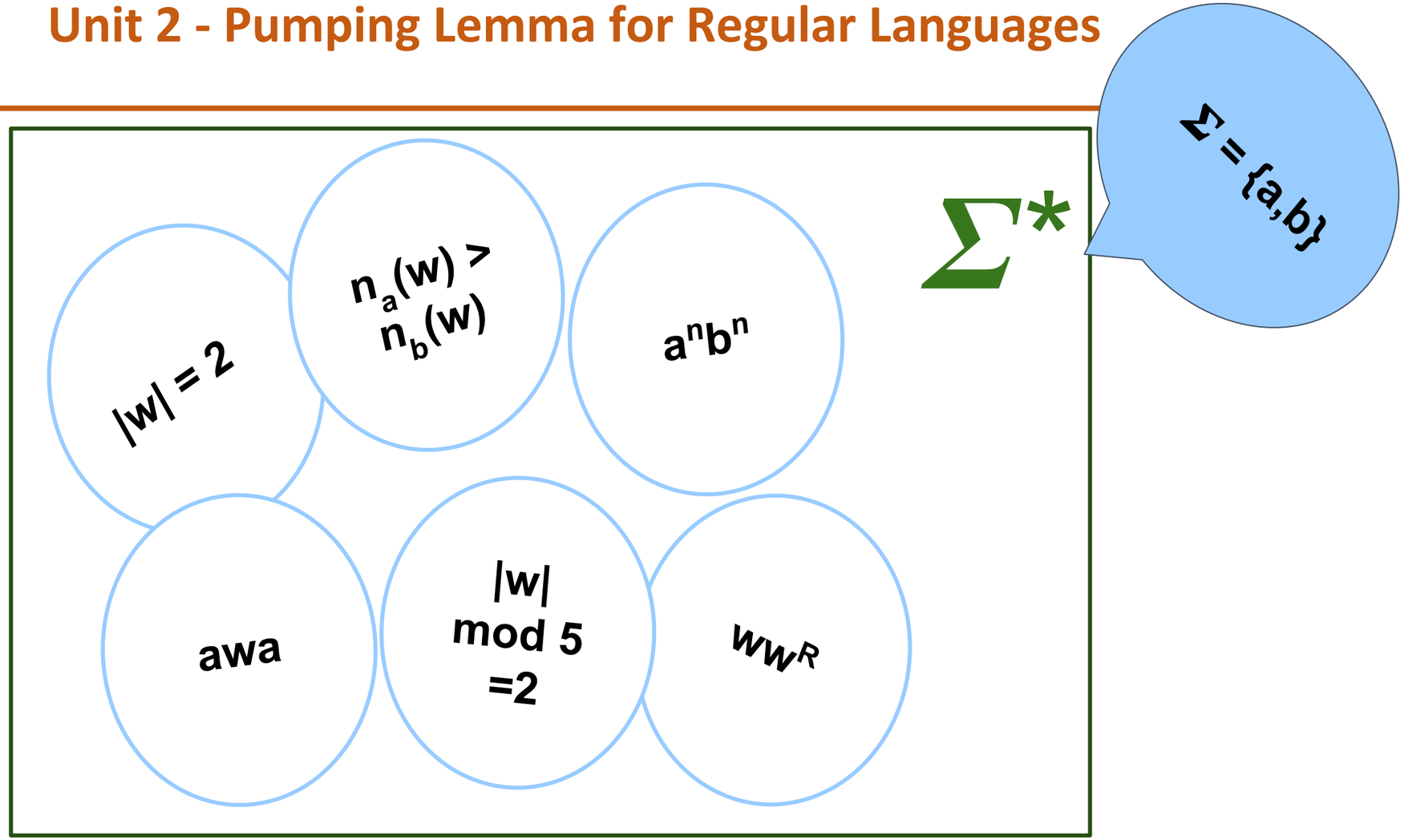
Unit 2

Preet Kanwal

Department of Computer Science & Engineering

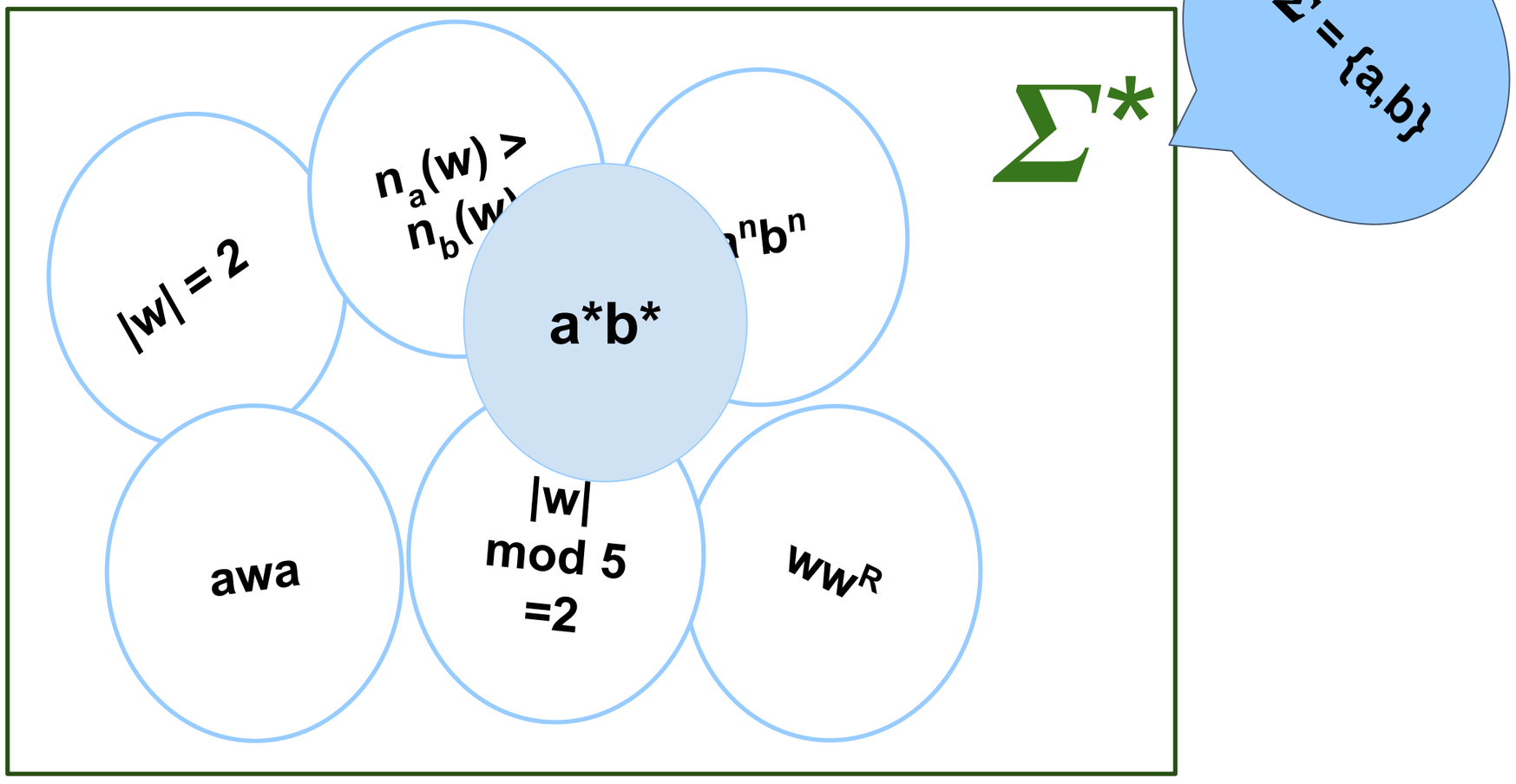
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages



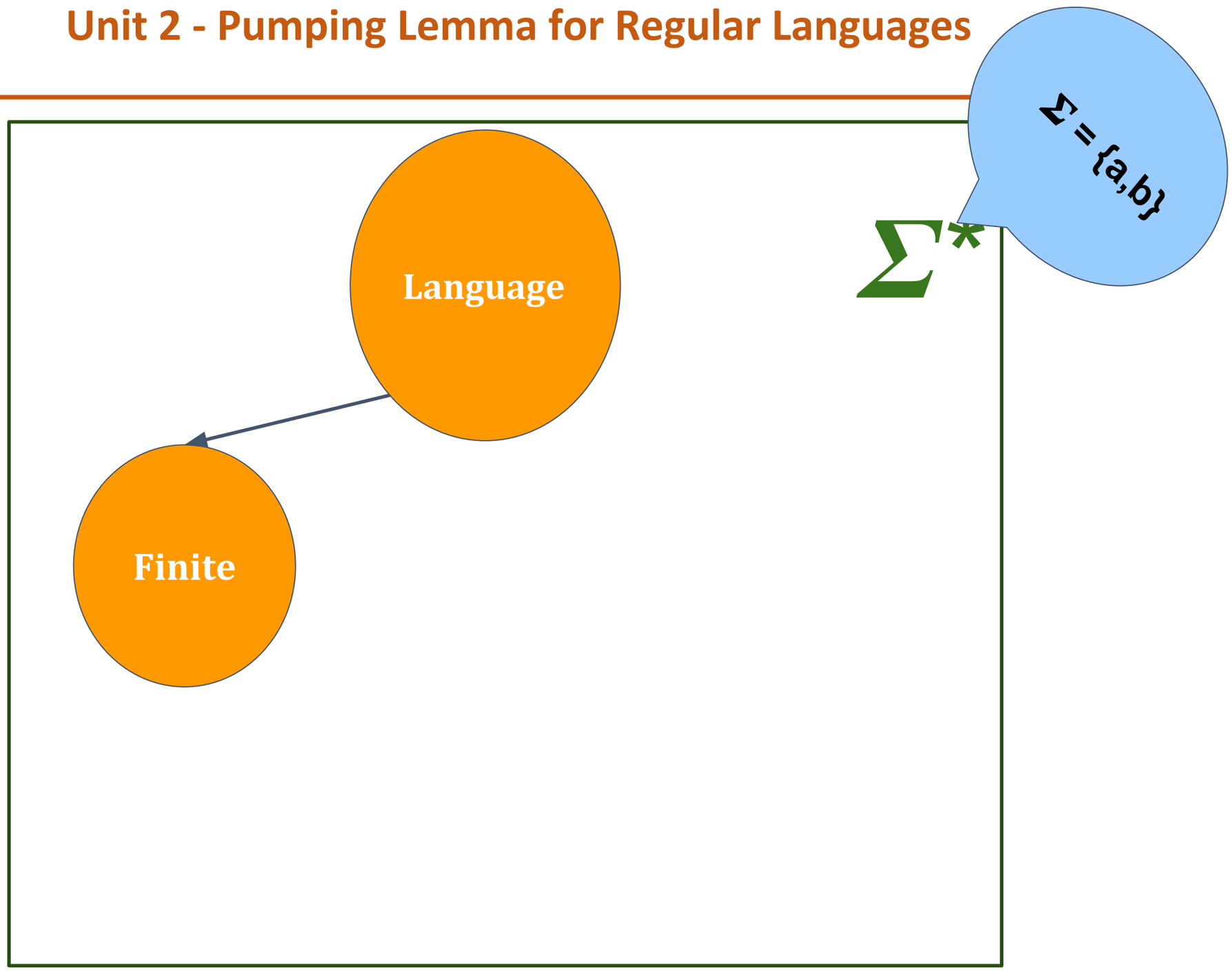
Automata Formal Languages and Logic

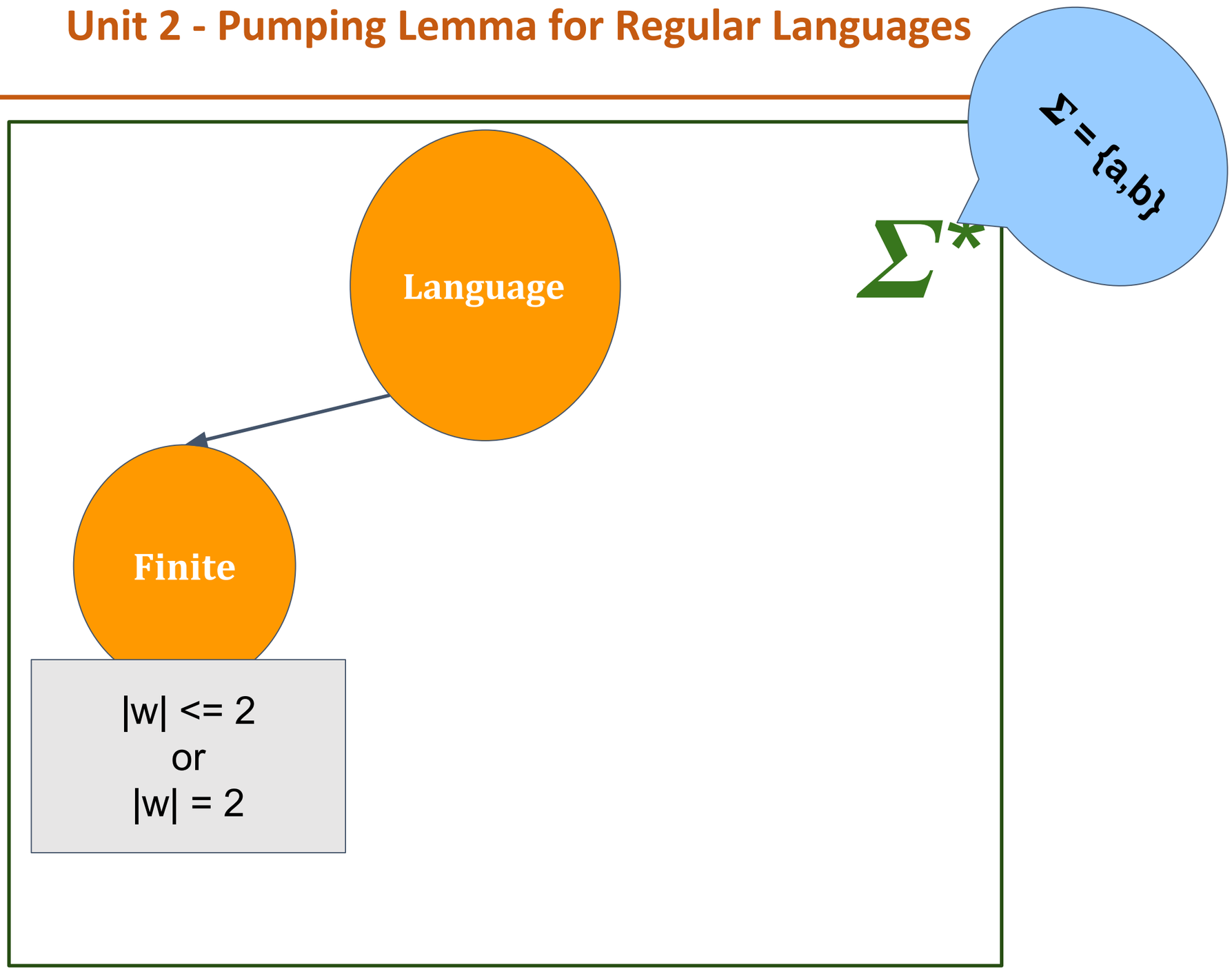
Unit 2 - Pumping Lemma for Regular Languages

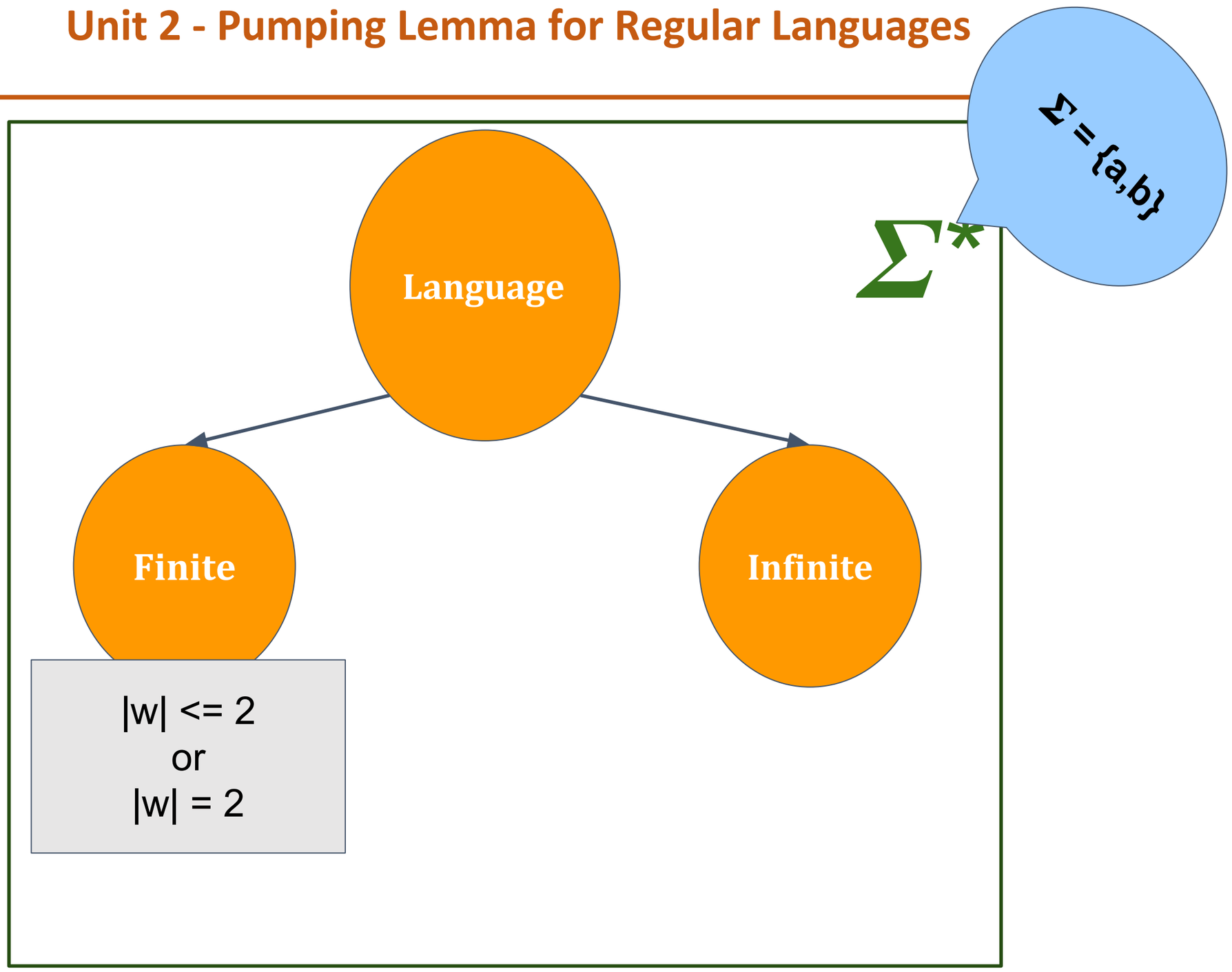


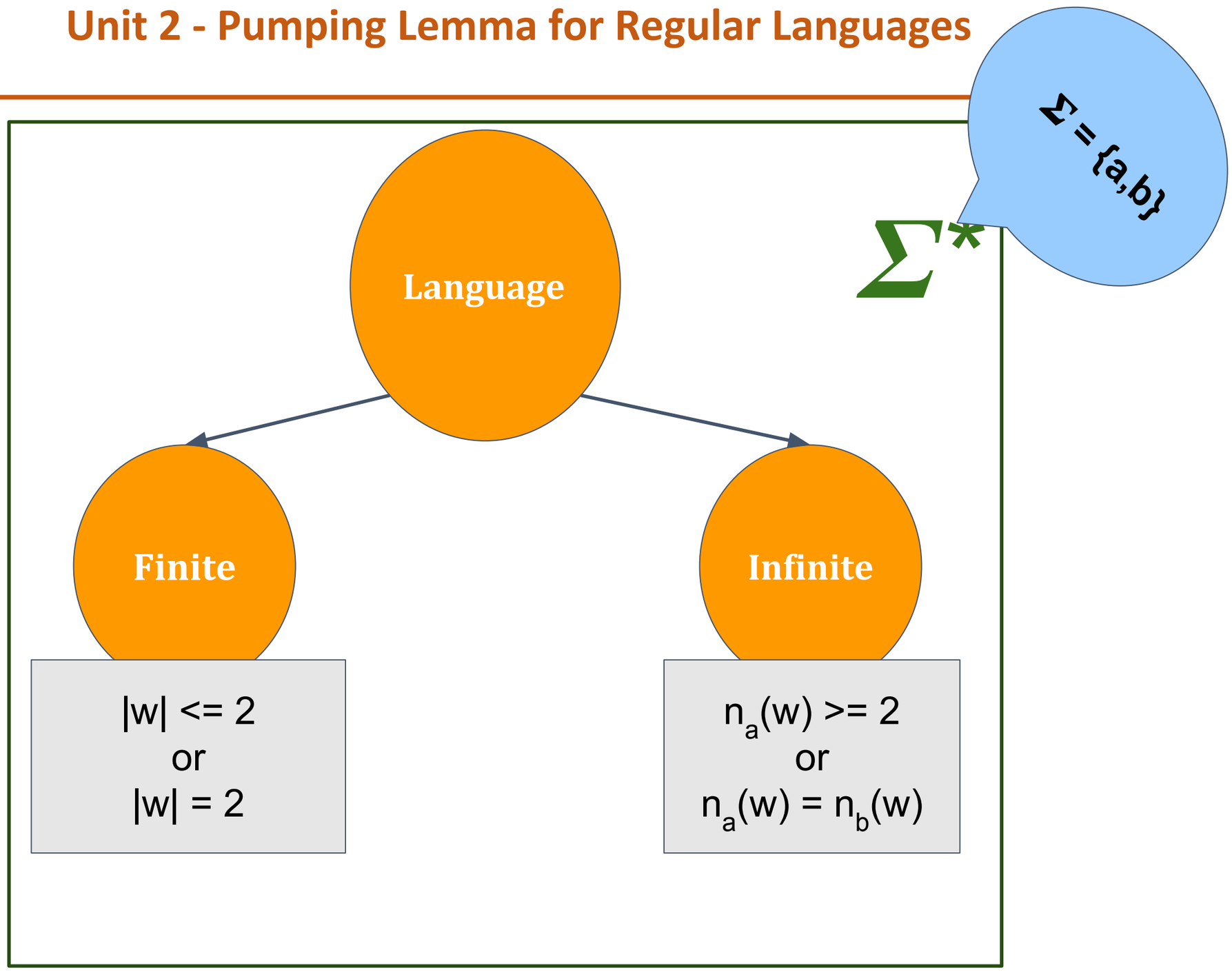
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages




Language
Finite
|w| <= 2
or
|w| = 2
Σ*
Σ = {a,b}


Σ = {a,b}
Σ*
Language
Finite
Infinite
|w| <= 2
or
|w| = 2

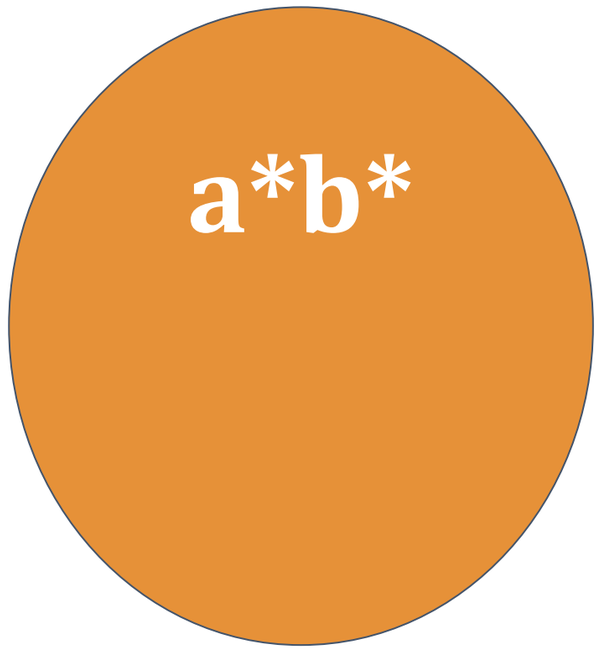

Σ = {a,b}
Σ*
Language
Finite
Infinite
|w| <= 2
or
|w| = 2
n_a(w) >= 2
or
n_a(w) = n_b(w)

PES UNIVERSITY ONLINE

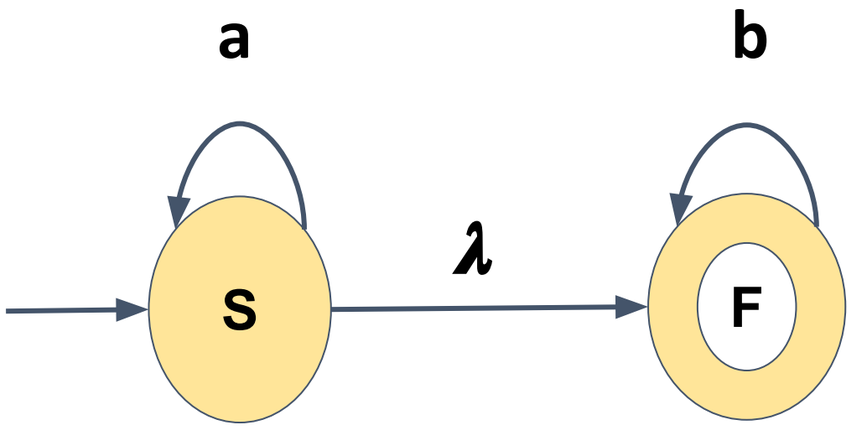
Is there any infinite language for which we cannot construct a Finite Automata?

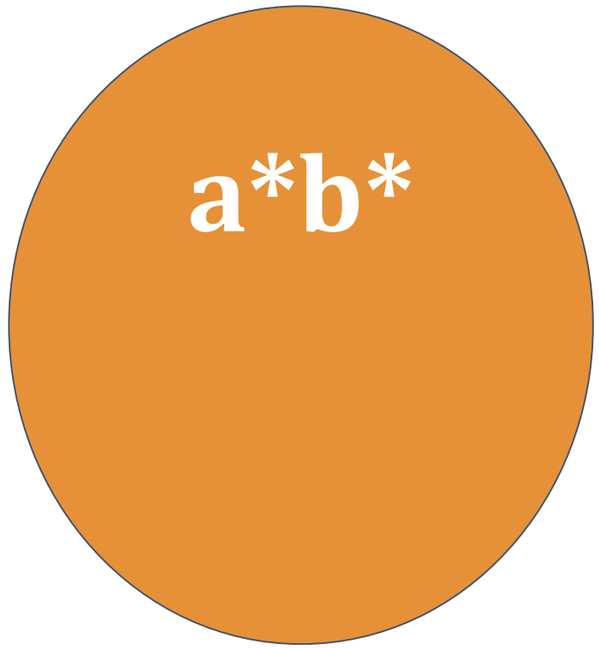
That means, a language which is not regular?

Let's look at an example :

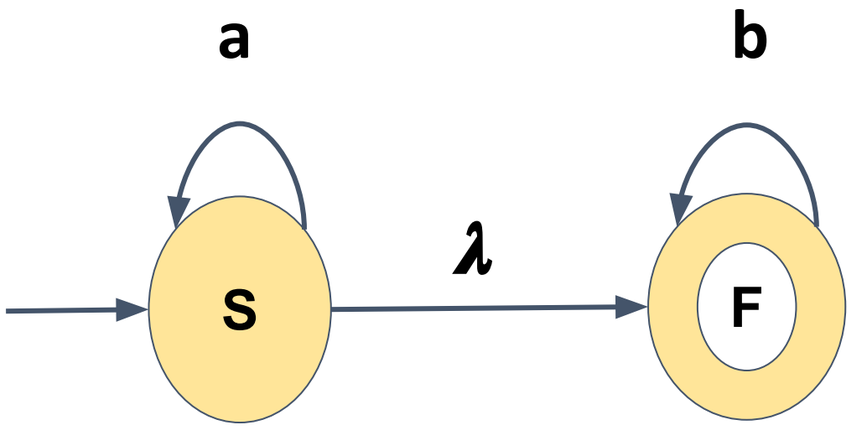


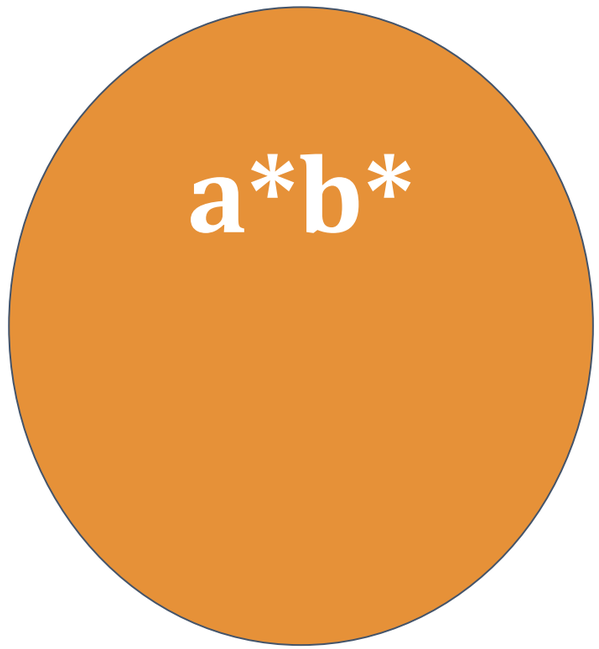
Let's look at an example :





Let's look at an example :

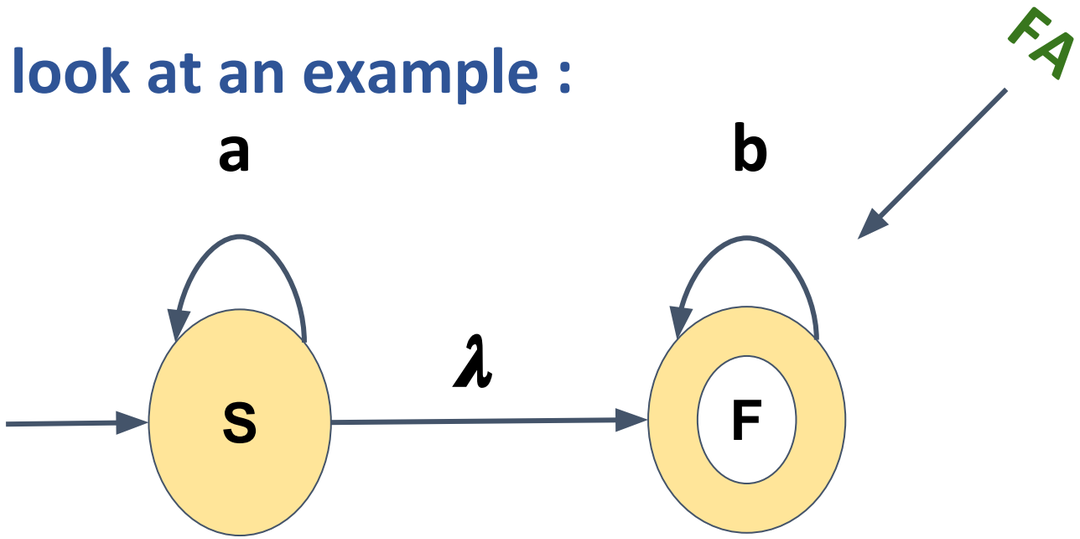


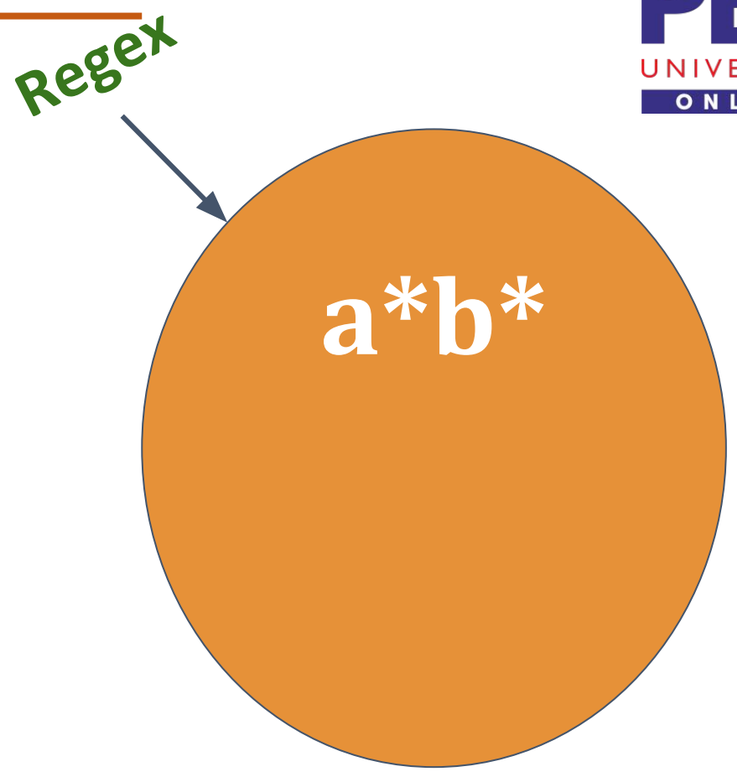


S -> aS | F

F -> bF | λ

Let's look at an example :



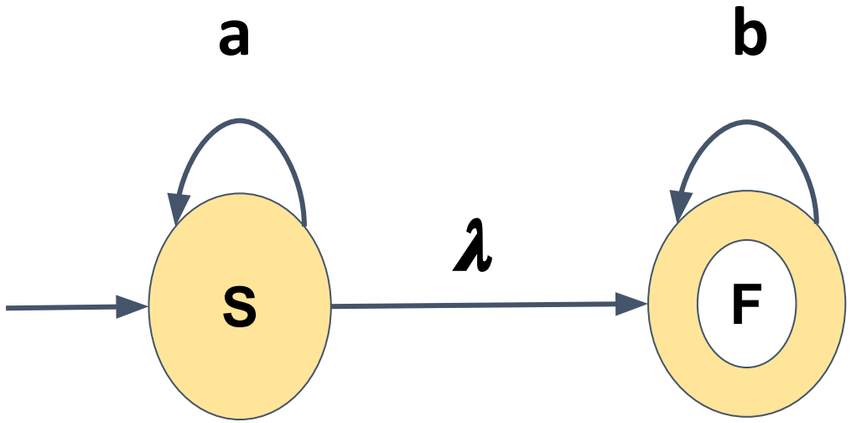


S -> aS | F

F -> bF | λ

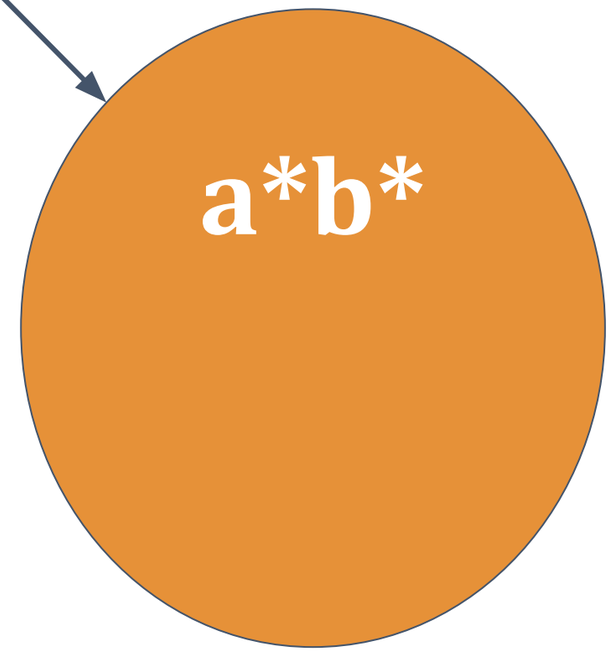
Regular Grammar

Let's look at an example :



FA

Regex



Definitely Regular !

S -> aS | F

F -> bF | λ

Regular Grammar

PES UNIVERSITY ONLINE

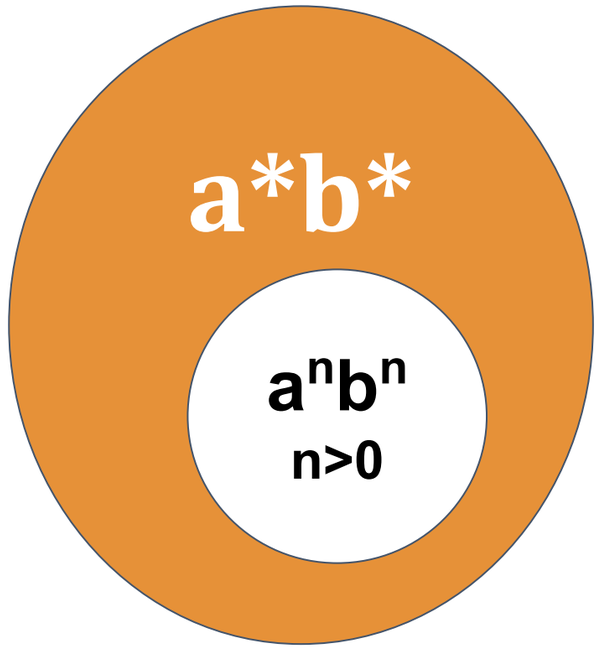
For the language $a^n b^n$ Can we construct either of :

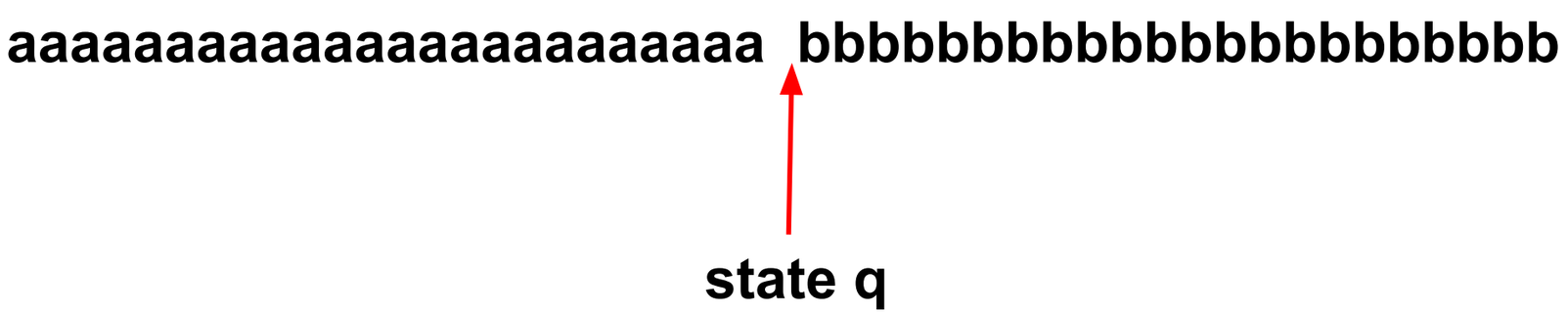
Finite Acceptor or

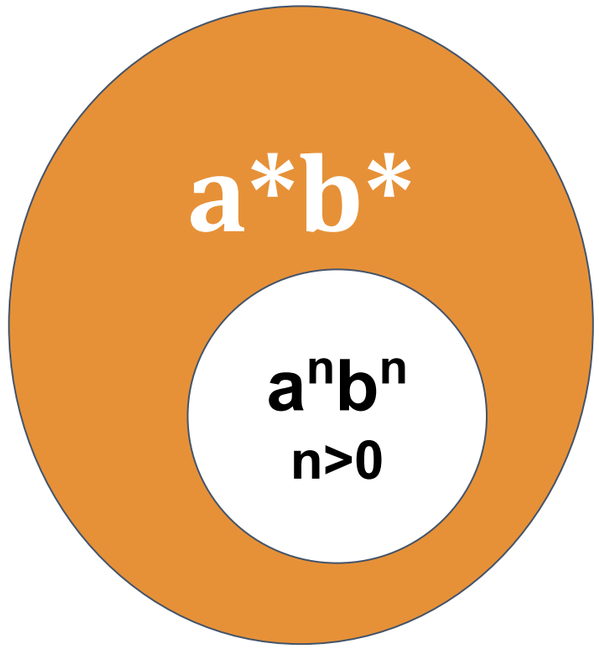
Regular Grammar or

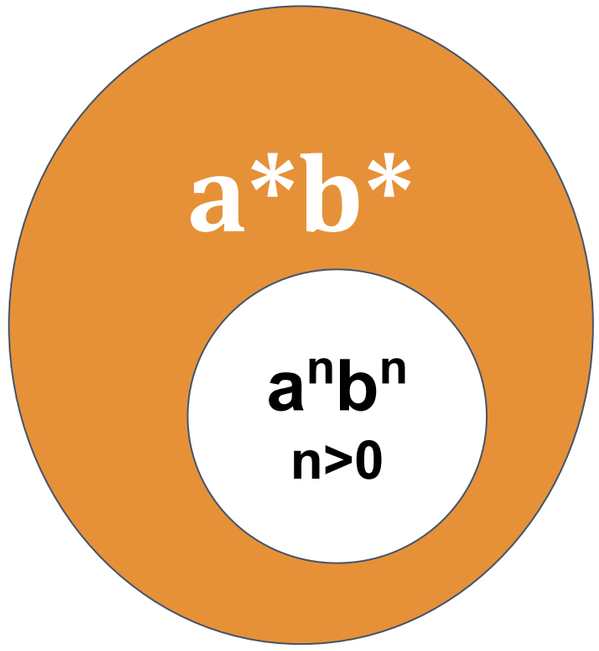
Regular Expression

????????????




aaaaaaaaaaaaaaaaaaaaaaaaa bbbbbbbbbbbbbbbbbbbbbbbbb
state q


a*b*
$a^n b^n$
n>0



Basically,

There is no way to remember how many a's you have seen to compare with the upcoming b's !

The value of n could be anything!

We cannot come up with a FA that takes care of all n!

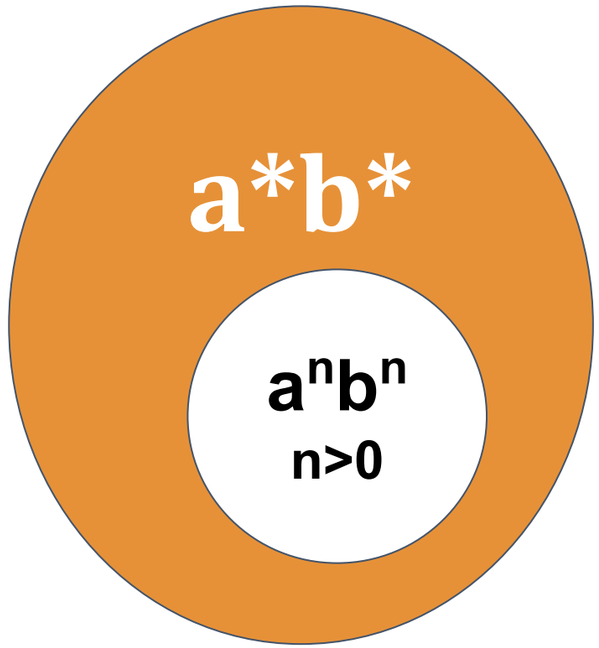
Basically,

There is no way to remember how many you have seen to compare with!

Definitely Non-Regular !

The value of n could be anything!

We cannot come up with a FA that takes care of all n!

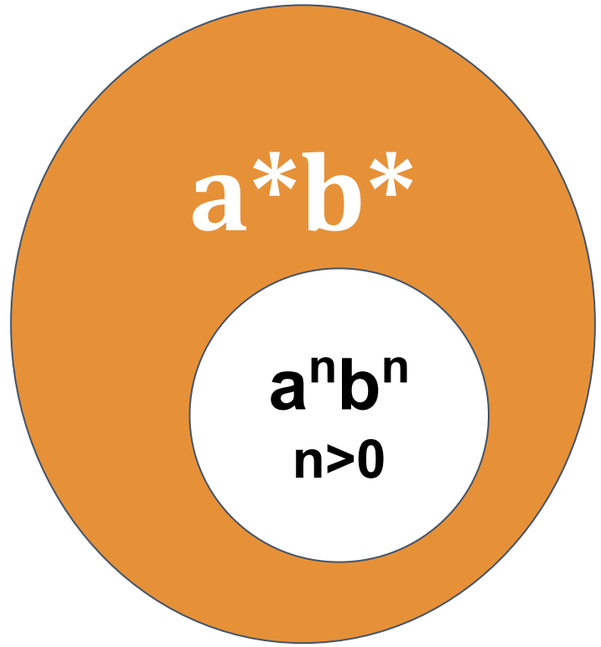


Basically,

There is no way to re… you have seen to compa…

The value … can co…

We cannot come up w…h a FA that takes care of all n!



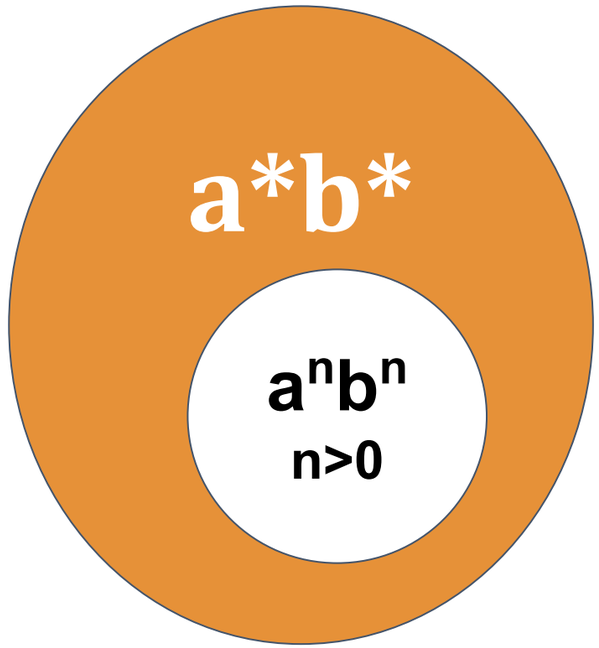
Limits of Finite Automata :

- **Finite Memory !**

- **String comparison not possible - A finite automata can only "count" ; that is, It can maintain a counter, where different states correspond to different values of the counter.**

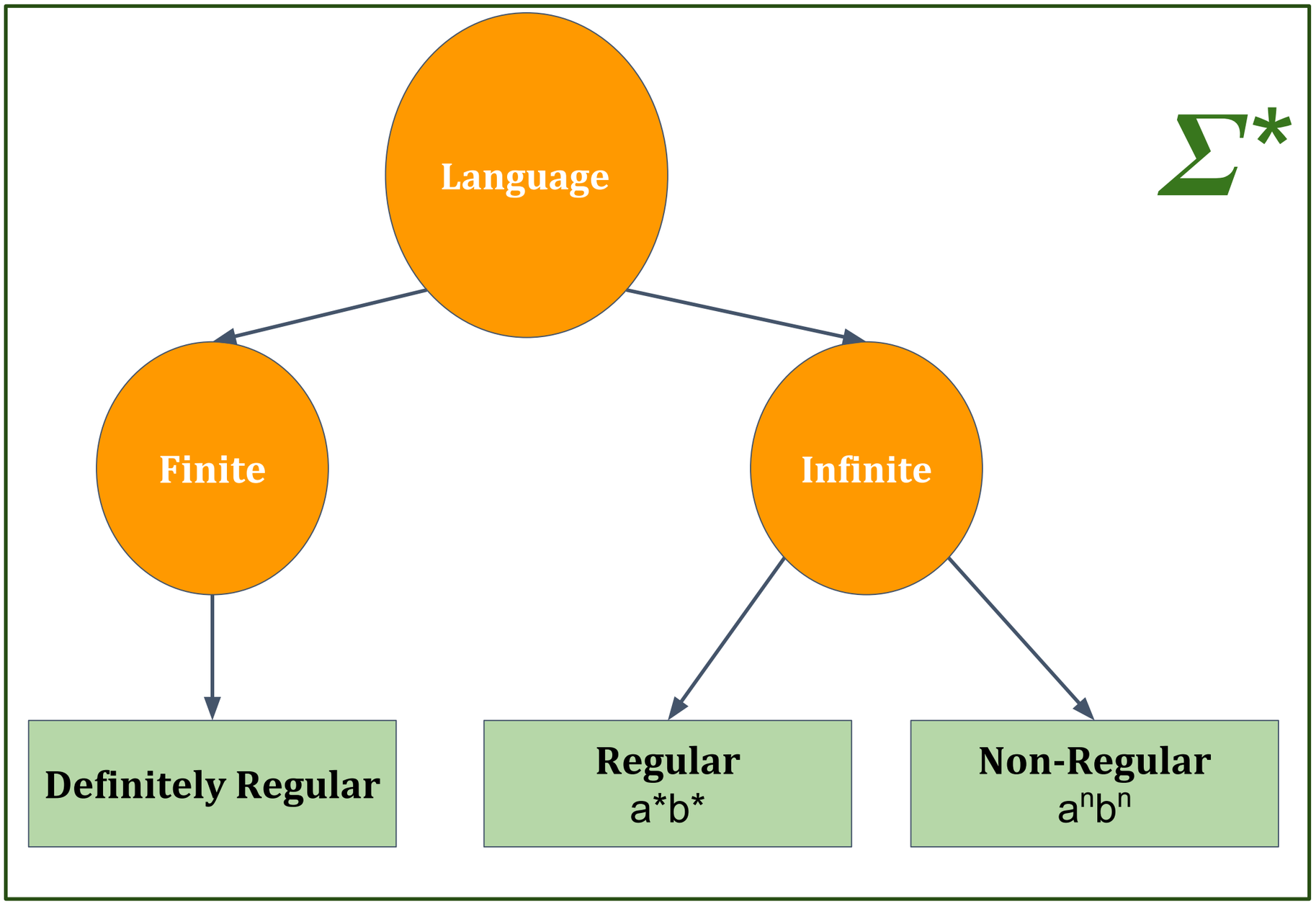
- **Linear Power :**

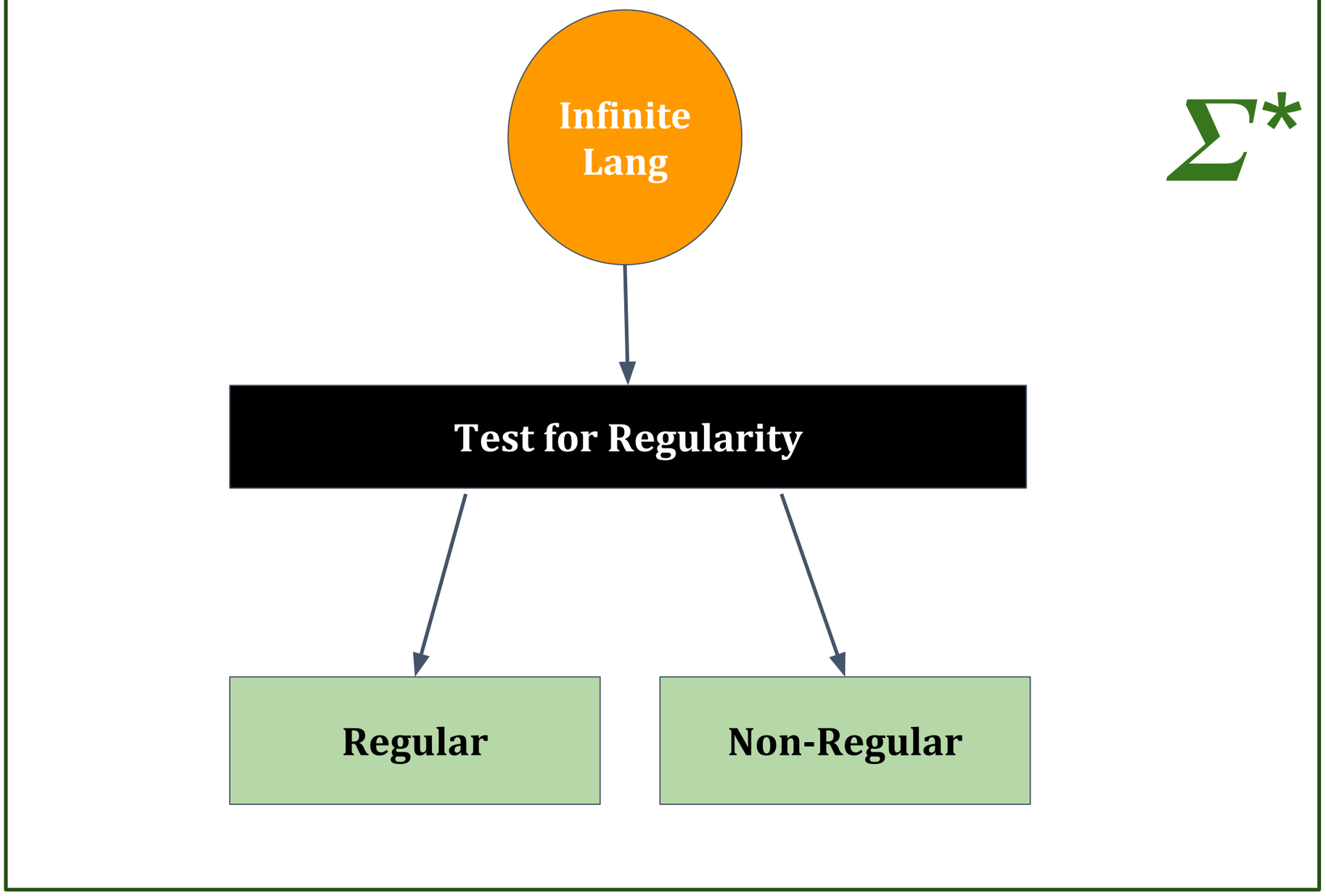
  an n>0 is possible but $a^{n\hat{}2}$, $a^{2\hat{}n}$, a^{i} where i is prime is not



Automata Formal Languages and Logic

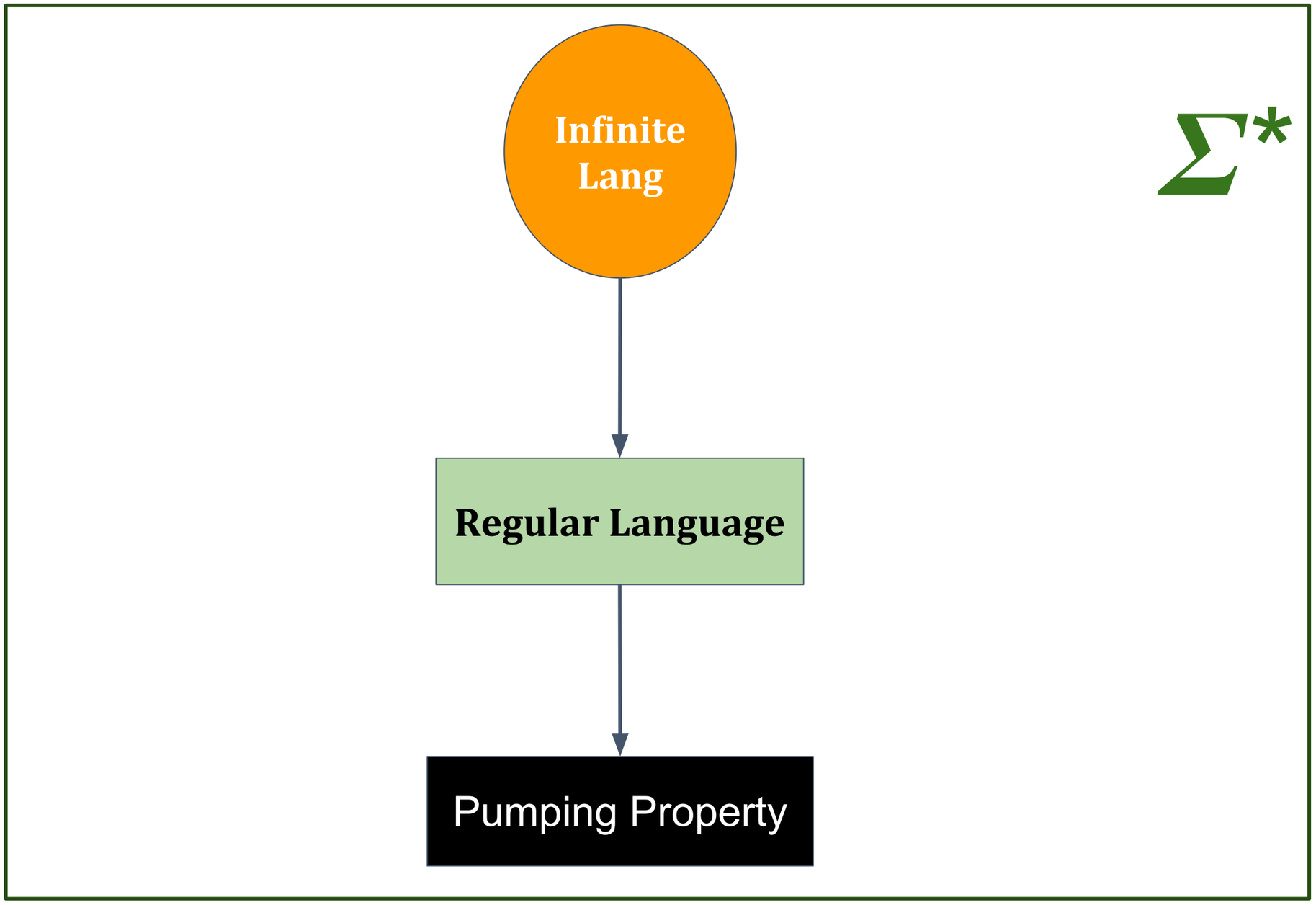
Unit 2 - Pumping Lemma for Regular Languages




Infinite Lang
Σ*
Test for Regularity
Regular
Non-Regular

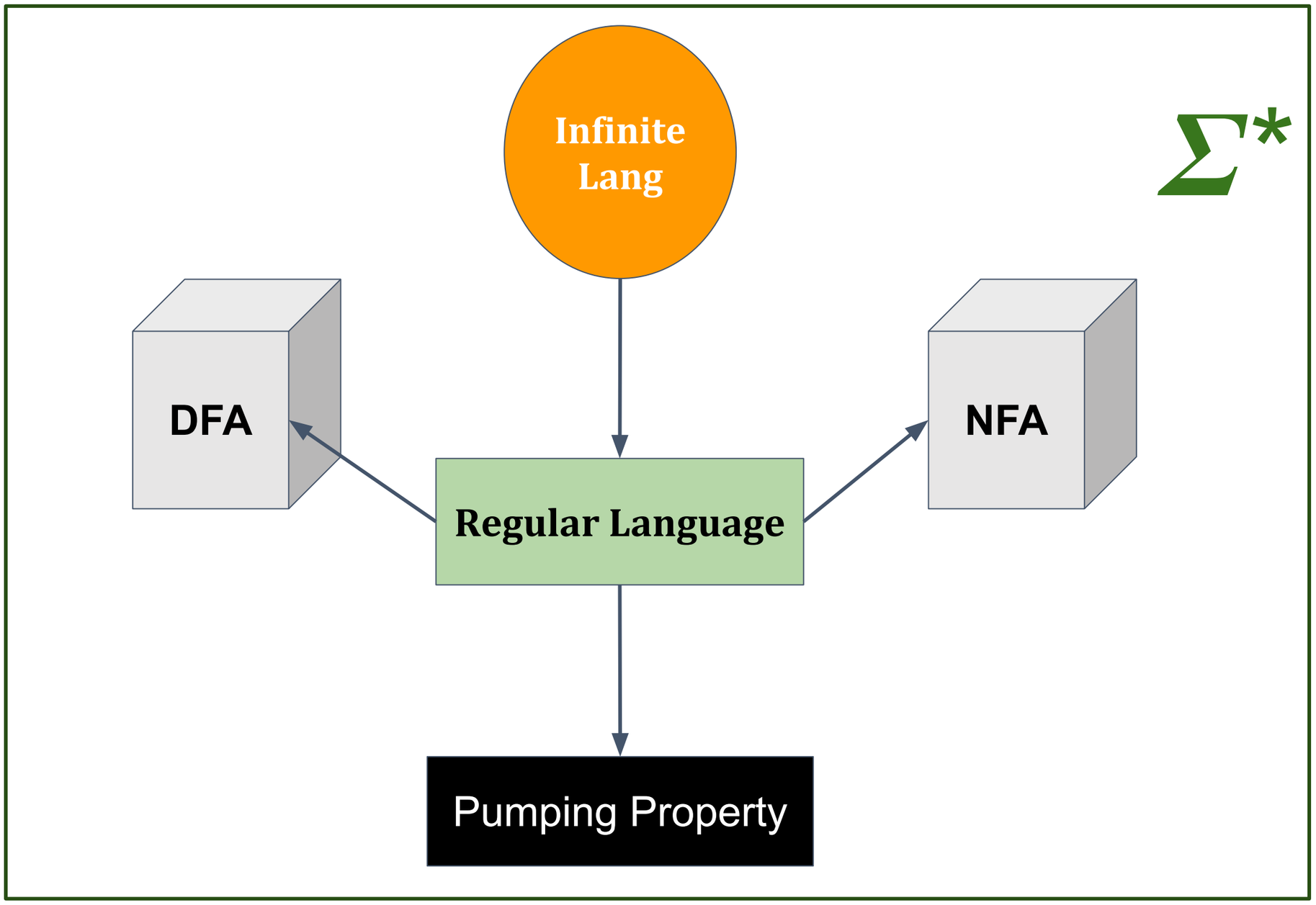
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

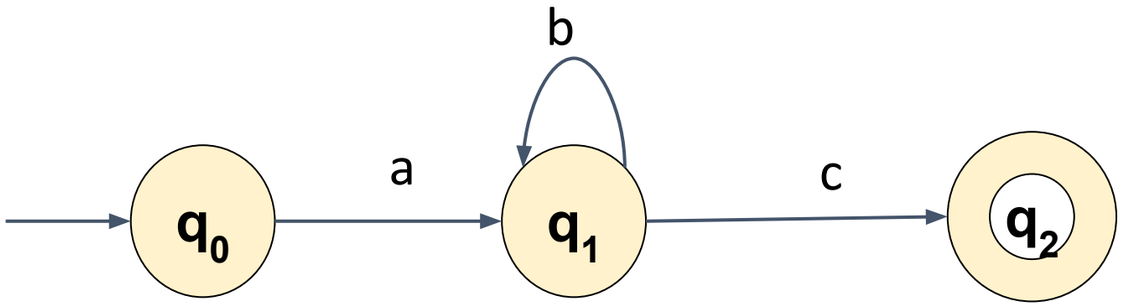


Automata Formal Languages and Logic

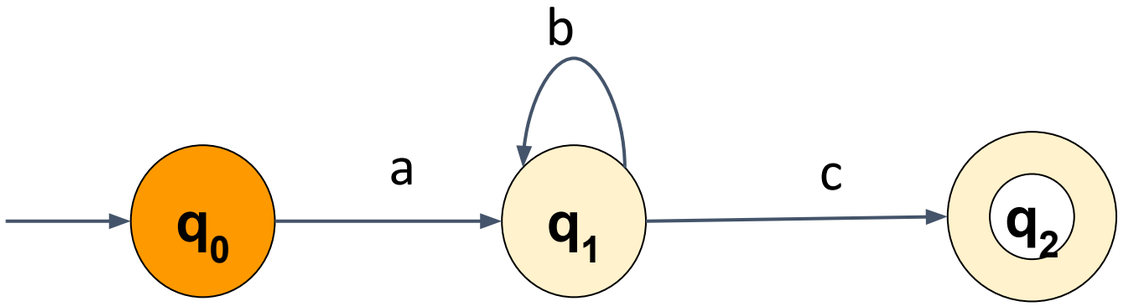
Unit 2 - Pumping Lemma for Regular Languages



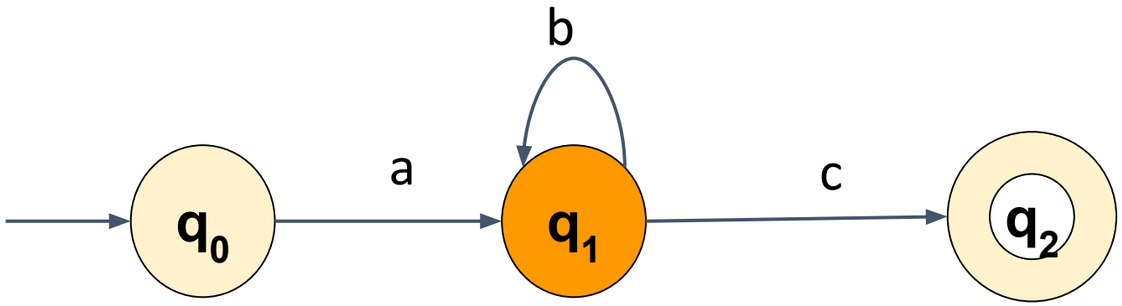
Let’s take an example of infinite regular language ab*c

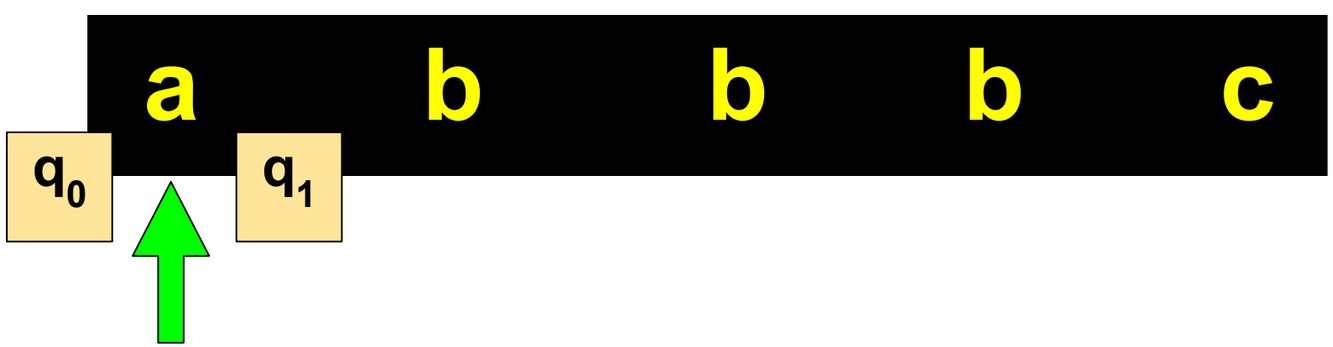


Let’s take an example of infinite regular language ab*c

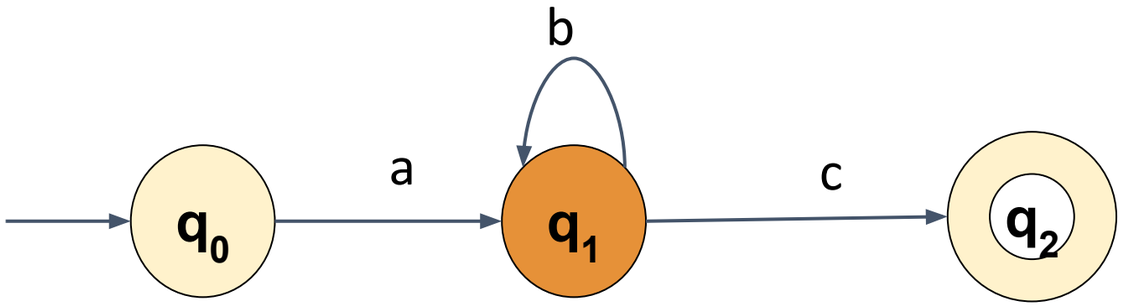


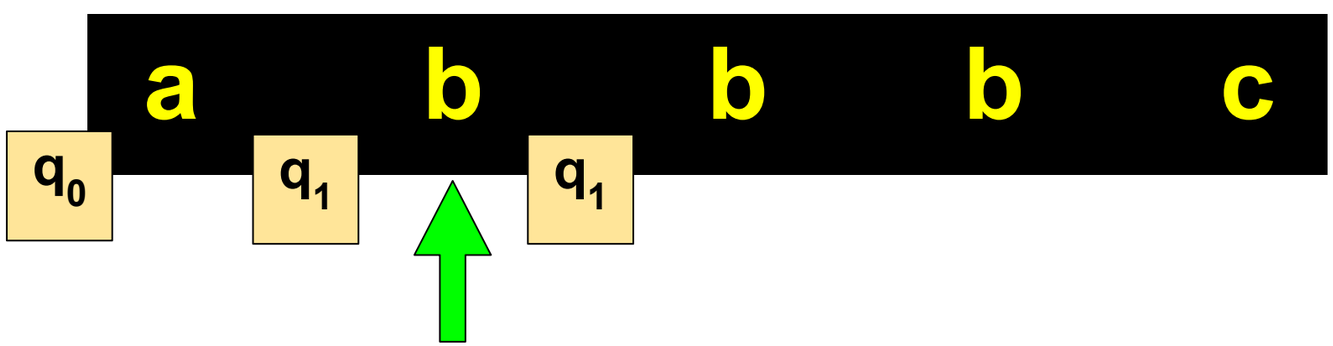
Let’s take an example of infinite regular language ab*c



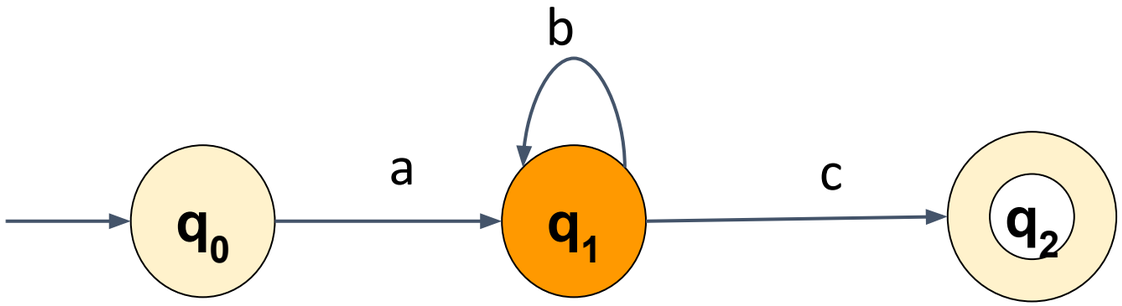


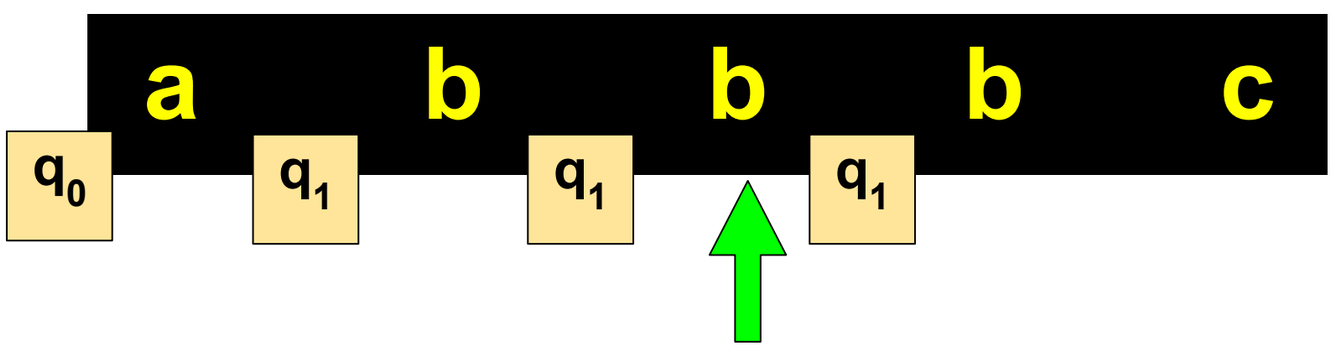
Let’s take an example of infinite regular language ab*c



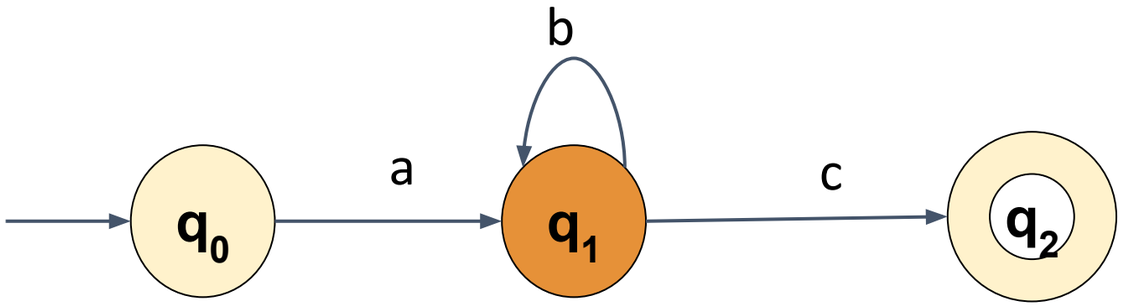


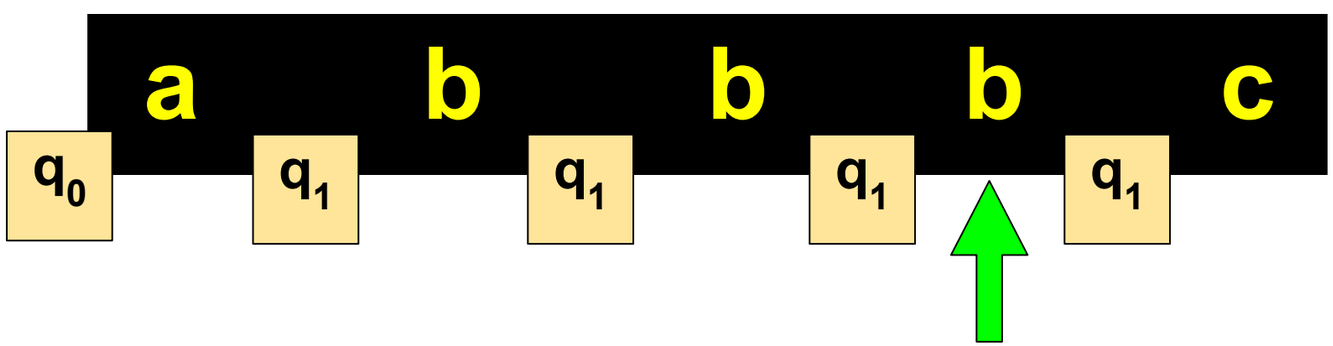
Let’s take an example of infinite regular language ab*c



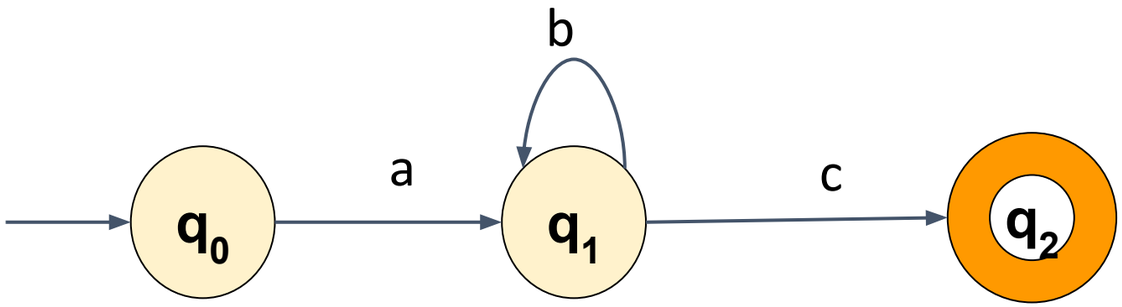


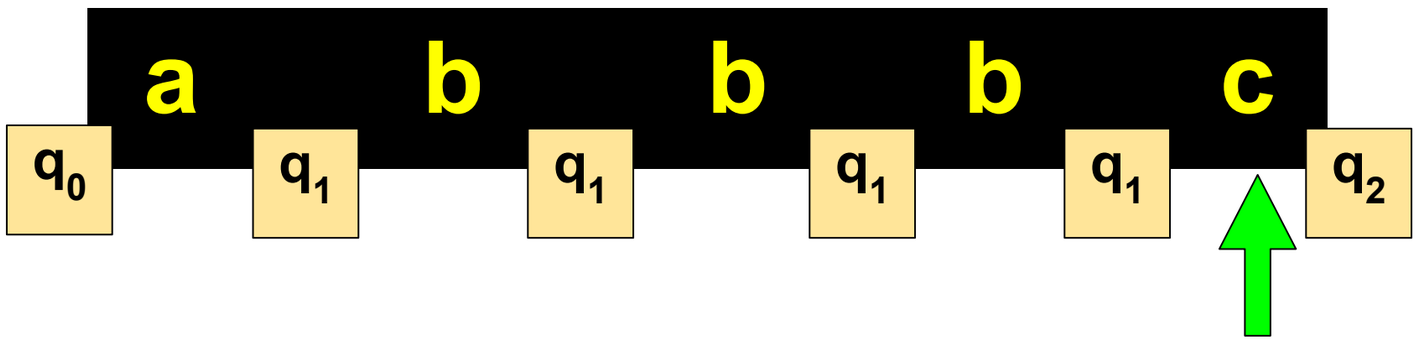
Let’s take an example of infinite regular language ab*c





Let’s take an example of infinite regular language ab*c





if |w| >= n

we visit a set of states more than once

if |w| >= n

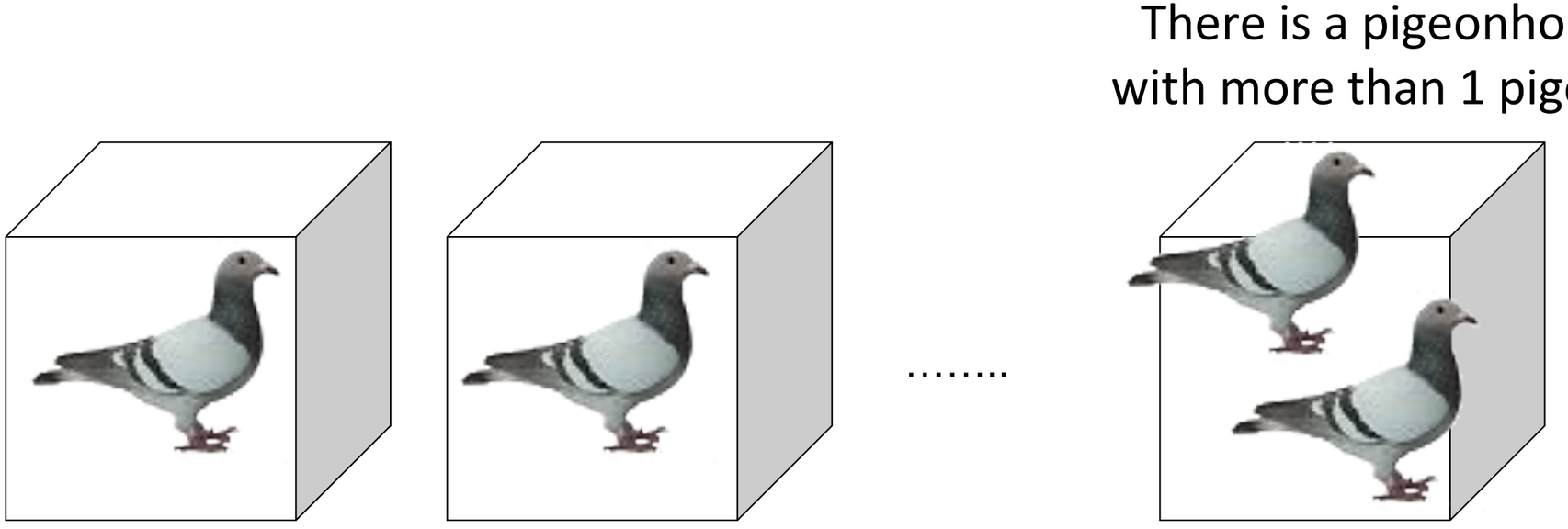
we visit a set of states more than once

The Pigeonhole Principle

The Pigeonhole Principle

$$n > m$$

pigeons pigeonholes



The Pigeonhole Principle

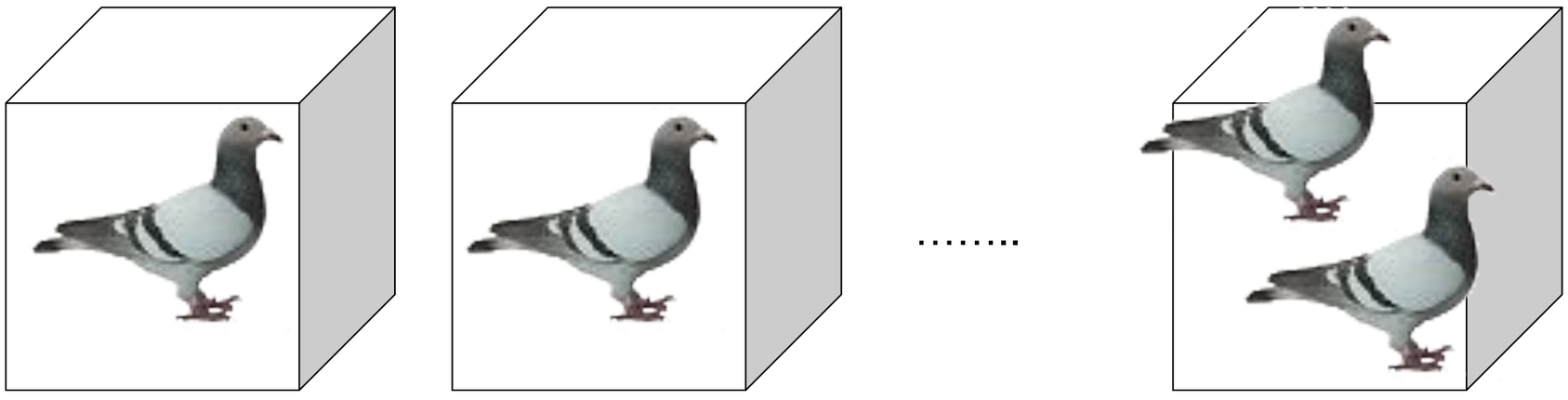
$$n > m$$

pigeons — **String**

pigeonholes — **# States n**

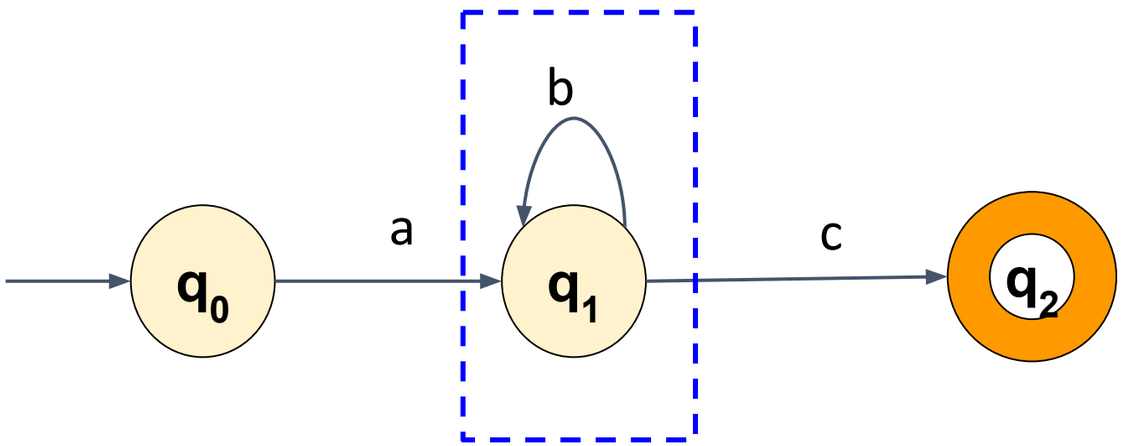
if |w| > n
A set of states will be visited more than once

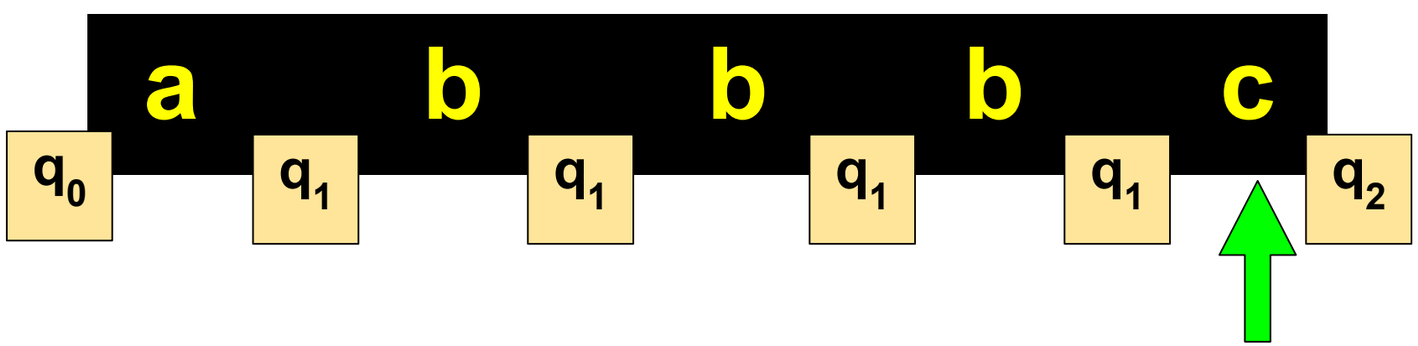
There is a pigeonhole with more than 1 pigeon



PES UNIVERSITY ONLINE

Let's take an example of infinite regular language ab*c





if |w| >= n

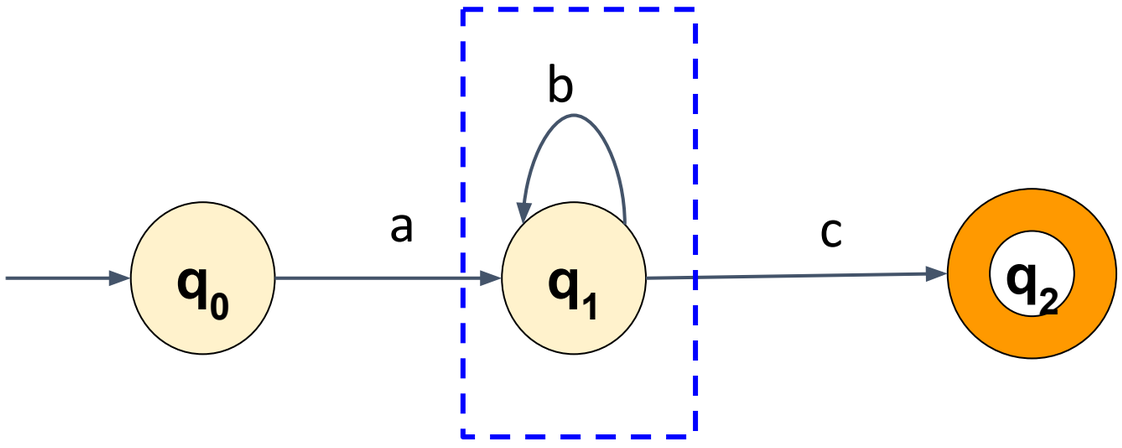
we visit a set of states more than once which means,

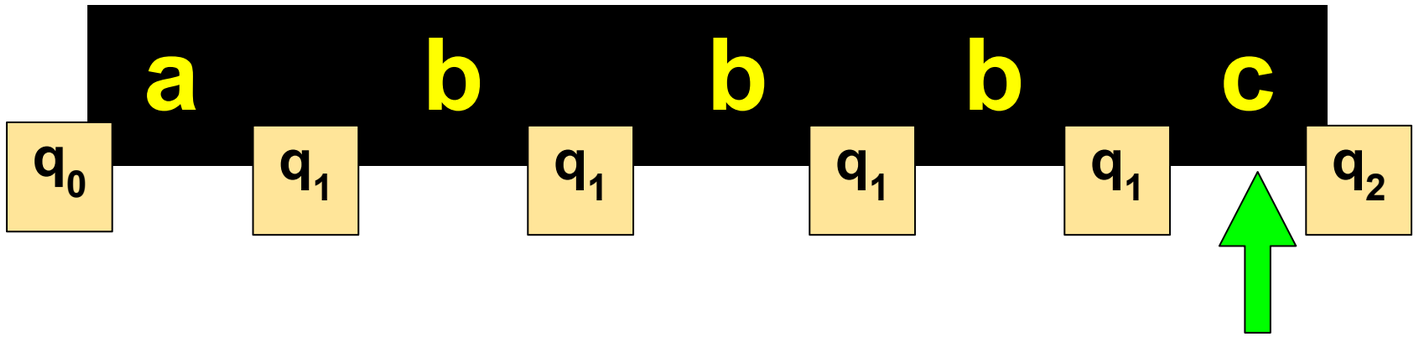
there exists a loop in our Automata (within these n states)

if we pump that loop 0 or more no. of times,

the resultant string will always be in the language

Let’s take an example of infinite regular language ab*c





There exists 3 parts to a string w:

w = x y z

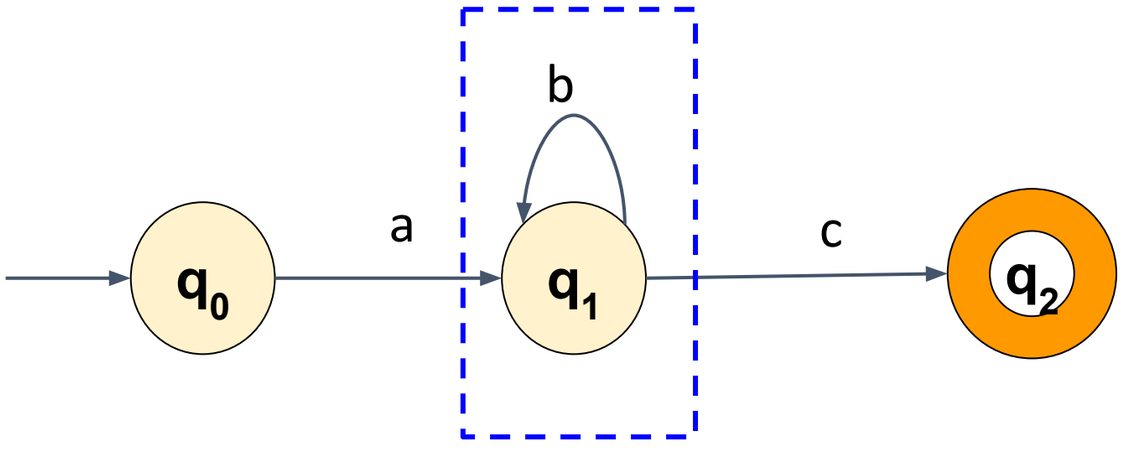
Part before the loop

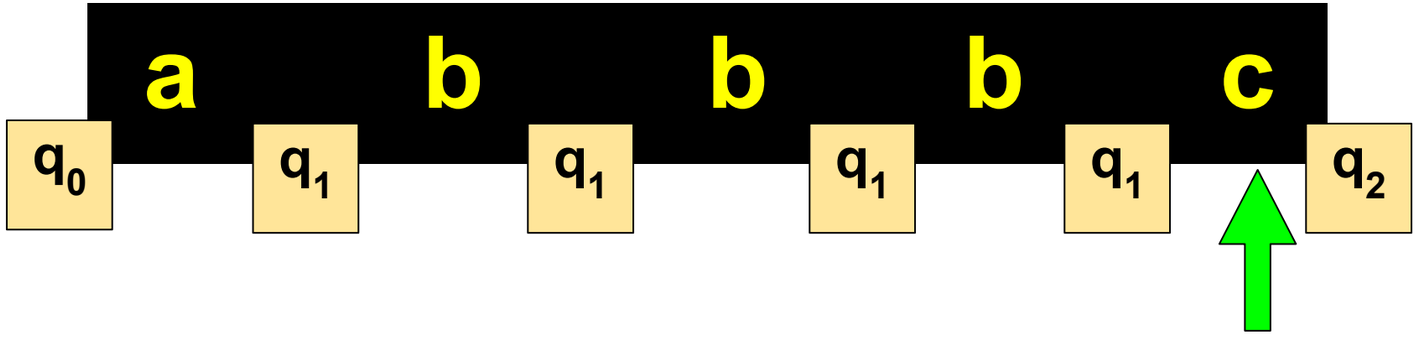
The loop

Part after the loop

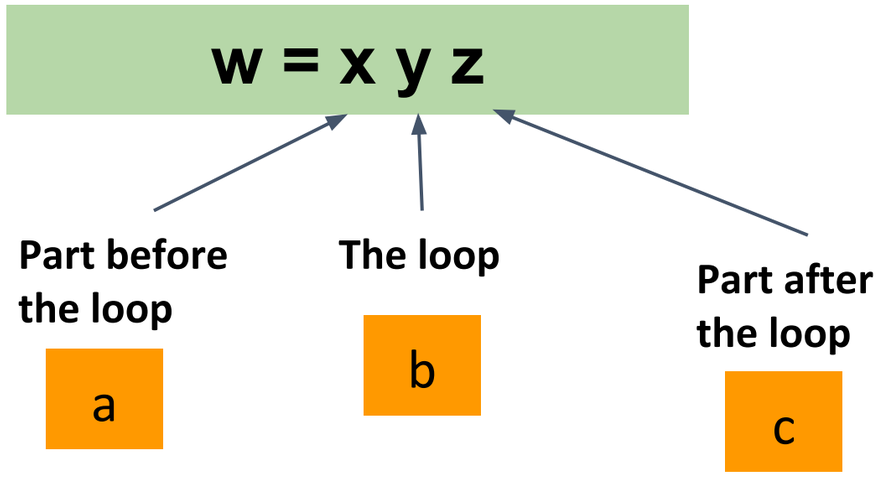
PES UNIVERSITY ONLINE

Let’s take an example of infinite regular language ab*c



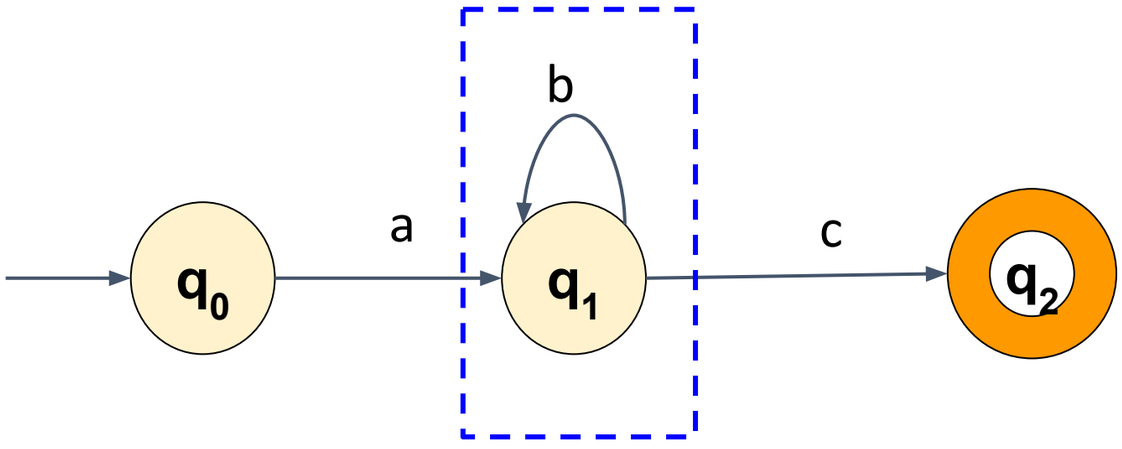


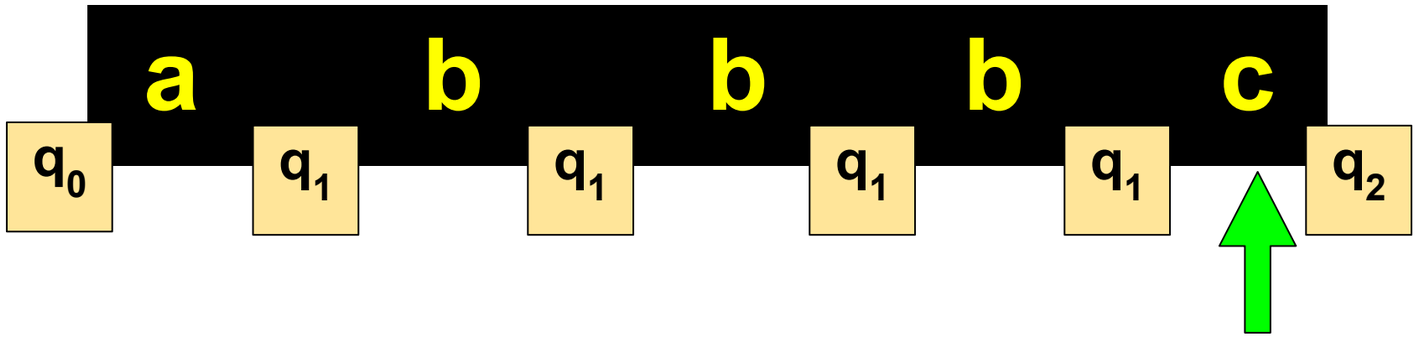
There exists 3 parts to a string w:



PES UNIVERSITY ONLINE

Let’s take an example of infinite regular language ab*c





There exists 3 parts to a string w:

w = x y z

Part before the loop

a

The loop

b

Part after the loop

c

$y \neq \epsilon$ that is $|y| >= 1$

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

In our example of infinite regular language ab*c

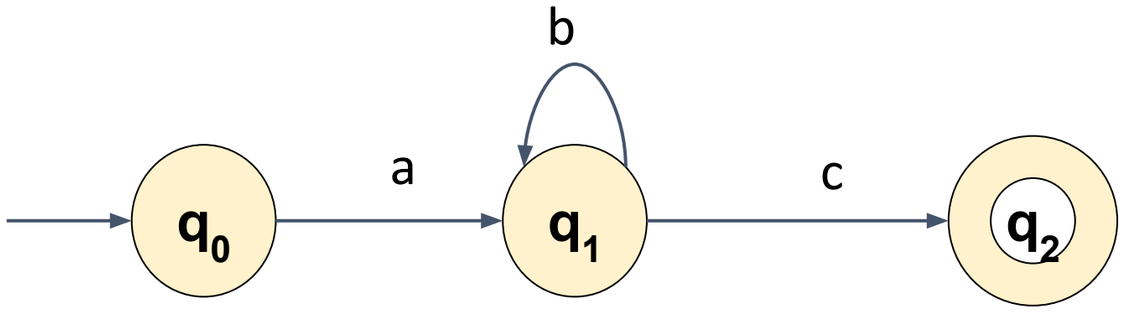
The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

n =3

In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

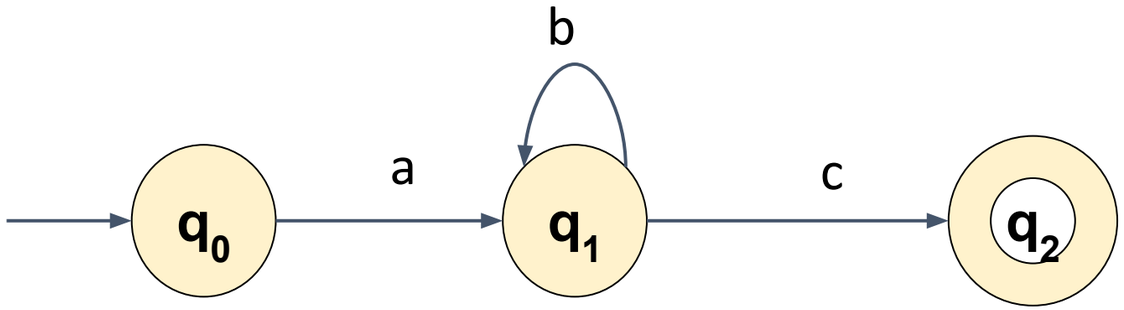
For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

n =3

n is also called Pumping Constant

In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

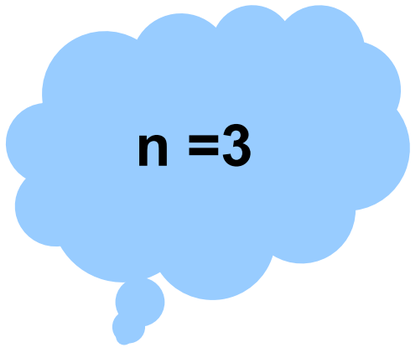
The Pumping property States,

For every Regular language L, (infinite)

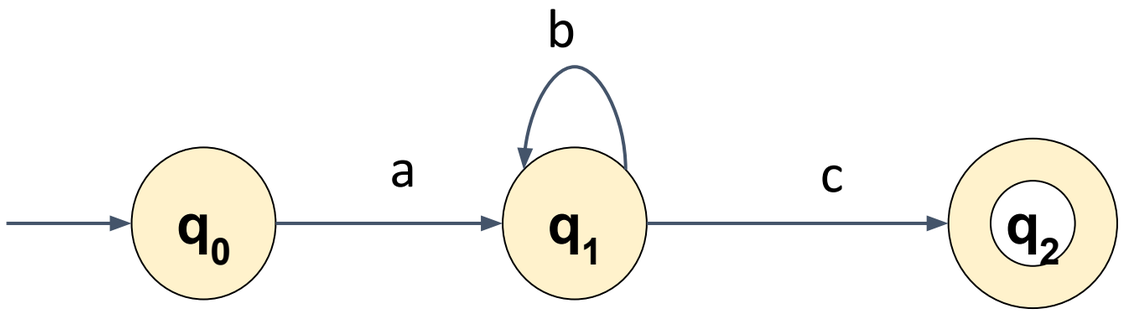
there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

|w| >= n



In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

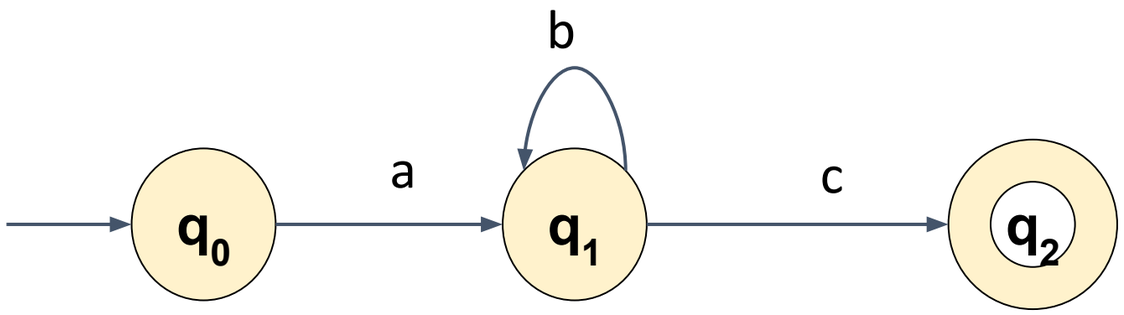
For every string w that belongs to L such that,

|w| >= n

w = abbbc
|w| = 5 > n

n =3

In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

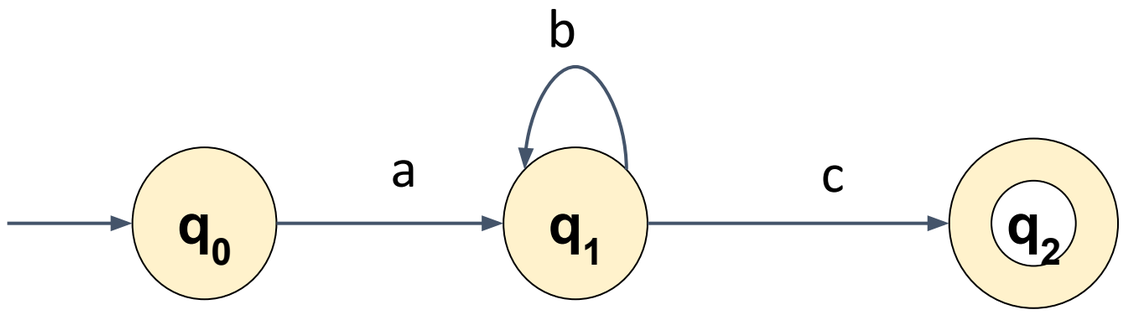
|w| >= n

There exists a break up of the string in three parts w = xyz such that |y| >=1 and |xy| <= n

w = abbbc
|w| = 5 > n

n =3

In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

|w| >= n

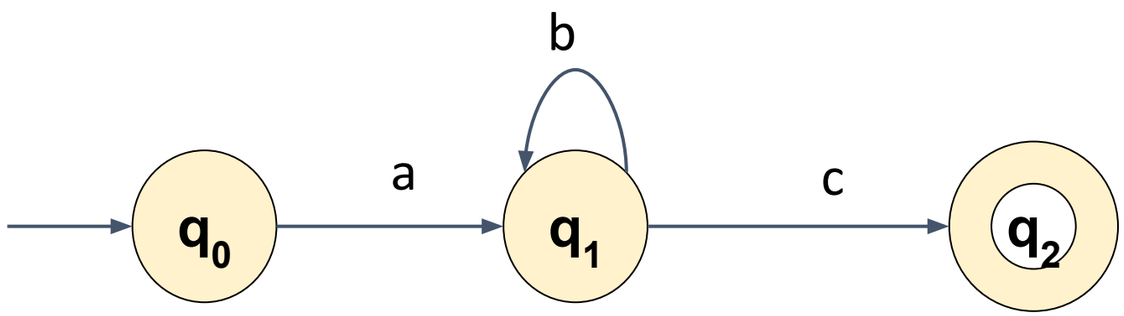
There exists a break up of the string in three parts w = xyz such that |y| >=1 and |xy| <= n

w = abbbc
|w| = 5 > n

n =3

w = abc
x = a
y = b
z = c

In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

$$|w| >= n$$

There exists a break up of the string in three parts w = xyz such that |y| >=1 and |xy|<=n,

for every i >= 0,

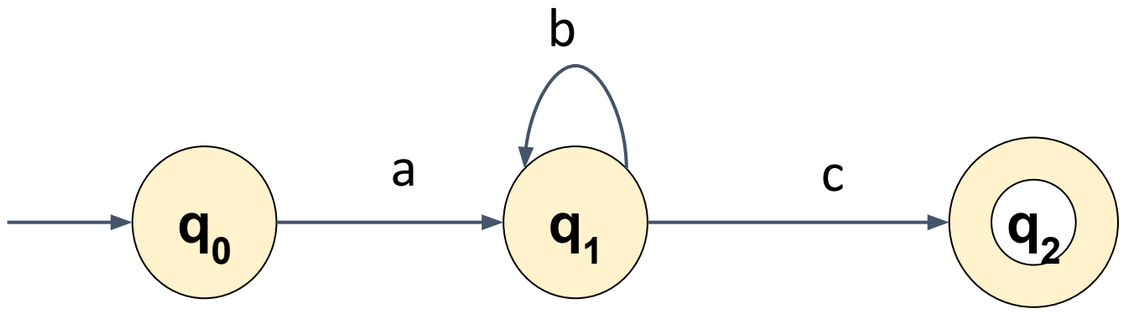
$$xy^iz \text{ belongs to } L$$

w = abbbc
|w| = 5 > n

n =3

w = abc
x = a
y = b
z = c

In our example of infinite regular language ab*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, (infinite)

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

$$|w| >= n$$

There exists a break up of the string in three parts w = xyz such that |y| >=1 and |xy|<=n,
for every i >= 0,

$$xy^iz \text{ belongs to } L$$

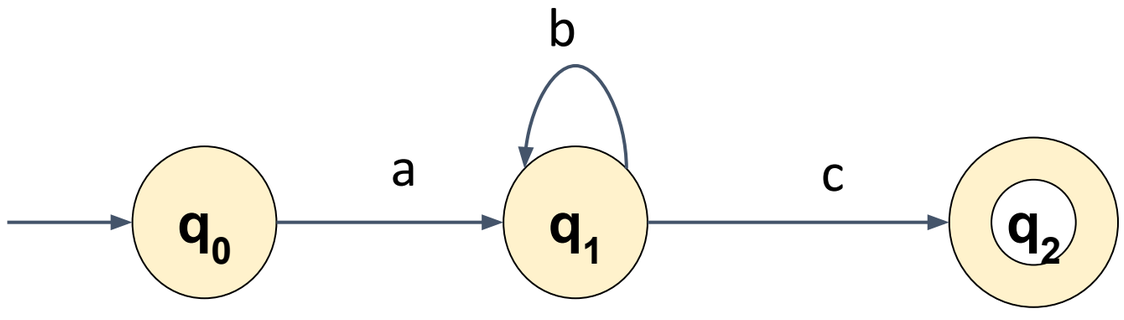
w = abbbc
|w| = 5 > n

n =3

w = abc
x = a
y = b
z = c

for i>=0,
ab^ic is in lang ab*c

In our example of infinite regular language ab*c



PES UNIVERSITY ONLINE

For Regular Languages (infinite)

Pumping Property

For every Regular language L,

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

$$|w| >= n$$

There exists a break up of the string in three parts w = xyz such that |y| >= 1 and |xy|<=n,

for every i >= 0,

$$xy^iz \text{ belongs to } L$$

PES UNIVERSITY ONLINE

For Regular Languages (infinite)

Pumping Property

For every Regular language L,

there exists n where n is the # states in Finite Automata for L

For every string w that **belongs to** L such that,

$$|w| >= n$$

There exists a break up of the string in three parts w = xyz such that |y| >= 1 and |xy|<=n,

for every i >= 0,

$$xy^iz \text{ belongs to } L$$

Replace
For every ----> ∀
There exists ----> ∃
belongs to ----> ∈

PES UNIVERSITY ONLINE

For Regular Languages (infinite)

Pumping Property

∀ Regular language L,

∃ n where n is the # states in Finite Automata for L

∀ string w ∈ L such that,

$|w| >= n$

∃ w = xyz such that |y| >= 1 and |xy|<=n,

∀ i >= 0,

$xy^iz \in L$

Replace
For every ----> ∀
There exists ----> ∃
belongs to ----> ∈

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

∀ Regular language L,

∃ n where n is the # states in Finite Automata for L

∀ string w ∈ L such that,

$|w| >= n$

∃ w = xyz such that |y| >= 1 and |xy|<=n,

∀ i >= 0,

$xy^iz \in L$

To Prove a lang is Non-Regular

~Pumping Property

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

∀ Regular language L,

∃ n where n is the # states in Finite Automata for L

∀ string w ∈ L such that,

$|w| >= n$

∃ w = xyz such that $|y| >= 1$ and $|xy| <= n$,

∀ $i >= 0$,

$xy^iz \in L$

To Prove a lang is Non-Regular

~Pumping Property

~ ∀ = ∃

~ ∃ = ∀

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

∀ Regular language L,

∃ n where n is the # states in Finite Automata for L

∀ string w ∈ L such that,

$|w| >= n$

∃ w = xyz such that |y| >= 1 and |xy|<=n,

∀ i >= 0,

$xy^iz \in L$

~ ∀ = ∃

~ ∃ = ∀

To Prove a lang is Non-Regular

~Pumping Property

∃ a language L which is claimed to be regular,

∀ n where n is the # states in Finite Automata for L

∃ string w ∈ L such that,

$|w| >= n$

∀ w = xyz such that |y| >= 1 and |xy|<=n,

∃ i >= 0,

$xy^iz \notin L$

This contradicts the claim made, hence proving that the language is not regular

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

∀ Regular language L,

∃ n where n is the # states in Finite Automata for L

∀ string w ∈ L such that,

$|w| >= n$

∃ w = xyz such that |y| >= 1 and |xy|<=n,

∀ i >= 0,

$xy^iz \in L$

~ ∀ = ∃

~ ∃ = ∀

To Prove a lang is Non-Regular

~Pumping Property

∃ a language L ... be regular,

∀ n whe... ata for L

∃ str...

∀ w = ...

∃ i >= 0...

This contradic... proving that the language is not ...

Proof by Contradiction

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

PES UNIVERSITY ONLINE

For Regular Languages (infinite)

Pumping Property

∀ Regular language L,

∃ n where n is the # states
L

∀ string w ∈ L such that

|w| >= n

∃ w = xyz such that |y| >= 1 and |xy =n,

∀ i >= 0,

$xy^iz \in L$

Pumping Lemma

~ ∀ =

~ ∃ =

∀

To Prove a lang is Non-Regular

~Pumping Property

∃ a language L be regular,

∀ n whe ata for L

∃ str

∀ w =

∃ i >= 0

This contradic e proving that the language is not

Proof by Contradiction

PES UNIVERSITY ONLINE

Procedure to prove a language is Not regular :

1. Assume the opposite: L is regular

2. Use Pumping Lemma to obtain a contradiction

   It suffices to show that only one string gives a contradiction

3. Thereby proving L is not regular

Procedure to prove a language is Not regular :

1. Assume the opposite: L is regular

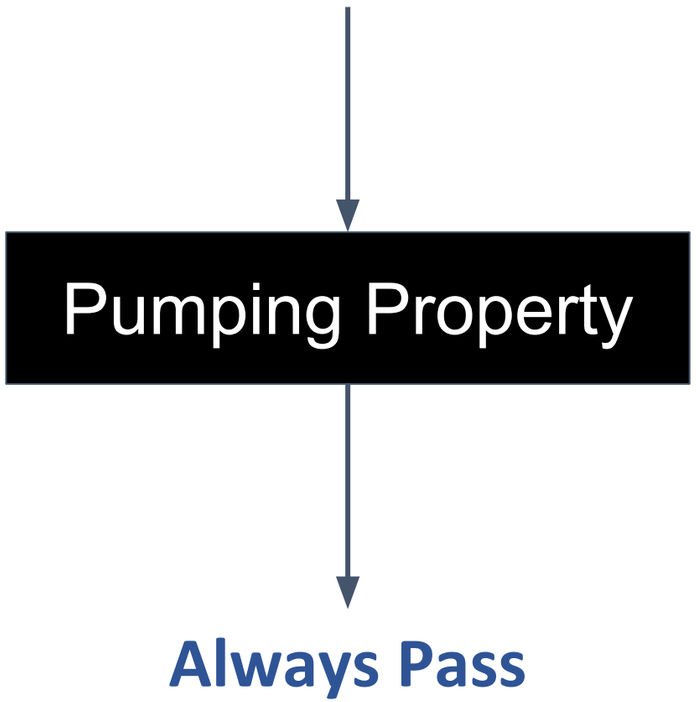
2. Use Pumping Lemma to obtain a contradiction

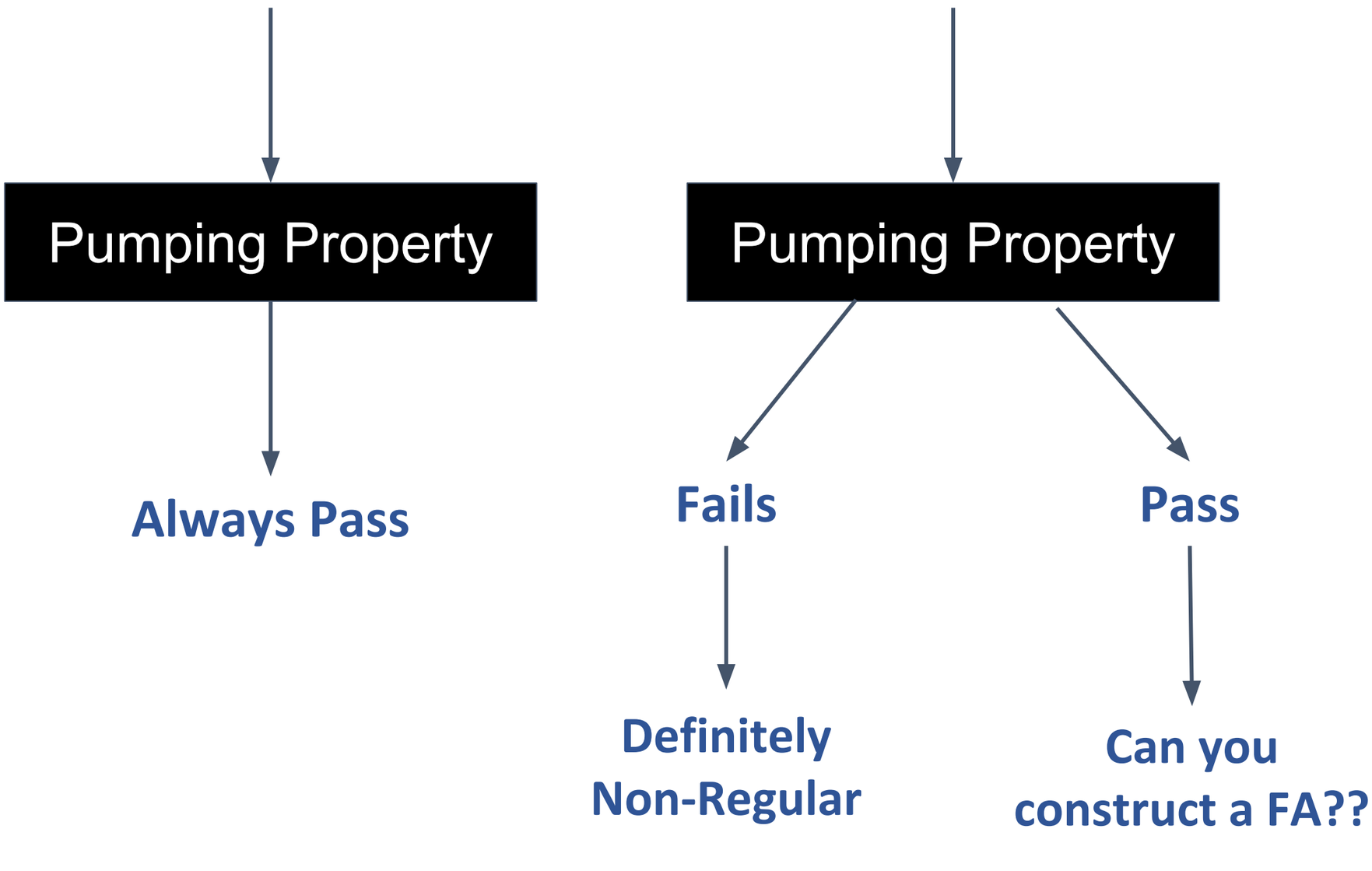
! String must be chosen appropriately

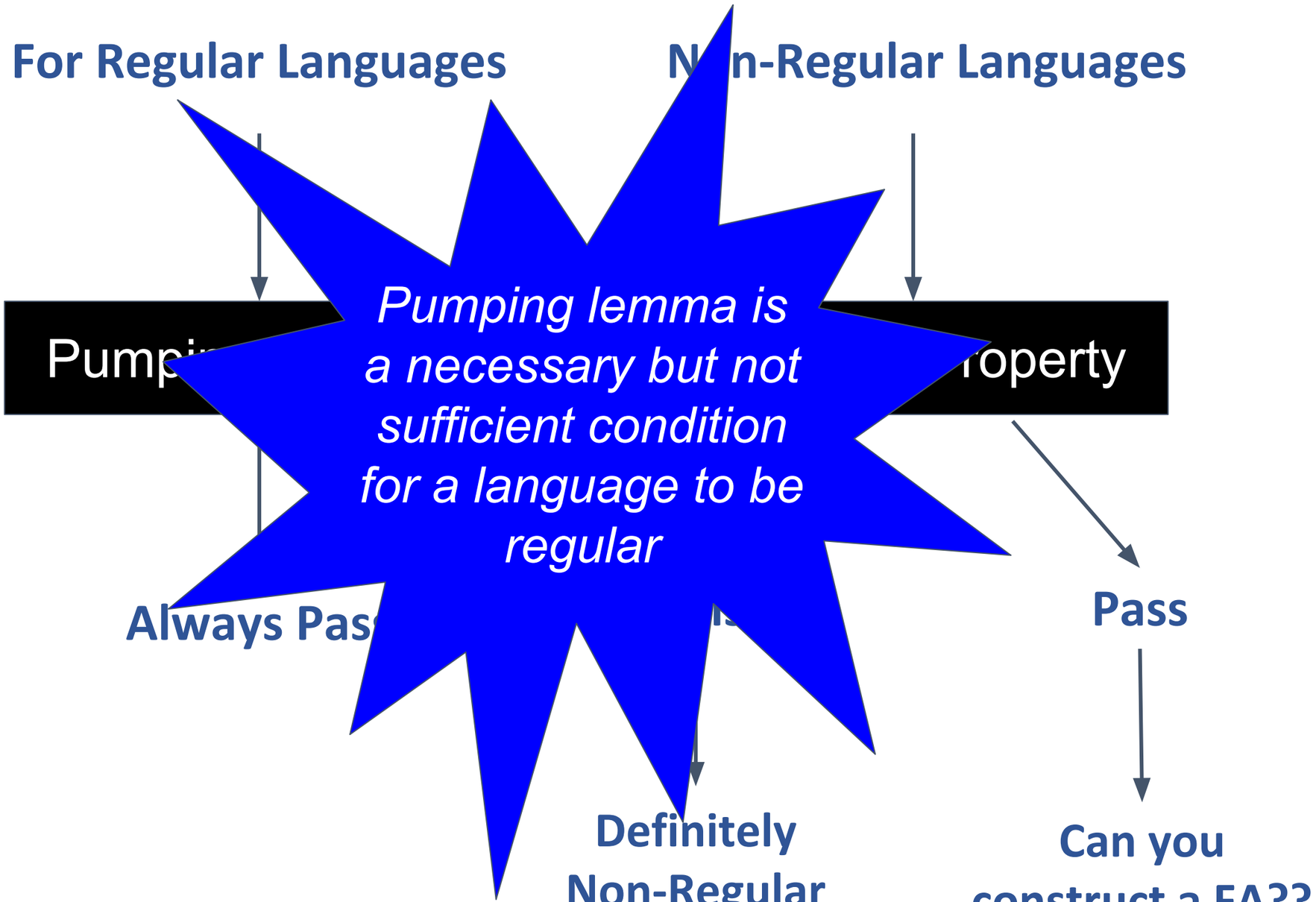
It suffices to show that only one *string* gives a contradiction

3. Thereby proving L is not regular

For Regular Languages




For Regular Languages
Non-Regular Languages
Pumping Property
Pumping Property
Always Pass
Fails
Pass
Definitely Non-Regular
Can you construct a FA??


For Regular Languages
Non-Regular Languages
Pumping lemma is a necessary but not sufficient condition for a language to be regular
Always Pass
Pass
Definitely Non-Regular
Can you construct a FA??

Pumping lemma as a game

PES UNIVERSITY ONLINE

Pumping lemma is a game between

You vs. Adversary

PES UNIVERSITY ONLINE

The role

You

Adversary

Claims L is regular

PES UNIVERSITY ONLINE

The role

You

Okay! Gimme the no. of states in your machine for L

Adversary

Claims L is regular

The role

You

Okay! Gimme the no. of states in your machine for L

Adversary

Claims L is regular

There are n states in my automata for L

The role

You

Okay! Gimme the no. of states in your machine for L

Okay! here is the string w from L such that
|w| >= n
Could you tell me where is the loop in your machine?

Adversary

Claims L is regular

There are n states in my automata for L

PES UNIVERSITY ONLINE

You

Okay! Gimme the no. of states in your machine for L

Okay! here is the string w from L such that
|w| >= n
Could you tell me where is the loop in your machine?

The role

Adversary

Claims L is regular

There are n states in my automata for L

The loop is xy^iz

PES UNIVERSITY ONLINE

You

The role

Adversary

Okay! Gimme the no. of states in your machine for L

Claims L is regular

The are n states in my automata for L

Okay! here is the string w from L such that
|w| >= n
Could you tell me where is the loop in your machine?

The loop is xy^iz

Find some i, so that the resultant string is not in L

PES UNIVERSITY ONLINE

You

The role

Adversary

Okay! Gimme the no. of states in your machine for L

Claims L is regular

The are n states in my automata for L

Okay! here is the string w from L such that
|w| >= n
Could you tell me where is the loop in your machine?

The loop is xy^iz

Find some i, so that the resultant string is not in L

Oh no! I lost!!!!
L is not regular

PES UNIVERSITY ONLINE

You

The role

Adversary

Okay! Gimme the ... states in your machi...

...laims L is regular

Okay! here is the st... from L such th... |w| >= n
Could you tell me w... the loop in your machine?

...e are n states in my automata for L

The loop is xy^iz

Okay! but for some i, the resultant string is not in L

Oh no! I lost!!!!
L is not regular

Using Pumping lemma prove that the language $a^n b^n$ is not regular

PES UNIVERSITY ONLINE

You

The role

Adversary

Claims $L = a^n b^n$ is regular

PES UNIVERSITY ONLINE

The role

You

Okay! Gimme the no. of states in your machine for L

Adversary

Claims $L = a^n b^n$ is regular

PES UNIVERSITY ONLINE

The role

You

Okay! Gimme the no. of states in your machine for L

Adversary

Claims $L = a^n b^n$ is regular

There are 10 states in my automata for L

PES UNIVERSITY ONLINE

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string a^6b^6

$|w| >= 10$

Now tell me where is the loop in your automata?

The role

Adversary

Claims $L = a^nb^n$ is regular

There are 10 states in my automata for L

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

Loop must be within first 10 symbols

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

$a^5(a)^i b^6$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

Pump down, i=0 → $a^5b^6 \notin L$

$a^5(a)^0b^6$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

$a^5(a)^2b^6$

Pump down, i=0 $\rightarrow a^5b^6 \notin L$

Pump up, i=2 $\rightarrow a^7b^6 \notin L$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

$a^5(a)^i b^6$

Pump do… L

i = 1 doesn't help !!

…, i=2 → $a^7b^6 \notin L$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

Pump up, i=2 → $a^5ababb^5 \notin L$

$a^5(ab)^ib^5$

PES UNIVERSITY ONLINE

Where is the loop ?

$$aaaaaabbbbbb$$

Pump up, i=2 → $a^5ababb^5 \notin L$

i = 0 or i = 1 doesn't help !!

$$a^5(ab)^ib^5$$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

Pump down, i=0 → $a^6b^5 \notin L$

Pump up, i=2 → $a^6b^7 \notin L$

$a^6(b)^ib^5$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbbb

Pump down, i=0 → a^6b^5

Pump up, i=2 → $a^6b^7 \notin L$

i = 1 doesn't help !!

$a^6(b)^ib^5$

PES UNIVERSITY ONLINE

Where is the loop ?

aaaaaabbbbbb

For every break up possible we got some i that will result in a string $\notin$ to L

PES UNIVERSITY ONLINE

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string a^6b^6

$|w| >= 10$

Now tell me where is the loop in your automata?

The role

Adversary

Claims $L = a^nb^n$ is regular

There are 10 states in my automata for L

We saw and explored different possibilities where the loop could be

PES UNIVERSITY ONLINE

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string a^6b^6

$|w| >= 10$

Now tell me where is the loop in your automata?

Okay! but for some i, nothing worked out!

The role

Adversary

Claims $L = a^nb^n$ is regular

There are 10 states in my automata for L

We saw and explored different possibilities where the loop could be

PES UNIVERSITY ONLINE

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string a^6b^6

$|w| >= 10$

Now tell me where is the loop in your automata?

Okay! but for some i, nothing worked out!

The role

Adversary

Claims $L = a^nb^n$ is regular

There are 10 states in my automata for L

We saw and explored different possibilities where the loop could be

Oh no! I lost!!!!
L is not regular

The role

You

Okay! Gimme the … states in your machi…

okay! I'll choose the …
a^6b^6
$|w| >= 10$
Now tell me where is the loop in your automata?

Okay! but for some i, nothing worked out!

Adversary

…ms $L = a^nb^n$ is regular

…e are 10 states in my automata for L

…e saw and explored different possibilities where the loop could be

Oh no! I lost!!!!
L is not regular

Using Pumping lemma prove that the language of the form ww over {a,b}* is not regular

PES UNIVERSITY ONLINE

The role

You

Adversary

Claims L = ww is regular

PES UNIVERSITY ONLINE

The role

You

Okay! Gimme the no. of states in your machine for L

Adversary

Claims L = ww is regular

PES UNIVERSITY ONLINE

The role

You

Okay! Gimme the no. of states in your machine for L

Adversary

Claims L = ww is regular

There are n states in my automata for L

The role

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string $a^n a^n$

|w| >= n

Now tell me where is the loop in your automata?

Adversary

Claims L = ww is regular

There are n states in my automata for L

PES UNIVERSITY ONLINE

Where is the loop ?

a...a..aa..aa...a..aa..a

PES UNIVERSITY ONLINE

Where is the loop ?

a...a.aa..aa...a.aa..a

Loop must exist within first n symbols

PES UNIVERSITY ONLINE

Where is the loop ?

$$a \ldots a..aa..aa \ldots a..aa..a$$

$$a^{n-2}(aa)^i a^n$$

Where is the loop ?

$$a...a..aa..aa...a..aa..a$$

$$a^{n-2}(aa)^i a^n$$

Let's Pump down, i=0

$= a^{n-2}a^n$

$= a^{n-2}a^2a^{n-2}$

$= a^{n-1}a^{n-1}$

Where is the loop ?

$$a...a..aa..aa...a..aa..a$$

$$a^{n-2}(aa)^i a^n$$

Let's Pump up, i=3

$$= a^{n-2}(aa)^3 a^n$$
$$= a^{n-2}(a^2)^3 a^n$$
$$= a^{n-2}a^6 a^n$$
$$= a^{n-2}a^4 a^2 a^n$$
$$= a^{n-2+4} a^{n+2}$$
$$= a^{n+2} a^{n+2}$$

Where is the loop ?

a…a..aa..aa…a..aa..a

$a^{n-2}(aa)^i a^n$

Pump up or Pump down,
resultant string will always belong to L

PES UNIVERSITY ONLINE

Where is the loop ?

$a \ldots a \mid a \ldots a$

$a^{n-2}(aa)^i a^n$

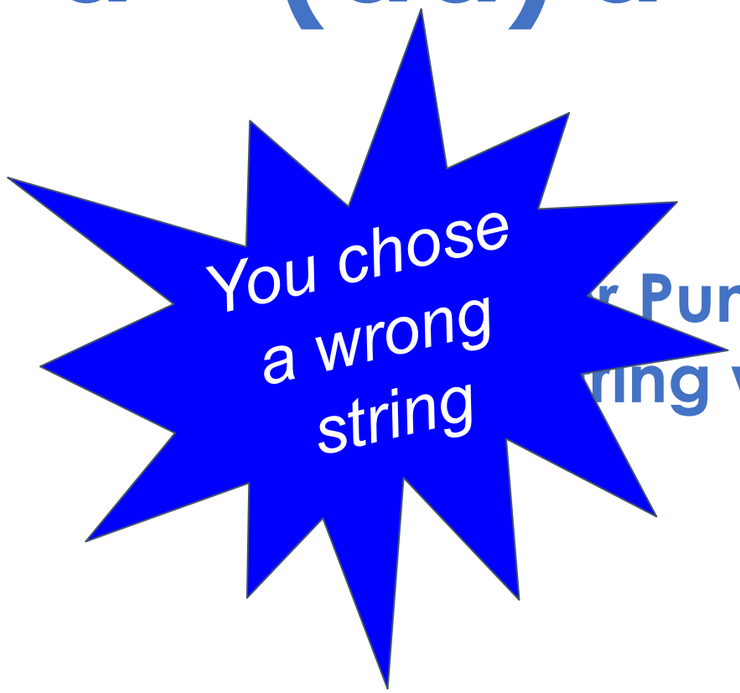
Pump up or Pump down,
resultant string will always belong to L

PES UNIVERSITY ONLINE

Where is the loop ?

$a \ldots a \; | \; a \ldots a$

$a^{n-2}(aa)^i a^n$



r Pump down,
ring will always belong to L

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

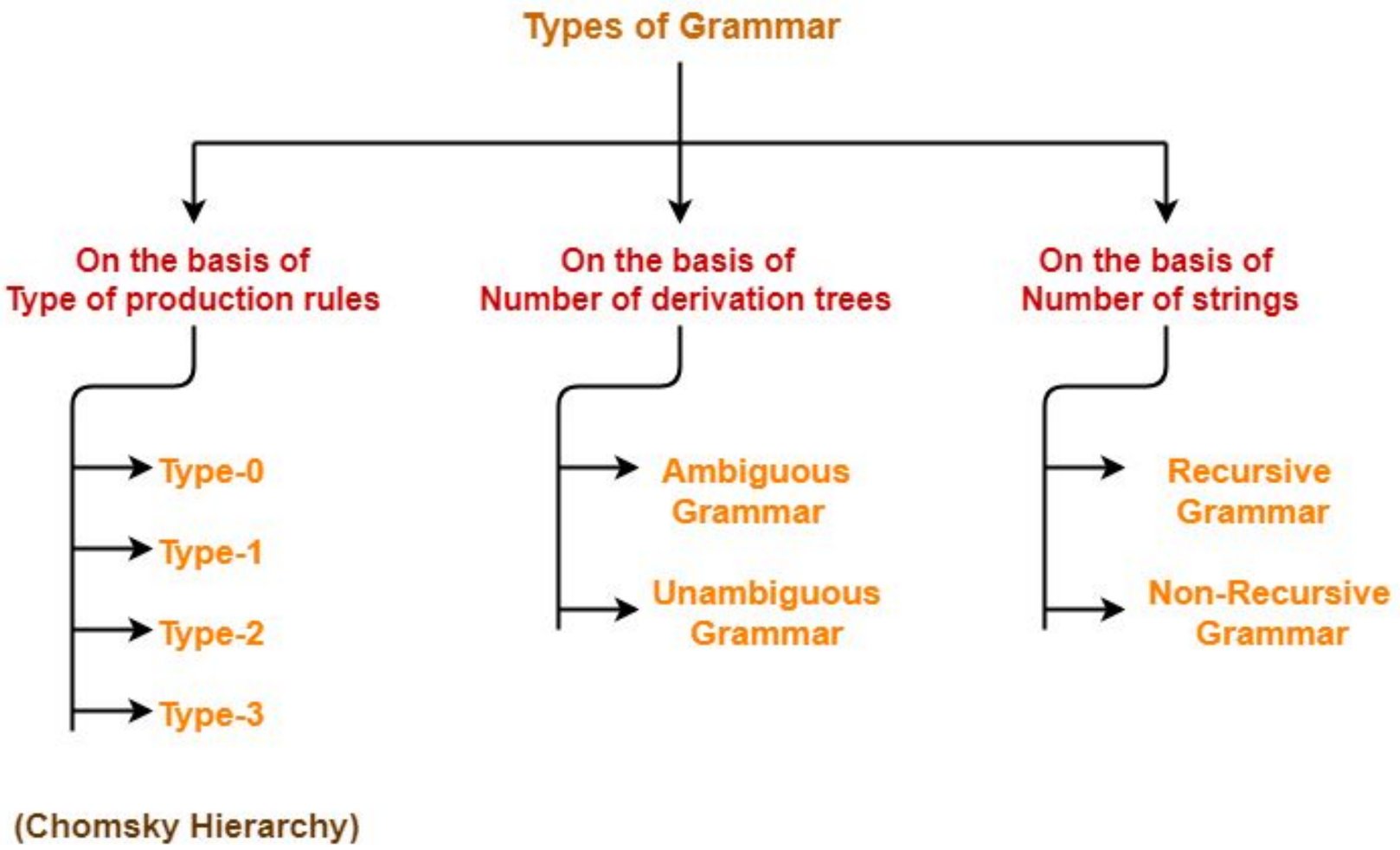
Unit 3

Preet Kanwal

Department of Computer Science & Engineering

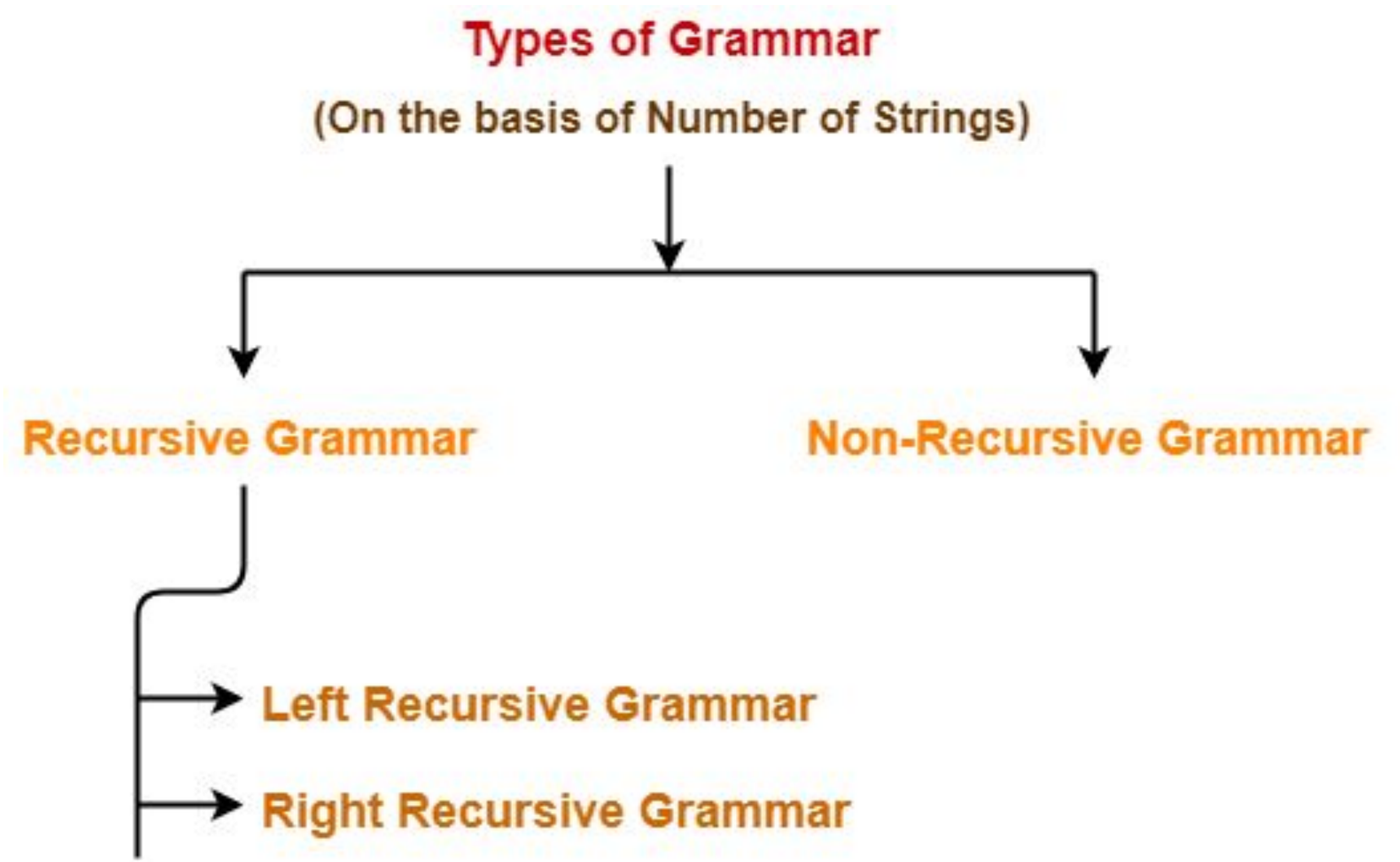
Grammar

- **V - Variables** — expr, var, S, A, B…, w,x,y
- **T - Terminals** — a, b, c …, 0, 1, … id, num
- **P - Production Rules** — $\alpha = (V \cup T)^*$; $A \rightarrow \alpha$
- **S - Start Symbol** — L(G) = {w | S ⇒ w, where w ∈ Σ^*}

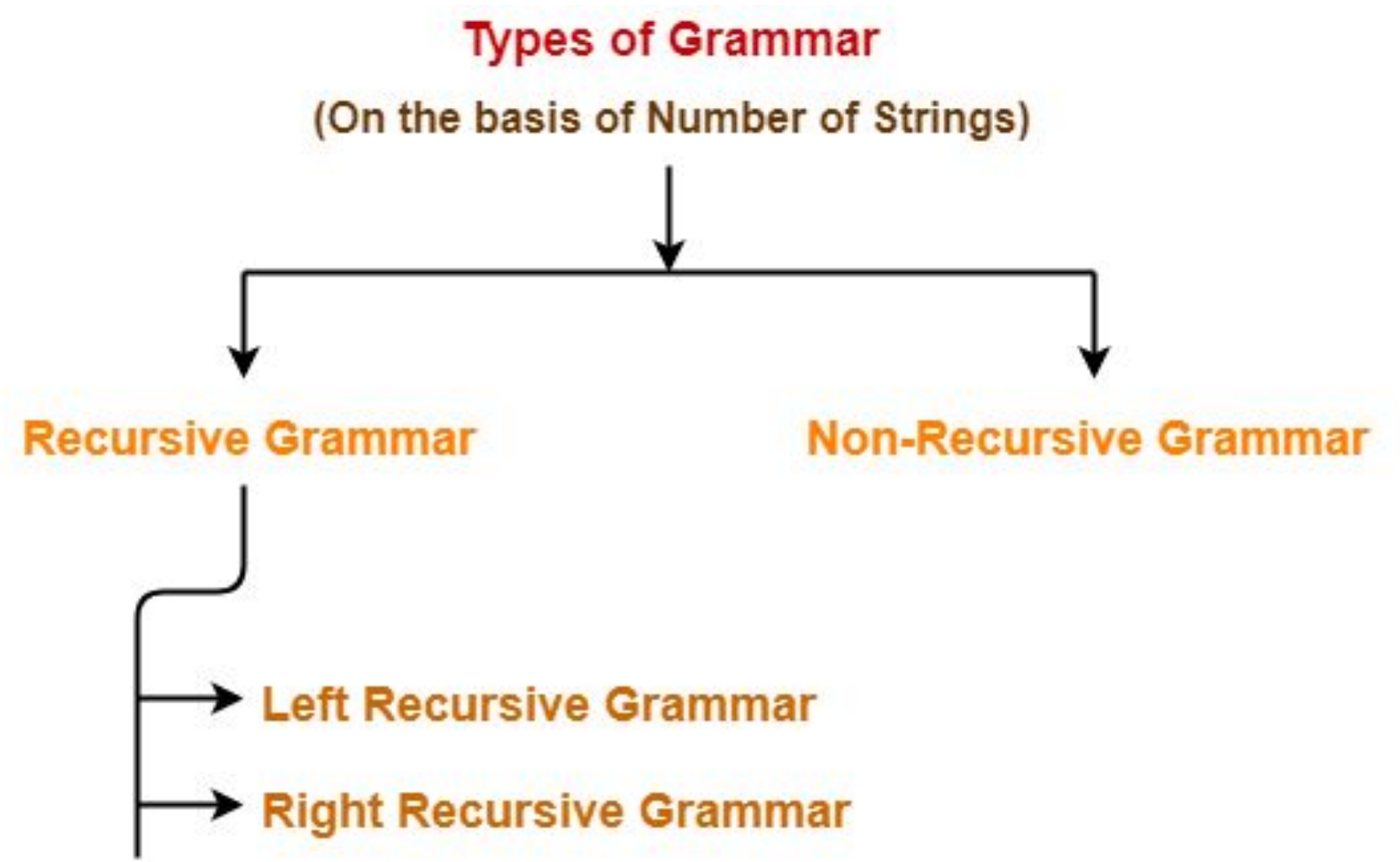


(Chomsky Hierarchy)

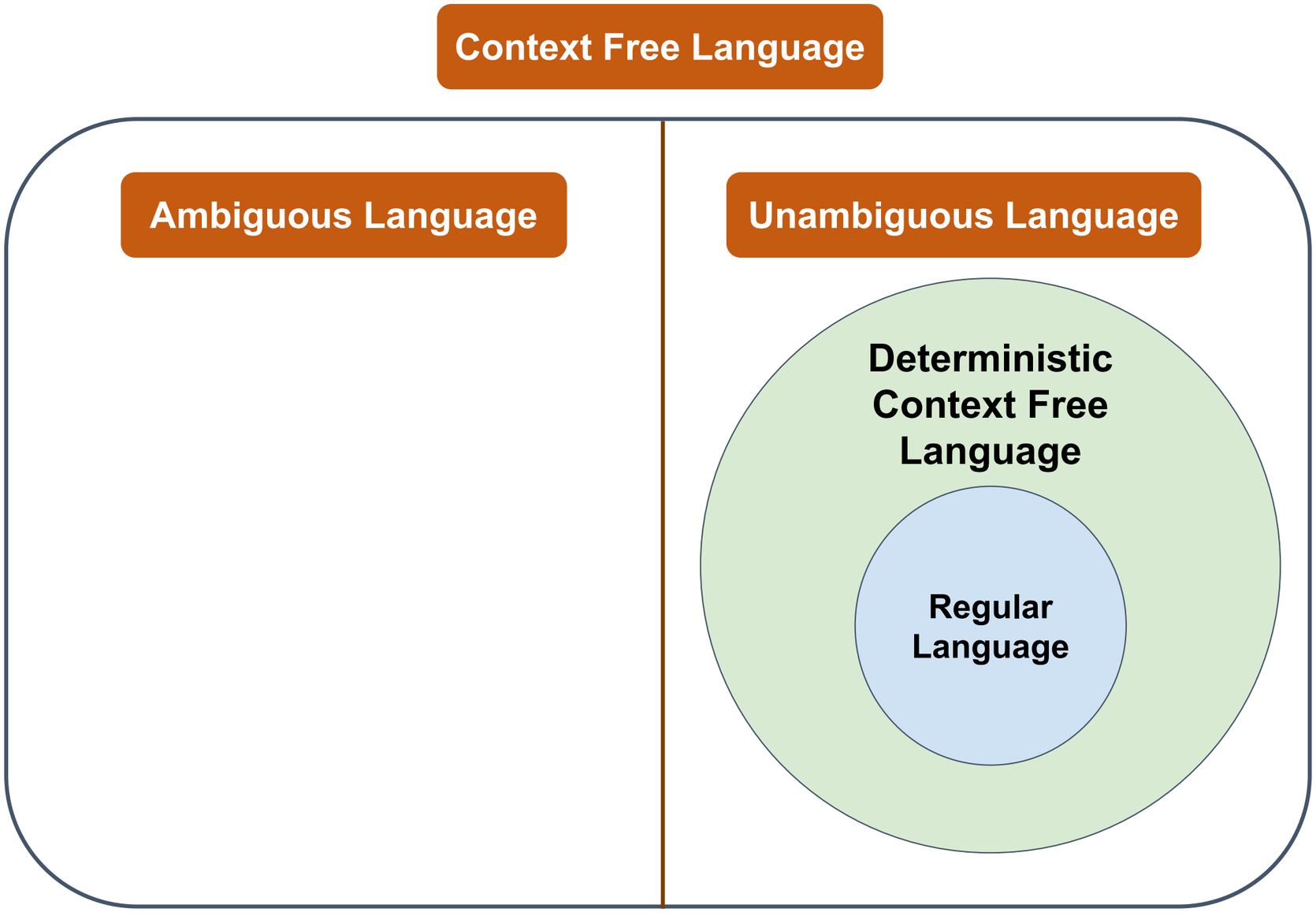
https://www.gatevidyalay.com/what-is-grammar-types-of-grammar-automata/



https://www.gatevidyalay.com/recursive-grammar-left-recursive-grammar/



https://www.gatevidyalay.com/recursive-grammar-left-recursive-grammar/


Context Free Language
Ambiguous Language
Unambiguous Language
Deterministic Context Free Language
Regular Language

Example 1:

Construct linear grammar for the even palindromes. $L=\{ww^R, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSa|bSb|\lambda$

Example 2:

Construct linear grammar for the even palindromes. $L=\{wCw^R, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSa|bSb|C$

Example 3:

Construct linear grammar for $L=\{ww^R, w \in \{ab\}^* \mid (ba)^*\}$

Solution :

$S \rightarrow abSba \mid baSab \mid \lambda$

Example 4:
Construct linear grammar for $L=\{a^n w w^R b^n \mid w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSb \mid A$

$A \rightarrow aAa \mid bAb \mid \lambda$

Example 5:
Construct linear grammar for $L=\{a^n b^{n+1} , n>=0\}$

Solution :

$S \rightarrow aSb \mid b$

Example 6:
Construct linear grammar for $L=\{a^{n+2}b^n, n>=1\}$

Solution :

$S \rightarrow aSb \mid aaab$

Example 7:
Construct linear grammar for $L=\{a^n b^{2n}, n>=0\}$

Solution :

$S \rightarrow aSbb \mid \lambda$

Example 8:
Construct linear grammar for $L=\{a^n b^{n-3}, n>=3\}$

Solution :

$S \rightarrow aSb \mid aaa$

PES UNIVERSITY ONLINE

Example 9:
Construct linear grammar for $L=\{a^n b^m, n>m\}$

Solution :

$S \rightarrow aSb \mid aS \mid a$

Example 10:
Construct linear grammar for $L=\{a^n b^m, n \neq m\}$

Solution :

$S \rightarrow A \mid B$

$A \rightarrow aAb \mid aA \mid a$

$B \rightarrow aBb \mid bB \mid b$

Example 11:
Construct linear grammar for $L=\{a^n b^m, n=2+(m \bmod 3)\}$

Solution :

$S \rightarrow aaA \mid aaabA \mid aaaabbA$

$A \rightarrow bbbA \mid \lambda$

Example 12:
Construct linear grammar for $L=\{a^n b^m, n \neq 2m\}$

Solution :

$S \rightarrow aaSb|A|B|aC$

$A \rightarrow aA|a$

$B \rightarrow Bb|b$

$C \rightarrow Cb|\lambda$

Example 13:
Construct linear grammar for $L=\{a^{n+2}b^m, m>n, n>=0\}$

Solution :

$S \rightarrow aSb|aab|Sb$

Example 14:
Construct linear grammar for $L=\{a^n b^m c^m d^n, n,m>=1\}$

Solution :

$S \rightarrow aSd|aAd$

$A \rightarrow bAc|bc$

Example 15:

Construct linear grammar for $L=\{a^n b^m c^k, k=n+m, n,m,k>=0\}$

Solution :

$S \rightarrow aSc|A$

$A \rightarrow bAc|\lambda$

Example 16:
Construct linear grammar for $L=\{a^n b^m c^k, m=2n, k=2, n>=0\}$

Solution :

$S \rightarrow AB$

$A \rightarrow aAbb \mid \lambda$

$B \rightarrow cc$

Example 17:
Construct linear grammar for $L=\{a^n b^m c^k, m,n>=0 , k=n+2m\}$

Solution :

$S \rightarrow aSc|A$

$A \rightarrow bAcc|\lambda$

Example 18:

Construct linear grammar for L={|w|mod 3 ≠ |w| mod 2,w∈{a}*}

Solution :

$S \rightarrow aaaaaaS \mid aa \mid aaa \mid aaaa \mid aaaaa$

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 3

Preet Kanwal

Department of Computer Science & Engineering

Example 1:

Construct CFG for $L=\{uvwv^R, |u|=|w|=2, |v|>1, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow A\ B$

$A \rightarrow aa|bb|ab|bb$

$B \rightarrow aBa|bBb|A$

Example 2:

Construct a CFG for $L=\{n_a(w)=n_b(w) , w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSb|bSa|\lambda|SS$

Example 3:

Construct a CFG for $L=\{n_a(w)=n_b(w)+1, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow AaA$

$A \rightarrow aAb|bAa|AA|\lambda$

or

$S \rightarrow aSb|bSa|abS|baS|a$

Example 4:

Construct a CFG for $L=\{n_a(w)=2\times n_b(w)\ ,\ w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSaSb|bSaSa|aSbSb|SS|\lambda$

Example 5:

Construct a CFG for $L=\{n_a(w)>n_b(w) , w \in \{a,b\}^*\}$

Solution :

$S \rightarrow AaA$

$A \rightarrow aAb|bAa|AA|aA|Aa|\lambda$

Example 6:

Construct a CFG for $L=\{n_a(w) \neq n_b(w) , w \in \{a,b\}^*\}$

Solution :

$S \rightarrow AaA|BbB$

$A \rightarrow aAb|bAa|Aa|aA|\lambda$

$B \rightarrow aBb|bBa|Bb|bB|\lambda$

Example 7:
Construct a CFG for $L=\{a^n b^n \cup a^n b^{2n}\}$.

Solution :

$S \rightarrow S1 \mid S2$

$S1 \rightarrow aS1b \mid \lambda$

$S2 \rightarrow aS2bb \mid \lambda$

Example 8:
Construct /a CFG for Language of proper nesting(parenthesis matching) where Σ = {(,)}.

Solution :

S⟶ (S) | SS
S⟶ λ

Example 9:
Construct /a CFG for Language of proper nesting(parenthesis matching) where Σ = {(,{,[,],},)}.

Solution :

$S \rightarrow (S) \mid \{S\} \mid [S] \mid SS \mid \lambda$

Example 10:
Construct a CFG for Language to generate arithmetic expressions.

Solution :

E⟶ E+E | E-E | E*E | E/E | (E) | id| number

Example 11:
Construct a CFG for nested if else.

Solution :

S⟶ if condition then S
S⟶ if condition then S else S
S⟶ {statement }

Example 12:
Construct a CFG to take care of variable declarations in C Language.

Solution :

D⟶ Type List
List⟶ List,id|id
Type⟶ int|float|char

Example 13:
Construct a CFG to generate nested while loops.

Solution :

S⟶ while(condition)S|{statement}

Do while loop:

S⟶ while(condition)S|do S while (condition){statement}

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

Automata Formal Languages & Logic

Unit 2 – Pushdown Automata (PDA)

Prakash C O and **Preet Kanwal**

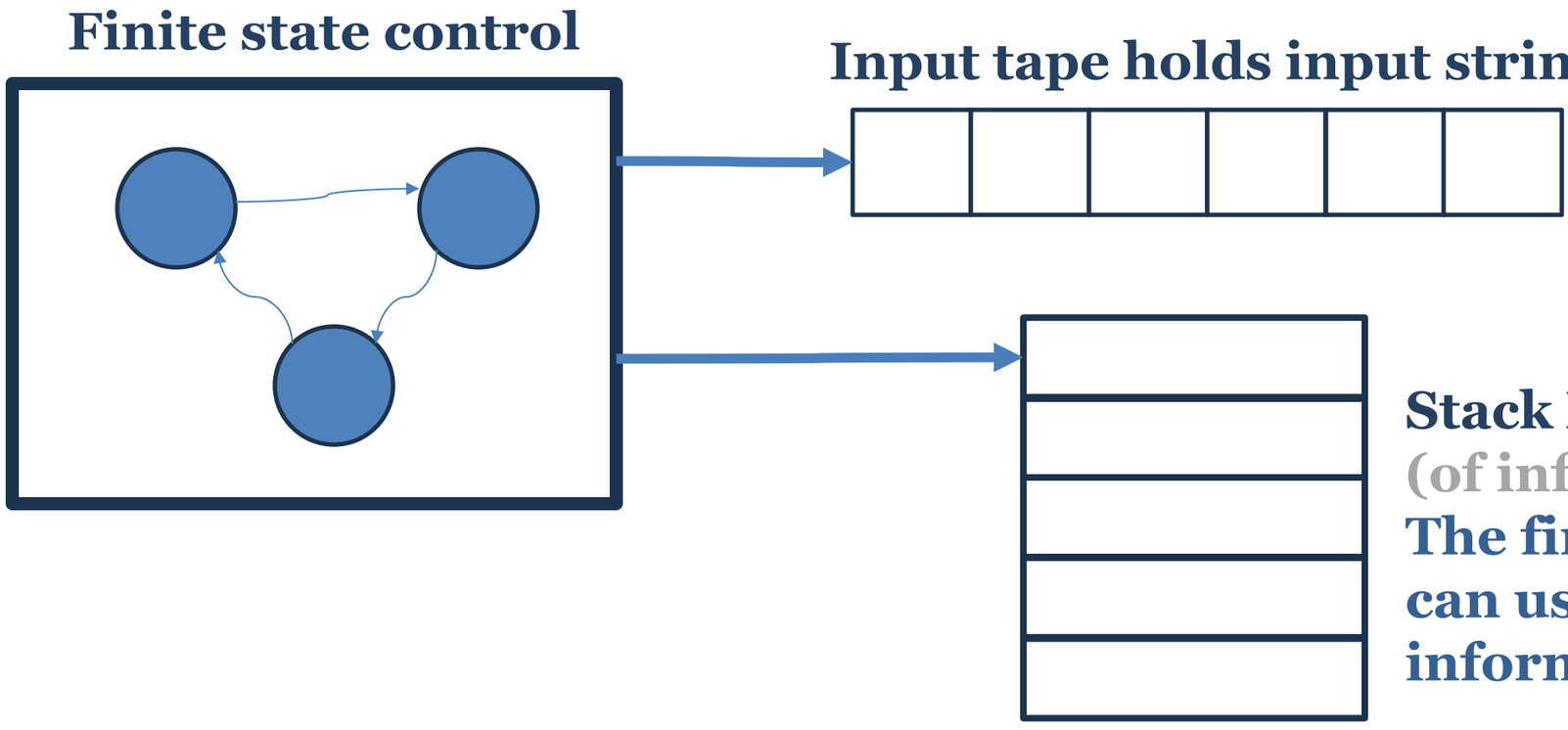
Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2: Deterministic Pushdown Automata (DPDA)

Prakash C O and **Preet Kanwal**

Department of Computer Science & Engineering



PDA = Finite Automata + Stack (of infinite length)

- **A pushdown automaton (PDA) is a finite automaton equipped with a stack based memory.**
- **PDAs are more capable than finite-state machines but less capable than Turing machines.**
- **A pushdown automaton (PDA) differs from a finite state machine in two ways:**
  - **It can use the top of the stack to decide which transition to take.**
  - **It can manipulate the stack as part of performing a transition.**

Formal Definition of PDA

A pushdown automaton P is a seven-tuple $(Q, \Sigma, \Gamma, \delta, q_o, Z_o, F)$ where,

- **Q** is a finite set of states,
- **Σ** is a finite set of input symbols,
- **Γ** is a finite set of stack symbols,
- **δ** is a transition function,

  In DPDA, δ is $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow Q \times \Gamma^*$

  In NPDA, δ is $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow 2^Q \times \Gamma^*$
- $q_o \in Q$ is the start/initial state,
- $Z_o \in \Gamma$, Initially, the stack holds a special symbol Z_o that indicates the bottom of the stack.
- $F \subseteq Q$ is a set of accepting states.

Note: In the above equation, Γ* represents stack new top symbols

- **A pushdown automaton reads a given input string from left to right.**

  In each step, it chooses a transition by considering

  1) the current state,
  2) the next input symbol, and
  3) the symbol at the top of the stack.

- **A pushdown automaton can also manipulate the stack, as part of performing a transition.**

  The manipulation can be

  - to push a particular symbol to the top of the stack, or
  - to pop off the top of the stack.

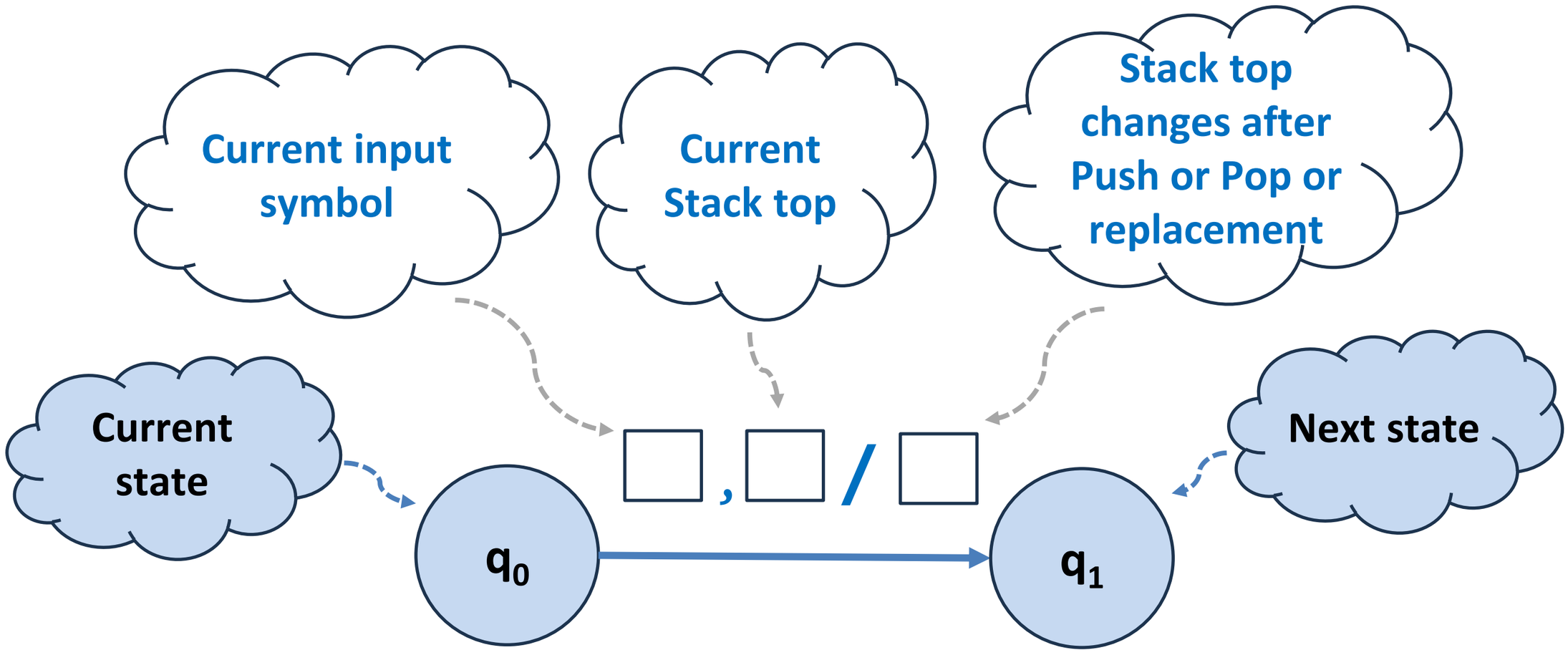
  The automaton can alternatively ignore the stack, and leave it as it is.

- **Put together**: **Given an input symbol, current state, and stack symbol, the automaton can follow a transition to another state, and optionally manipulate (push or pop) the stack.**

- **If, in every situation of the PDA, at most one transition action is possible, then the automaton is called** a **deterministic pushdown automaton (DPDA).**
- **In general, if several actions are possible, then the automaton is called a general, or nondeterministic pushdown automaton (NPDA).**
- **Deterministic pushdown automata(DPDA) can recognize all deterministic context-free languages (DCFLs) while nondeterministic PDAs can recognize all context-free languages,** with the former often used in parser design.
- **A language that can be recognized by a DPDA** is called a **deterministic context-free language(DCFL).**

PDA as a state diagram

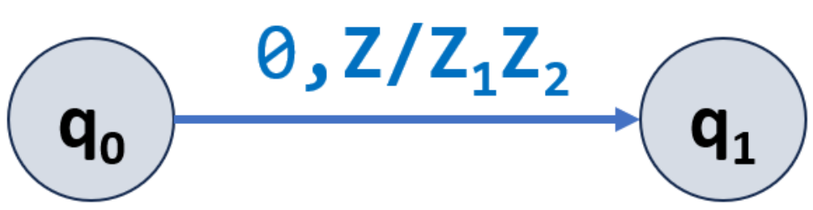
Transitions in Pushdown Automaton

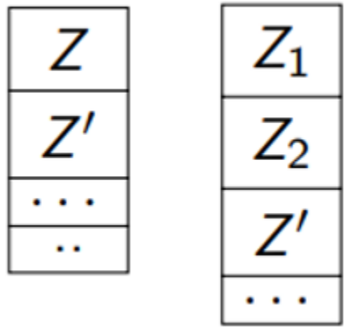


Transitions in Pushdown Automaton

Let $\Sigma=\{0,1\}$, $\Gamma=\{Z_0,Z_1,Z_2,...\}$ and $\delta(q_0,0,Z) = (q_1,Z_1Z_2)$

We draw the PDA transitions as follows

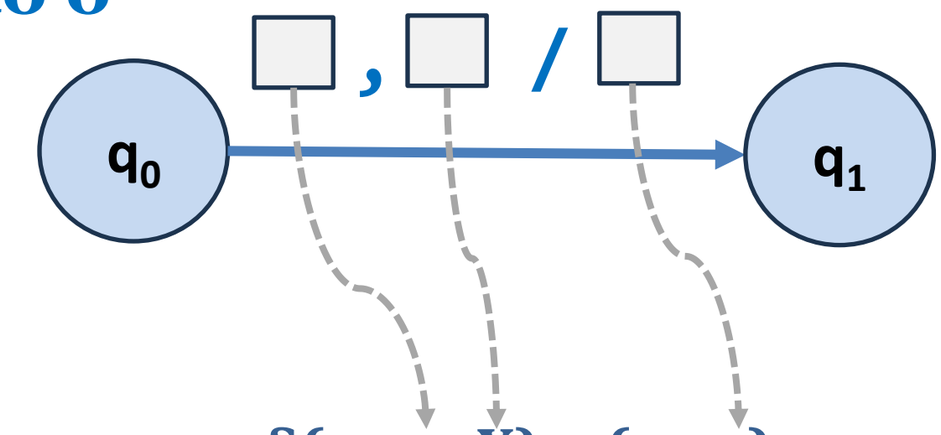




- To execute the transition, top symbol of the stack must be Z
- After transition Z will be popped and word Z_1Z_2 will be pushed on to the stack

Understanding stack interaction due to δ

- State transition from q_0 to q_1
- a is the current input symbol
- X is the current stack *top* symbol



- Popping X from stack: $\delta(q_0, a, X) = (q_1, \varepsilon)$
- Stack top is unchanged: $\delta(q_0, a, X) = (q_1, X)$
- Pop X and Push Y: $\delta(q_0, a, X) = (q_1, Y)$
- Pushing Y in stack: $\delta(q_0, a, X) = (q_1, YX)$
- Pop X and Push $Y_1Y_2...Y_k$: $\delta(q_0, a, X) = (q_1, Y_1Y_2...Y_k)$
- Pushing $Y_1Y_2...Y_k$ in stack: $\delta(q_0, a, X) = (q_1, Y_1Y_2...Y_kX)$

Transition Label forms

1) A transition label of the form: ε, ε / ε or λ, λ / λ

This transition means don't consume any input, don't consider stack top symbol, and don't pop anything from a stack, or don't add anything to a stack . **It's the equivalent of an ε-transition in an NFA.**

2) A transition label of the form: a, b / c

Means "If the current input symbol is a and the current stack symbol is b, then, pop b, and push c.

3) A transition label of the form: a, b / b

Means "If the current input symbol is a and the current stack symbol is b, then, pop b, and push b (or No stack operation or No changes in stack).

4) A transition label of the form: a, b/ ε or a, b/ λ

Means "If the current input symbol is a and the current stack symbol is b, then, pop b.

Transition Label forms cont...

5) A transition label of the form: a, Z / aZ

Means "If the current input symbol is a and the current stack symbol is Z, then, **push a.**

6)A transition label of the form: a, Z / aaZ

Means "If the current input symbol is a and the current stack symbol is Z, then, **push 2 a's.**

7) A transition label of the form: a, Z / aaaZ

Means "If the current input symbol is a and the current stack symbol is Z, then, **push 3 a's.**

8) A transition label of the form: a, b / aa

Means "If the current input symbol is a and the current stack symbol is b, then, **pop b and push 2 a's.**

9) A transition label of the form: a, ε / x or a, λ / x

Means "If the current input symbol is a (and not worried about stack top contents), it just pushes a new symbol x on to the stack.

A PDA configuration (or Instantaneous description (ID)) at any given instance is a triple: (q, w, γ)

- **q - current state**
- **w - remainder of the input (i.e., unconsumed part)**
- **γ - current stack contents as a string from top to bottom of stack**

If δ(q, a, X) = (p, A) is a transition, then the following are also true:

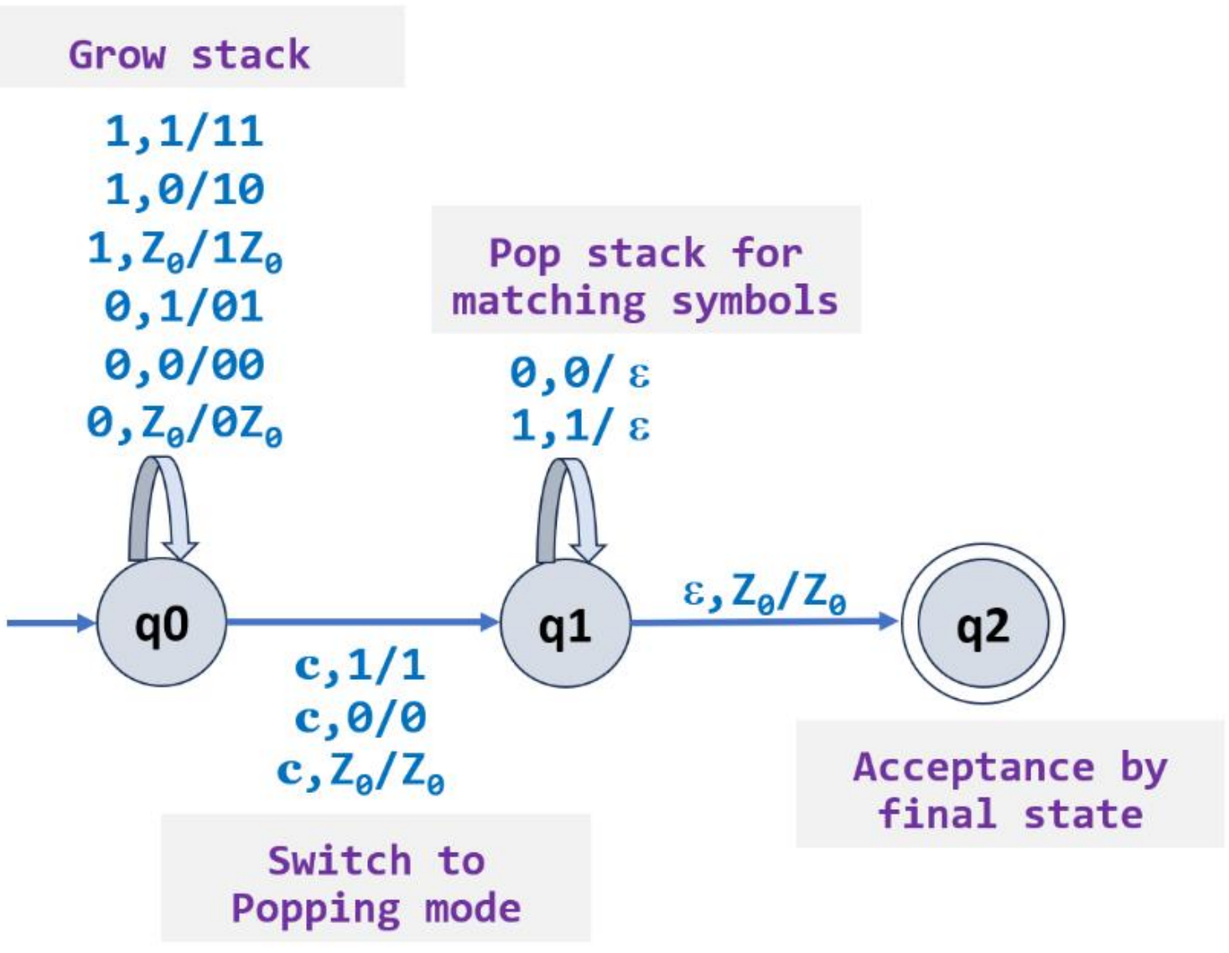
- **(q, a, X) ⊢ (p, ε, A)** ← ID: A move from state q to p, input is empty after reading a and stack top symbol X is replaced by A.
- **(q, aw, XB) ⊢ (p, w, AB)** ← ID: A move from state q to p, remaining input is w and stack top symbol X is replaced by A.

Note: ⊢ sign is called a "turnstile notation" and represents one move

⊢* sign represents a sequence of moves

1. PDA for L = { wcw^R | c is some special symbol not in w and $\sum$={0, 1} }

Solution :



How does the DPDA works on input 10c01 ?

$(q_0, 10c01, Z_0)$
⇩
$(q_0, 0c01, 1Z_0)$
⇩
$(q_0, c01, 01Z_0)$
⇩
$(q_1, 01, 01Z_0)$
⇩
$(q_1, 1, 1Z_0)$
⇩
(q_1, ε, Z_0)
⇩
(q_2, ε, Z_0) Acceptance/Successful run

PDA Configurations or Instantaneous descriptions (IDs)

Language of a PDA

- The common way to define the language of a PDA is **by final state**.
  - **If P is a PDA, then L(P) is the set of strings w** such that $(q_0, w, Z_0) \vdash^* (f, \varepsilon, \alpha)$ for final state f and any α.
- Another language defined by the same PDA is **by empty stack**.
  - **If P is a PDA, then N(P) is the set of strings w** such that $(q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon)$ for any state q.

Final State Acceptability

- A PDA is said to **accept a string when it reaches its final state after completely reading the string.** It is possible to transition from the initial state to a final state with a stack value.
- The **content of the stack doesn't matter as long as we arrive at a final state.**

Empty Stack Acceptability

- The PDA accepts a string after it has emptied its stack after reading the entire string.
- The language accepted by the empty stack of a PDA P is

$$L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon), q \in Q\}$$

Formal Definition of Deterministic PDA (DPDA)

A pushdown automaton P is a seven-tuple $(Q, \Sigma, \Gamma, \delta, q_o, Z_o, F)$

where,

- **Q** is a finite set of states,
- **Σ** is a finite set of input symbols,
- **Γ** is a finite set of stack symbols,
- **δ** is a transition function, $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow Q \times \Gamma^*$
- $q_o \in Q$ is the start/initial state,
- $Z_o \in \Gamma$, Initially, the stack holds a special symbol Z_o that indicates the bottom of the stack.
- $F \subseteq Q$ is a set of accepting states.

Note: In the above equation, Γ* represents stack new top symbols

Deterministic PDA: Definition

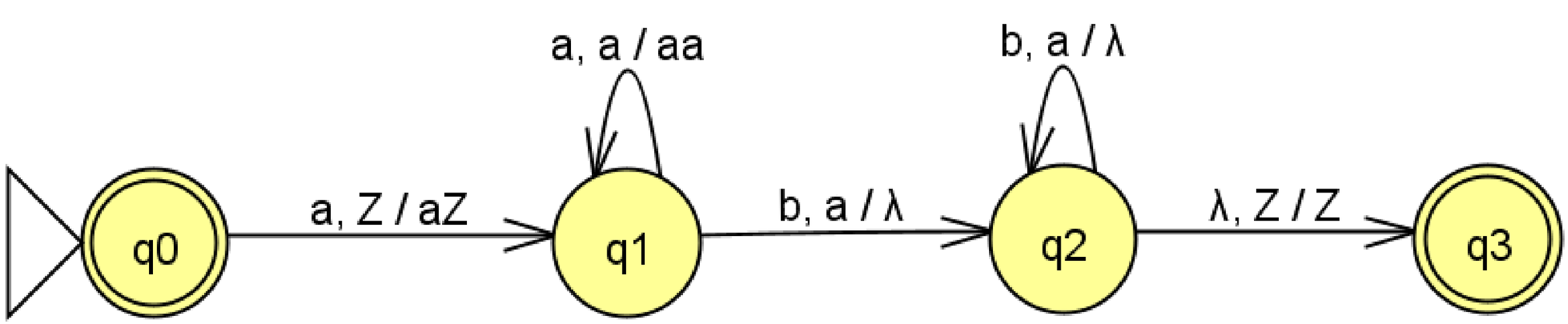
- **A PDA is deterministic, if there is never a choice for a next move in any transition.**
- **A PDA is deterministic if and only if:**
  - **δ(q, a, X) has at most one member for any q in Q, a in Σ∪{ε} and X in Γ.**
  - **If δ(q, a, X) is non-empty for some a in Σ, then δ(q, ε, X) must be empty.**
- A deterministic pushdown automaton is a PDA with the extra property that **for each state in the PDA, and for any combination of a current input symbol and a current stack symbol, there is at most one transition defined.**
- In other words, there is **at most one legal sequence of transitions that can be followed for any input.**

1. Construct a DPDA for the language $L = \{a^n b^n \mid n \geq 0\}$

Solution: $L = \{ \lambda, ab, aabb, aaabbb, aaaabbbb, \ldots \}$

1. Construct a DPDA for the language $L = \{a^n b^n \mid n \geq 0\}$

Solution: $L = \{ \lambda, ab, aabb, aaabbb, aaaabbbb, \ldots \}$



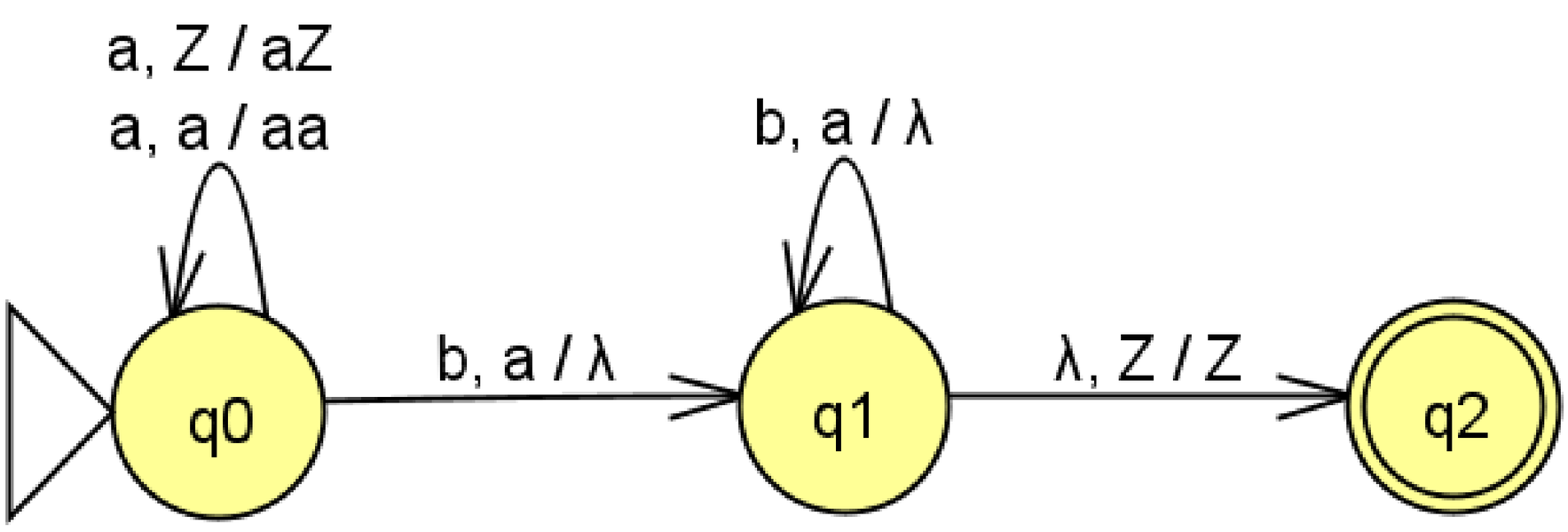
Note: Z is used instead of Z_o to indicate the bottom of the stack.

1a. Construct a DPDA for the language L = {$a^n b^n$ | n≥1}

Solution: L = { ab, aabb, aaabbb, aaaabbbb, … }

1a. Construct a DPDA for the language L = {$a^n b^n$ | n≥1}

Solution: L = { ab, aabb, aaabbb, aaaabbbb, … }

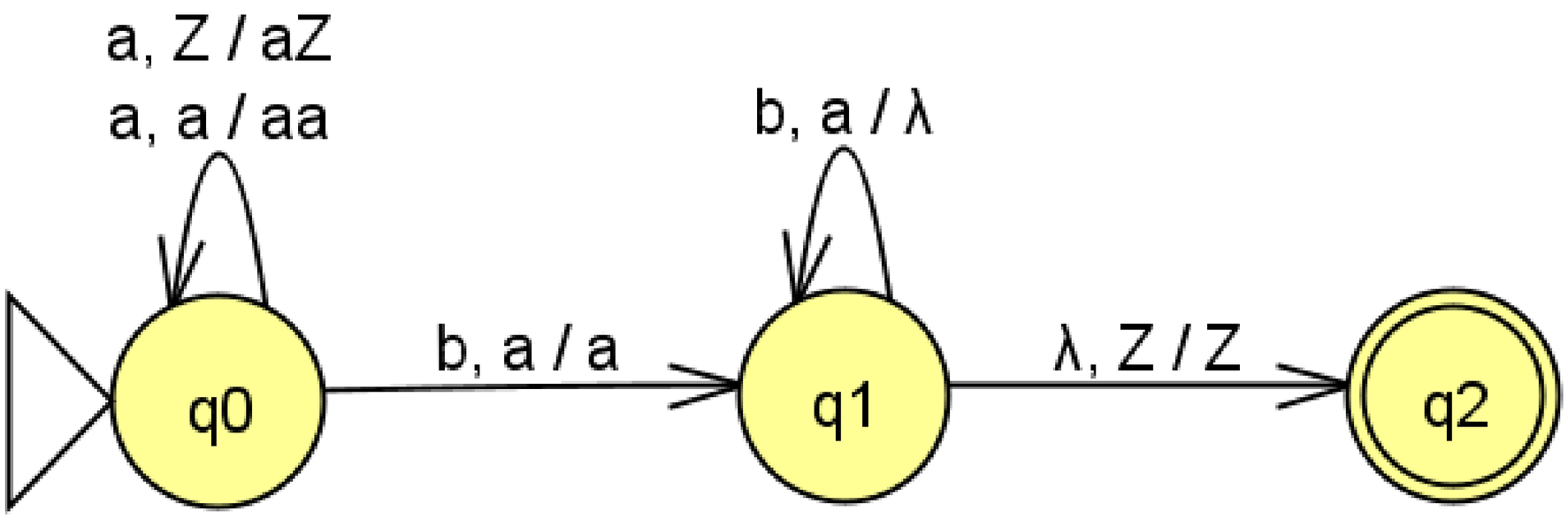


2. Construct a DPDA for the language L = $\{a^n b^{n+1} \mid n \geq 1\}$

Solution: L = { abb, aabbb, aaabbbb, aaaabbbbb, … }

2. Construct a DPDA for the language $L = \{a^n b^{n+1} \mid n \geq 1\}$

Solution: L = { abb, aabbb, aaabbbb, aaaabbbbb, … }

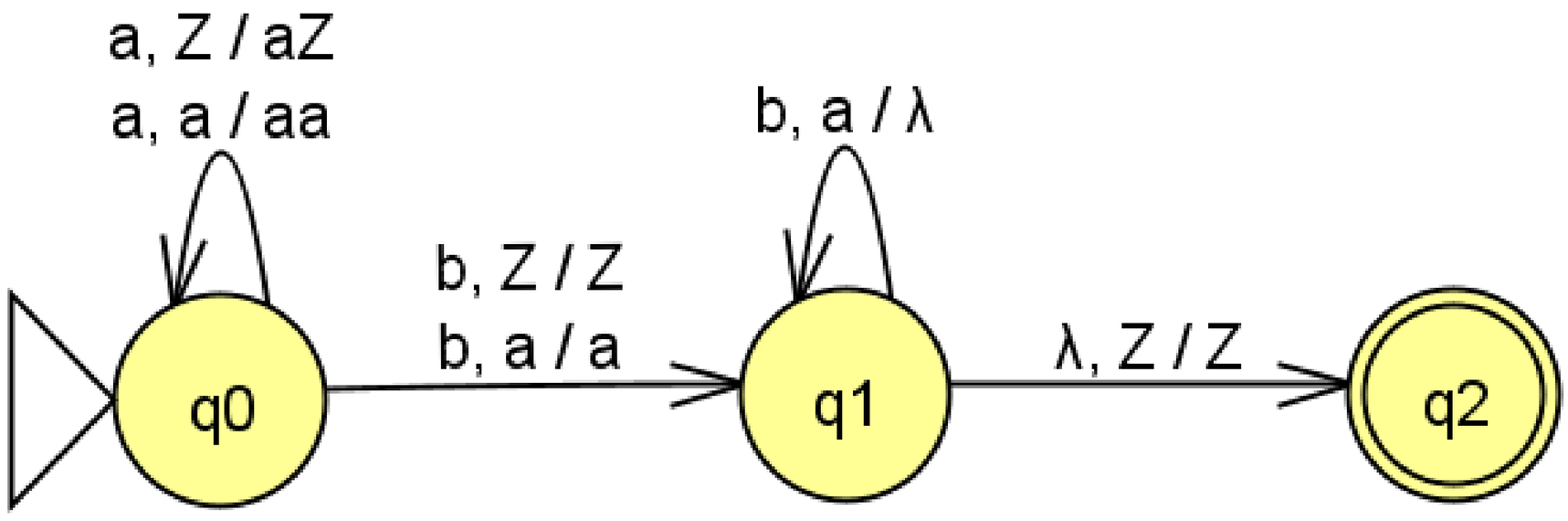


2a. Construct a DPDA for the language L = $\{a^n b^{n+1} \mid n \geq 0\}$

Solution: L = { b, abb, aabbb, aaabbbb, aaaabbbbb, … }

2a. Construct a DPDA for the language L = $\{a^n b^{n+1} \mid n \geq 0\}$

Solution: L = { b, abb, aabbb, aaabbbb, aaaabbbbb, … }



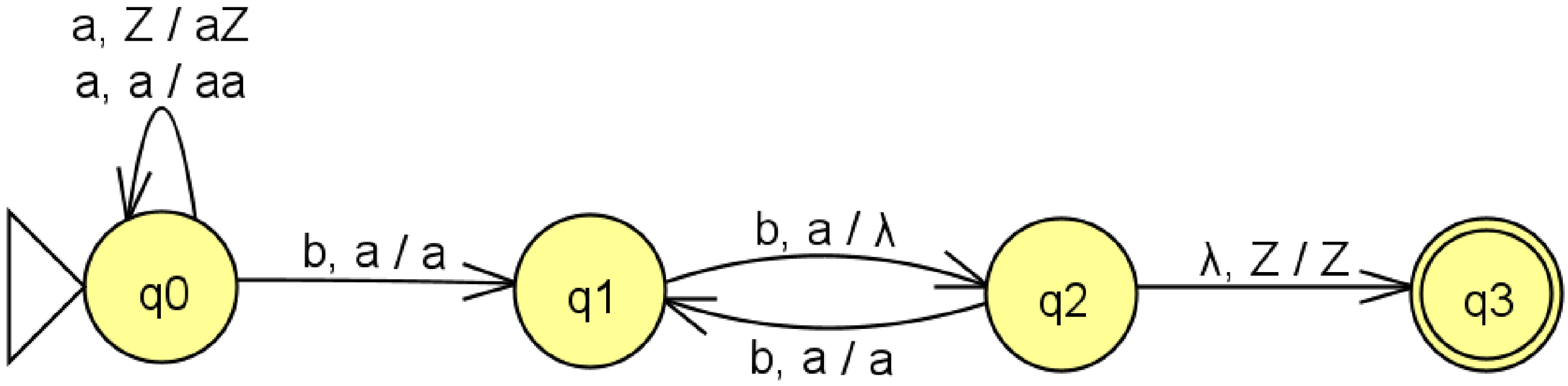
3. Construct a DPDA for the language L = $\{a^n b^{2n} \mid n \geq 1\}$

Solution: L = { abb, aabbbb, aaabbbbbb, aaaabbbbbbbb, … }

There are two ways to solve L = $\{a^n b^{2n} \mid n \geq 1\}$

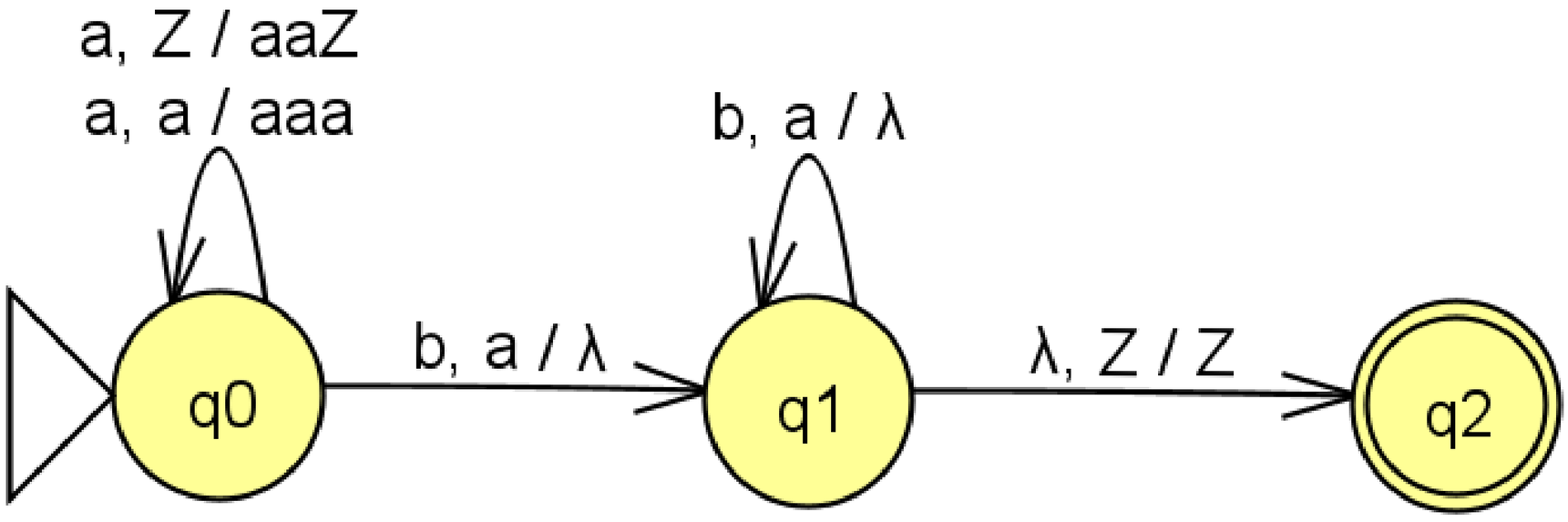
3. Construct a DPDA for the language L = $\{a^n b^{2n} \mid n \geq 1\}$

Solution-1: For every 2 b's pop one a



3. Construct a DPDA for the language L = $\{a^n b^{2n} \mid n \geq 1\}$

Solution-2: Push 2 a's by encountering one a

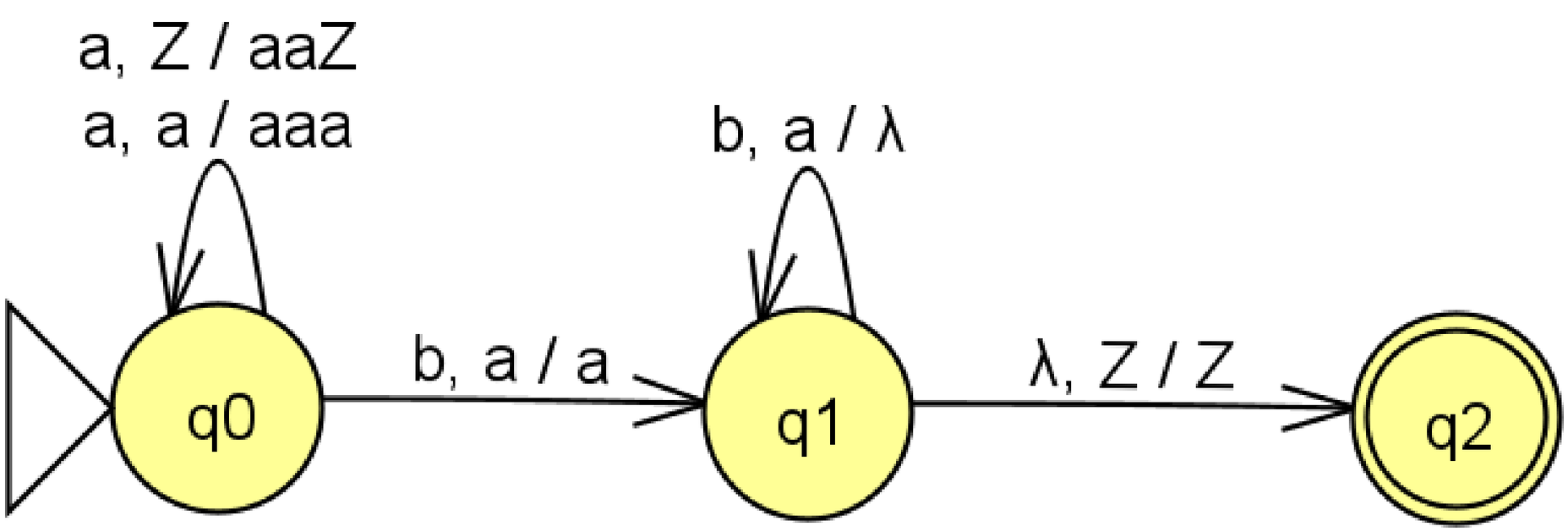


4. Construct a DPDA for the language L = $\{a^n b^{2n+1} | n \geq 1\}$

Solution: L = { abbb, aabbbbb, aaabbbbbbb, aaaabbbbbbbbb, … }

4. Construct a DPDA for the language L = $\{a^n b^{2n+1} | n \geq 1\}$

Solution: L = { abbb, aabbbbb, aaabbbbbbb, aaaabbbbbbbbb, … }

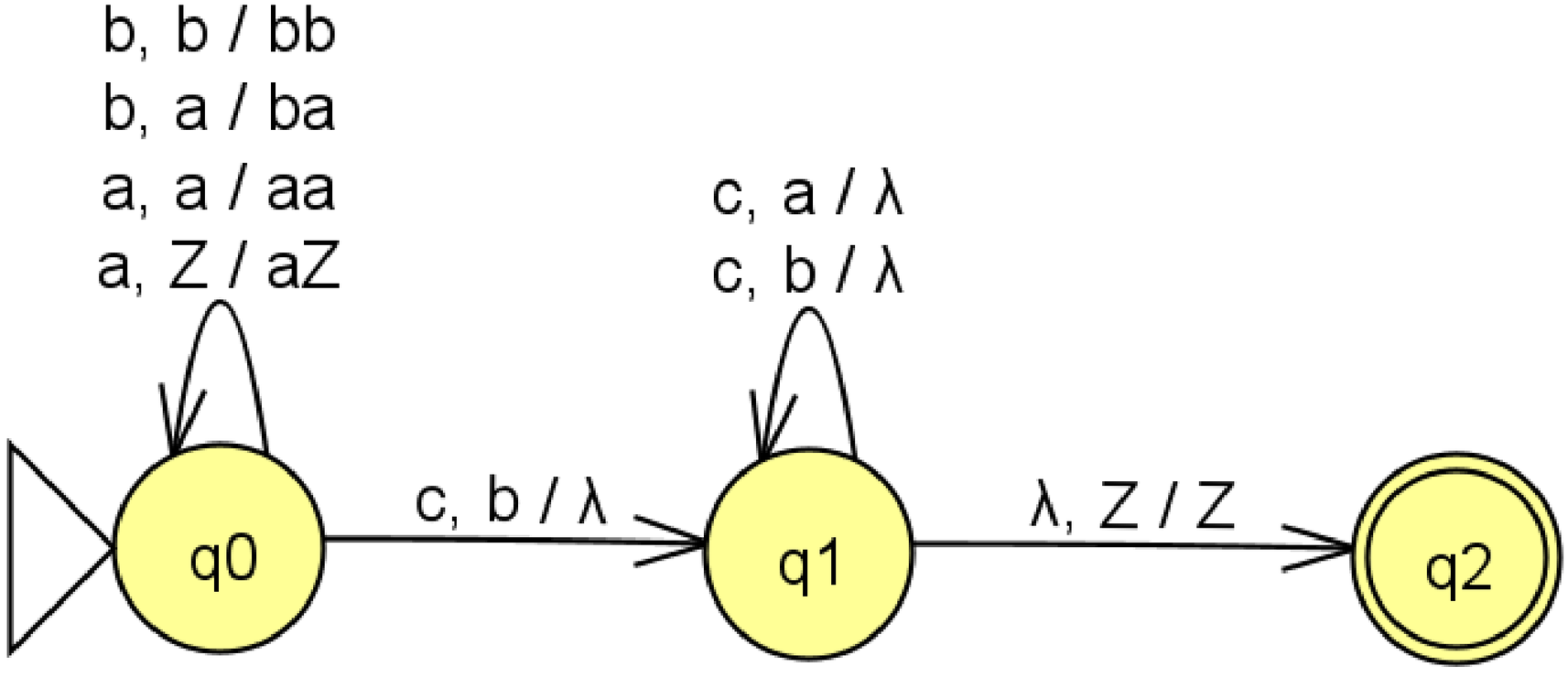


5. Construct a DPDA for the language L = $\{a^n b^m c^{m+n} \mid m,n \geq 1\}$

Solution: L = { abcc, abbccc, aabccc, ... }

5. Construct a DPDA for the language $L = \{a^n b^m c^{m+n} \mid m,n \geq 1\}$

Solution: L = { abcc, abbccc, aabccc, … }

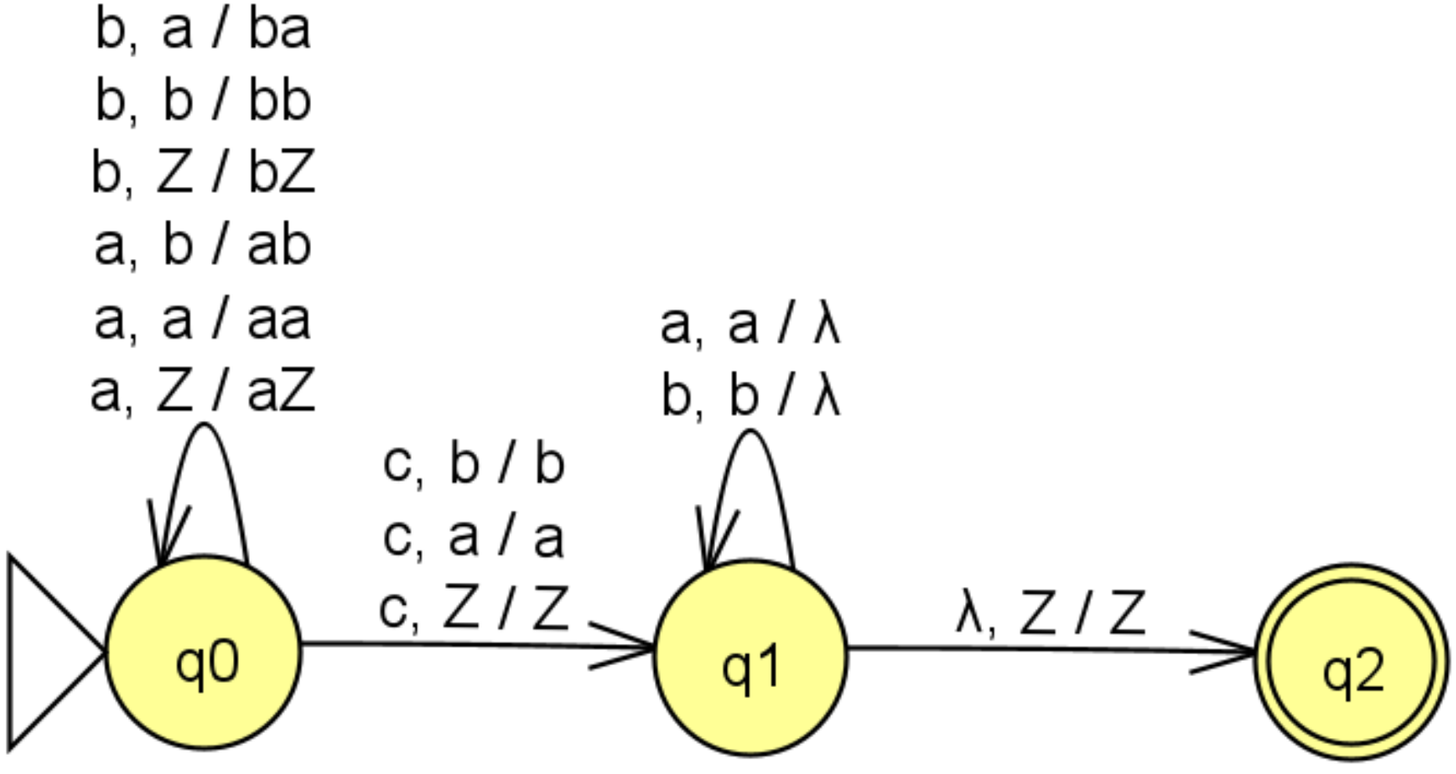


6. Construct a DPDA for the language L ={ wcw^R | w∈{a, b}* }

Solution: L={ c, aca, bcb, abcba, bacab, aacaa, bbcbb, … }

6. Construct a DPDA for the language L ={ wcw^R | w∈{a, b}* }

Solution: L={ c, aca, bcb, abcba, bacab, aacaa, bbcbb, … }



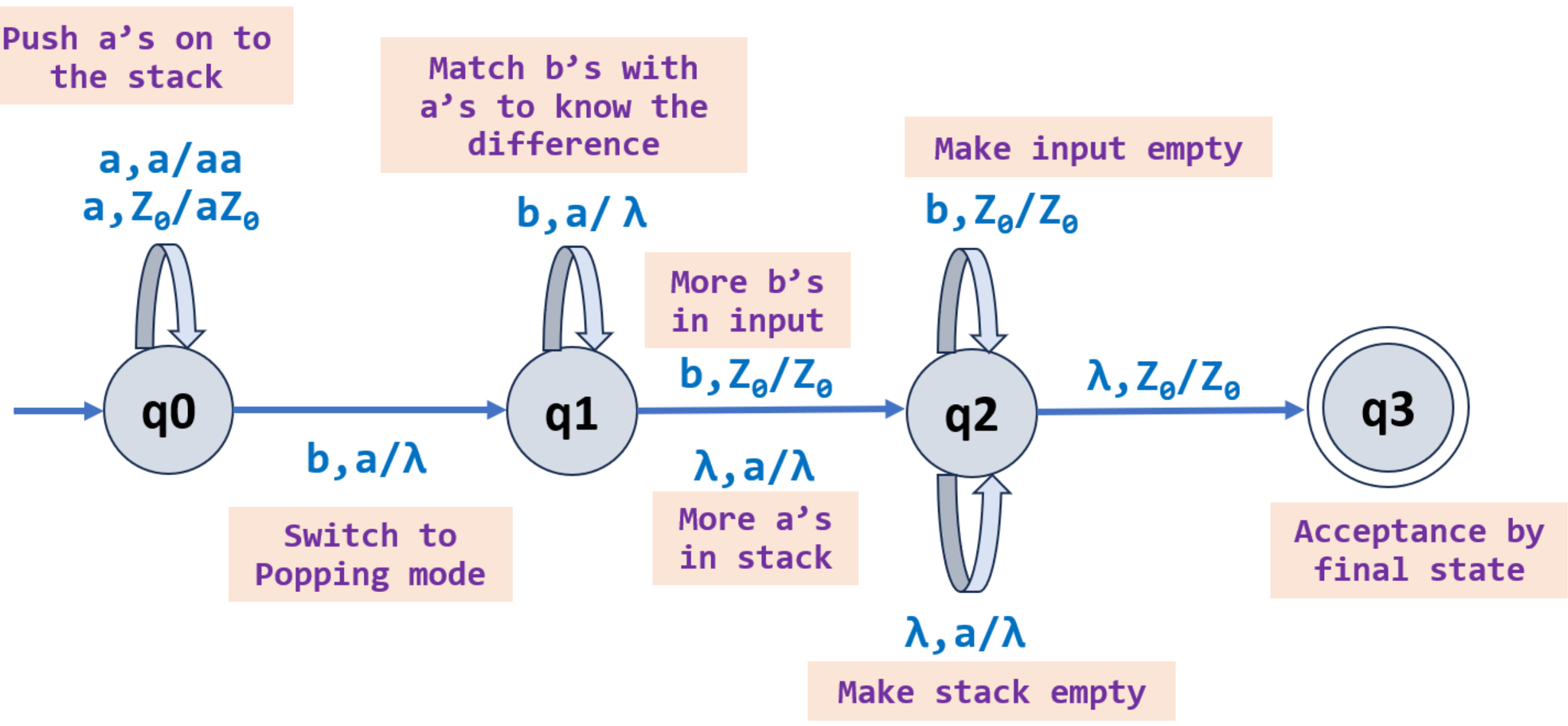
PES UNIVERSITY ONLINE

7. Construct DPDA for the language $L = \{a^n b^m \mid n,m \geq 1 \text{ and } n \neq m\}$

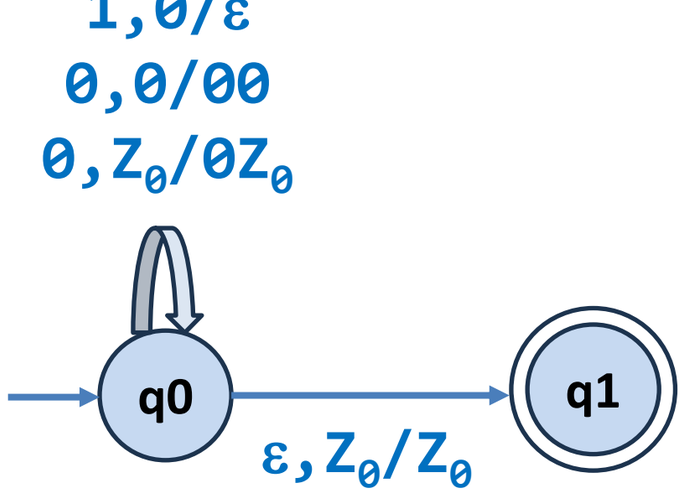
Solution:

7. Construct DPDA for the language L = {$a^n b^m$ | n,m ≥ 1 and n ≠ m}

Solution:

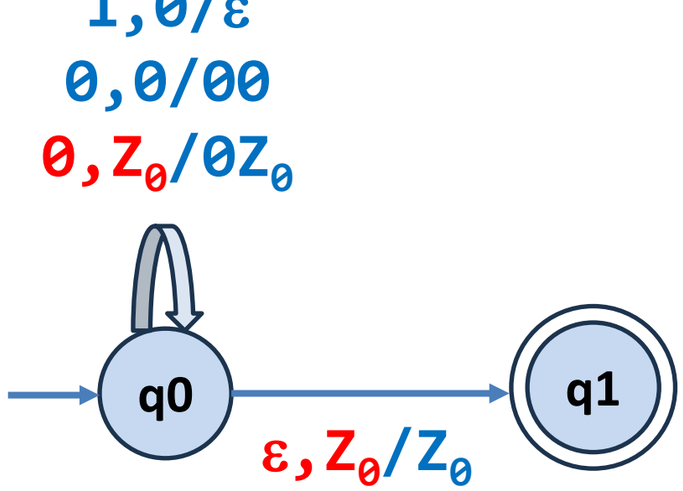


Is this a DPDA?



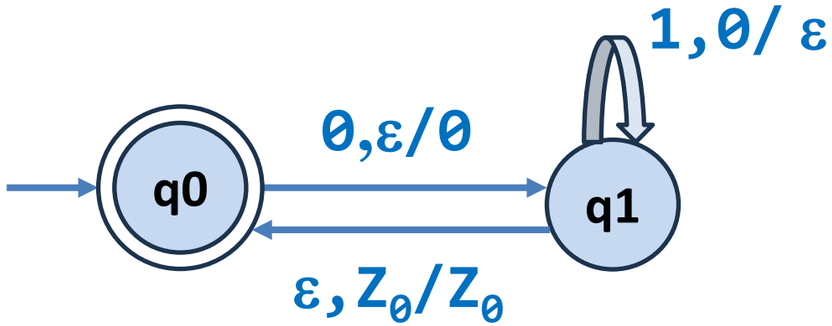
- **A PDA is deterministic if and only if:**
  - **δ(q, a, X) has at most one member for any q in Q, a in Σ∪{ε} and X in Γ.**
  - **If δ(q, a, X) is non-empty for some a in Σ, then δ(q, ε, X) must be empty.**

Is this a DPDA? NO



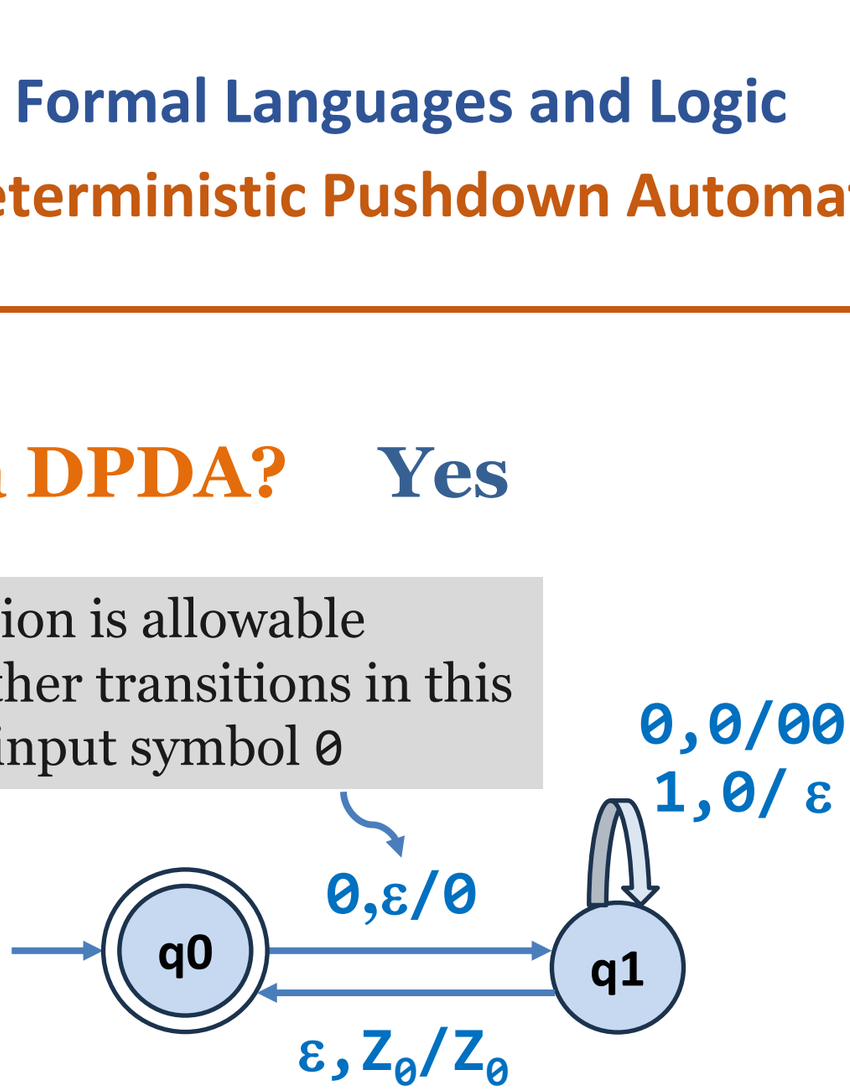
- **A PDA is deterministic if and only if:**
  - **δ(q, a, X) has at most one member for any q in Q, a in Σ∪{ε} and X in Γ.**
  - **If δ(q, a, X) is non-empty for some a in Σ, then δ(q, ε, X) must be empty.**

Is this a DPDA?



Is this a DPDA? Yes

This ε-transition is allowable because no other transitions in this state use the input symbol 0



This ε-transition is allowable because no other transitions in this state use the stack symbol Z_0

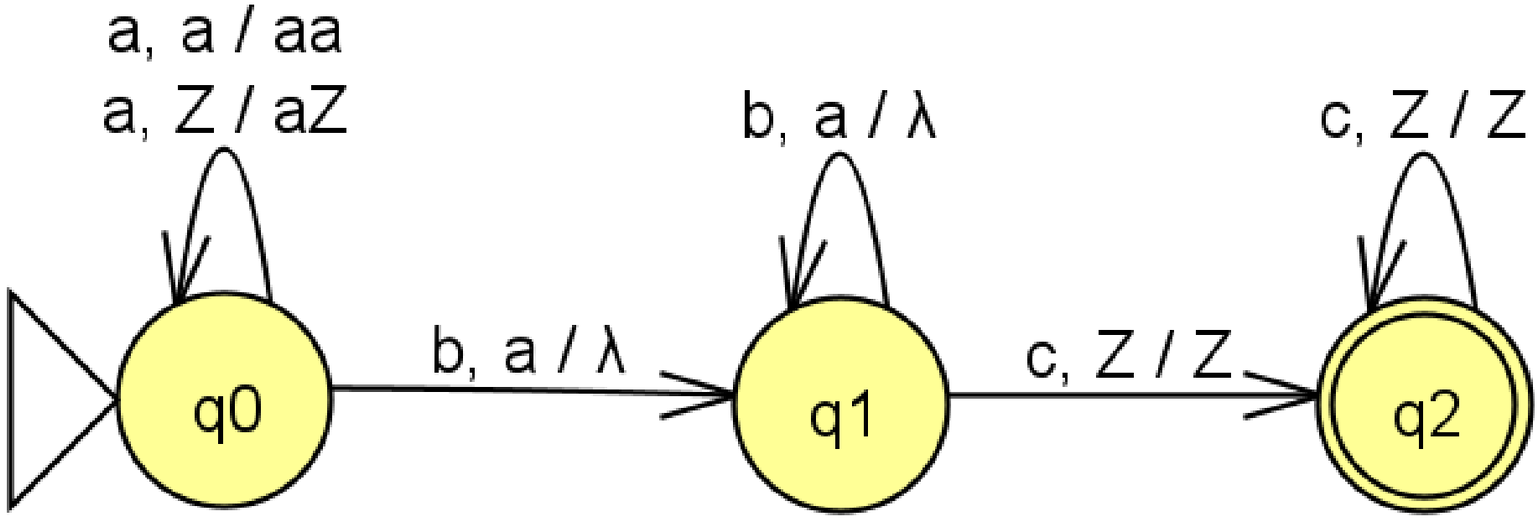
Exercise:

1) Construct DPDA for $a^n b^n c^m$ where n,m≥1
2) Construct DPDA for $a^{n+m} b^m c^n$ where n,m≥1

Exercise:

1) Construct DPDA for $a^n b^n c^m$ where n,m≥1.

Solution: L={abc, abcc, aabbc, … }



THANK YOU

Prakash C O and **Preet Kanwal**

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2 – Pushdown Automata (PDA)

Prakash C O and **Preet Kanwal**

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2: Non-Deterministic Pushdown Automata (NPDA)

Prakash C O and **Preet Kanwal**

Department of Computer Science & Engineering

Formal Definition of Non-Deterministic PDA (NPDA)

A pushdown automaton P is a seven-tuple $(Q, \Sigma, \Gamma, \delta, q_o, Z_o, F)$

where,

- **Q** is a finite set of states,
- **Σ** is a finite set of input symbols,
- **Γ** is a finite set of stack symbols,
- **δ** is a transition function, $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow 2^Q \times \Gamma^*$
- $q_o \in Q$ is the start/initial state,
- $Z_o \in \Gamma$, Initially, the stack holds a special symbol Z_o that indicates the bottom of the stack.
- $F \subseteq Q$ is a set of accepting states.

Note: In the above equation, Γ* represents stack new top symbols

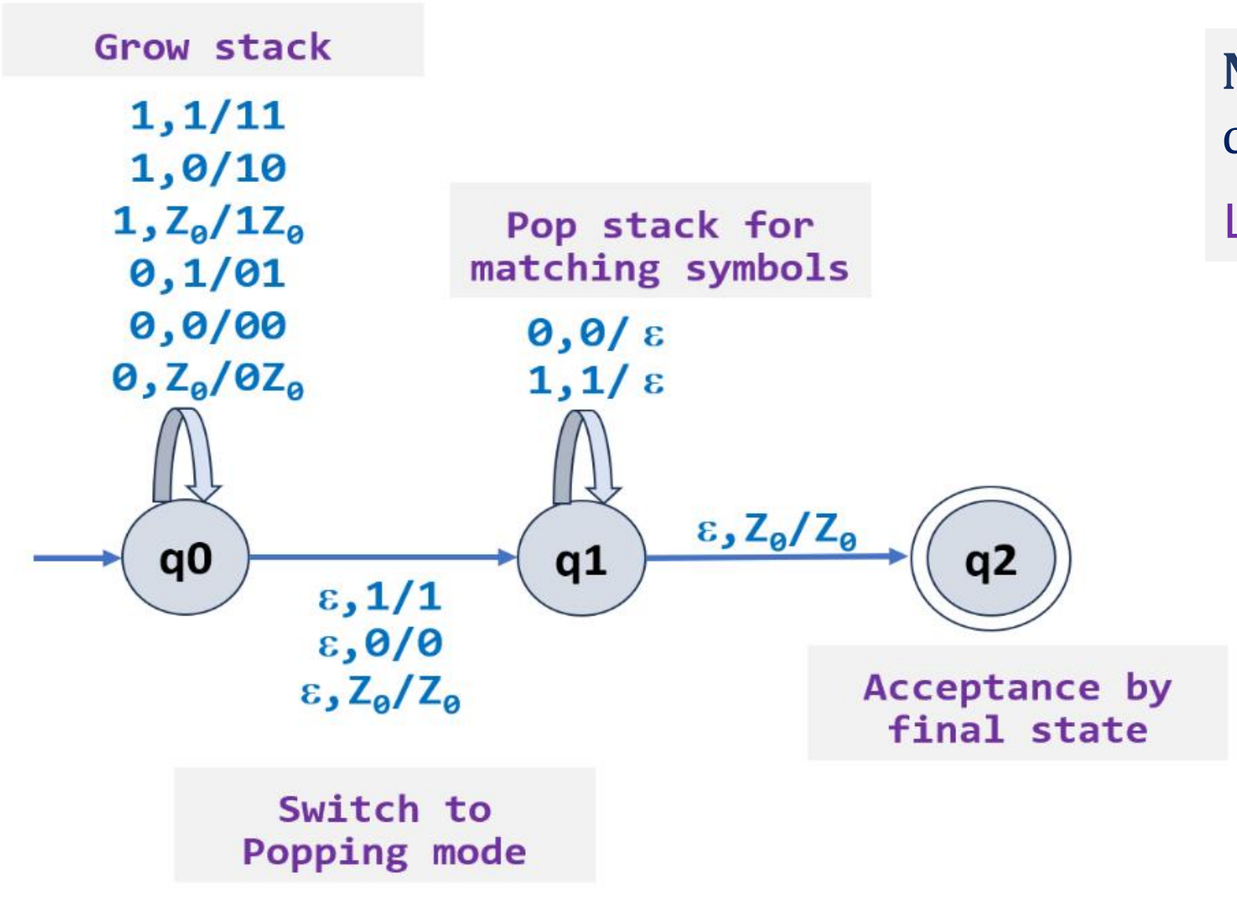
➢ Transition function of Non-Deterministic Pushdown Automaton

$$Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow 2^{Q} \times \Gamma^{*}$$

In any situation of the PDA, if several actions are possible, then the automaton is called a general, or nondeterministic pushdown automaton (NPDA).

Example:

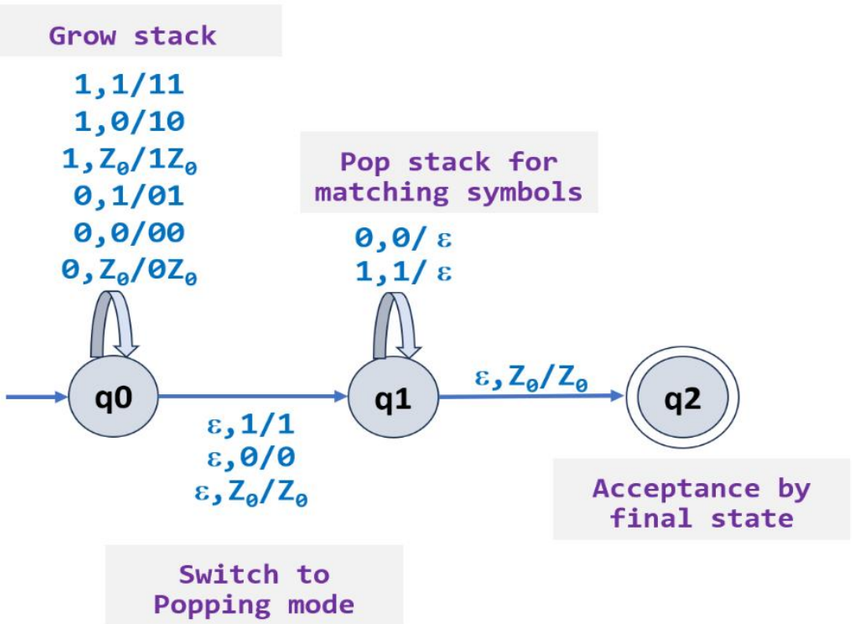
1) Consider the N-PDA recognizing L = { ww^R | w∈{0,1}* }

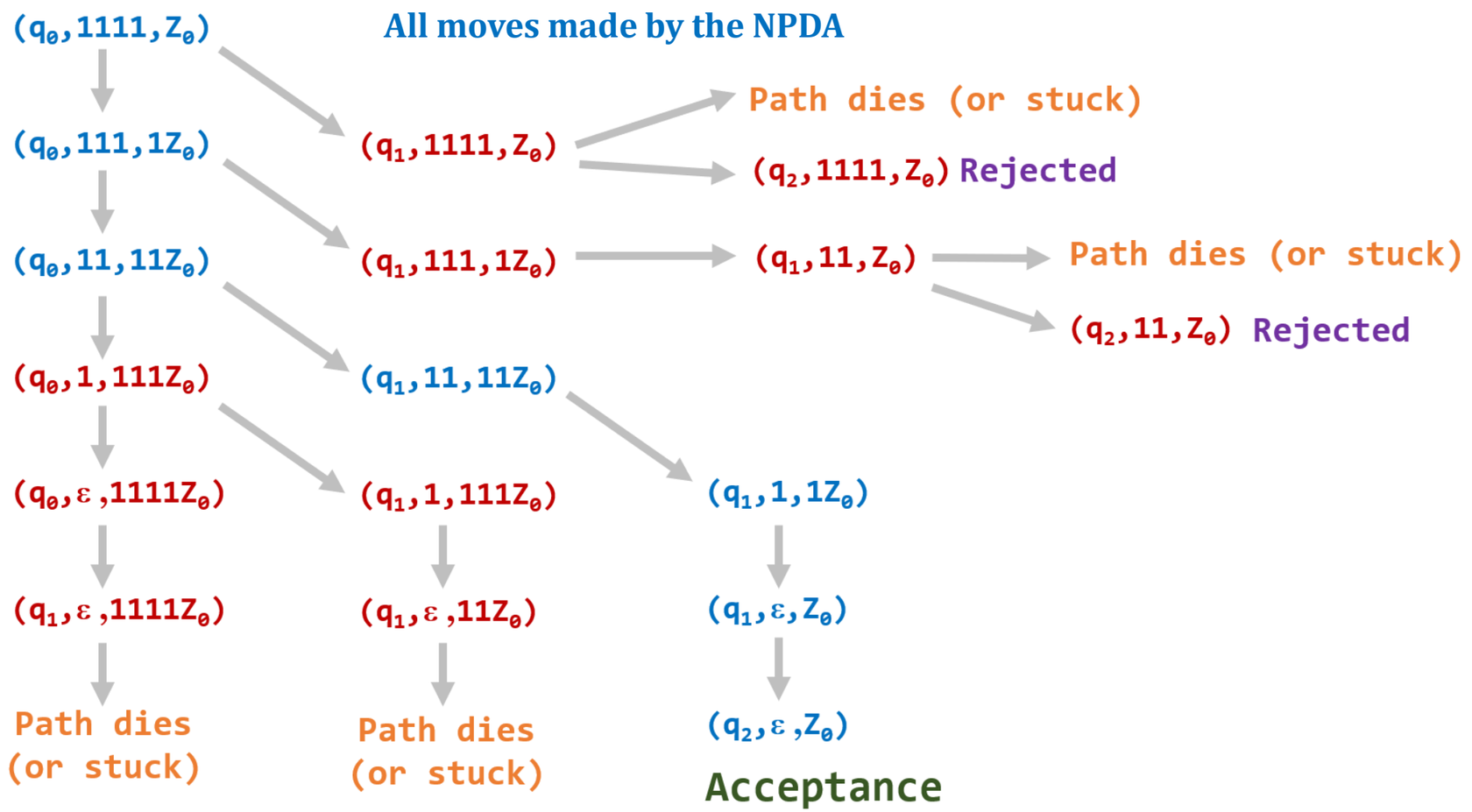


Note: The language L = { ww^R | w∈{0,1}* } contains even palindromes.

L={ε, aa, bb, abba, baab, aaaa, bbbb, …}

How does the N-PDA for Lww^R work on input "1111"?

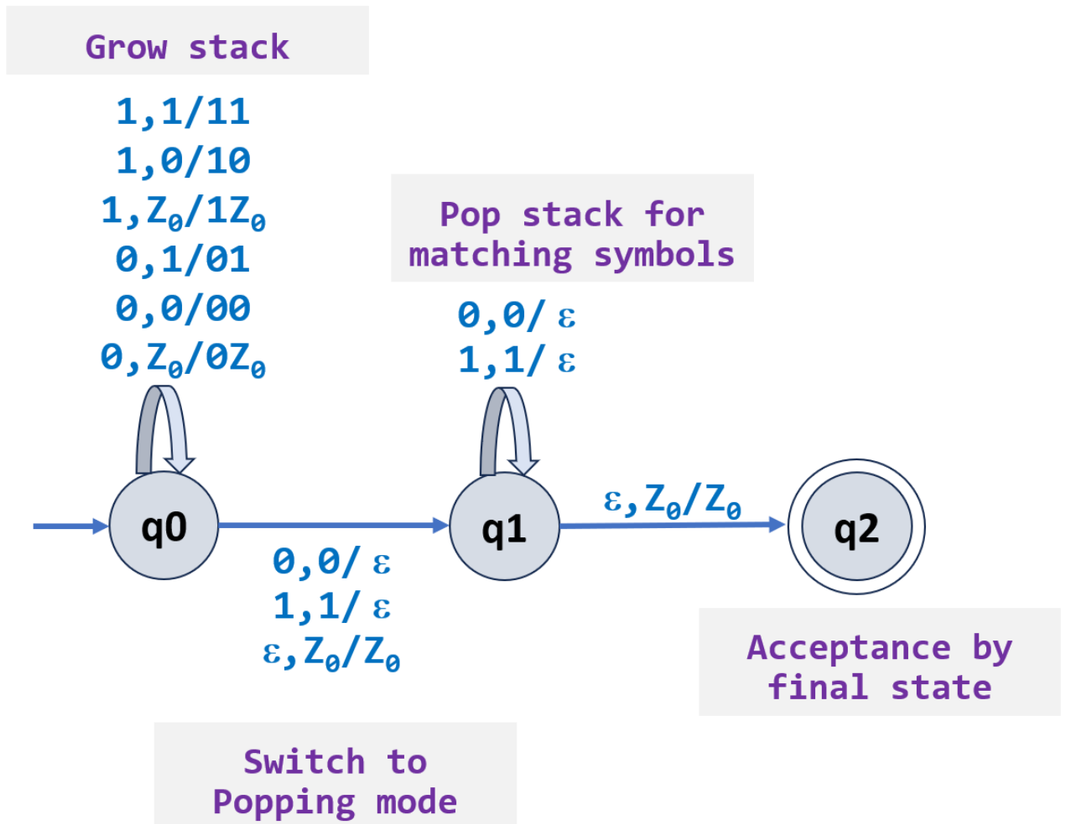




Example:

1) Consider the N-PDA recognizing L = { ww^R | w∈{0,1}* }

One more solution

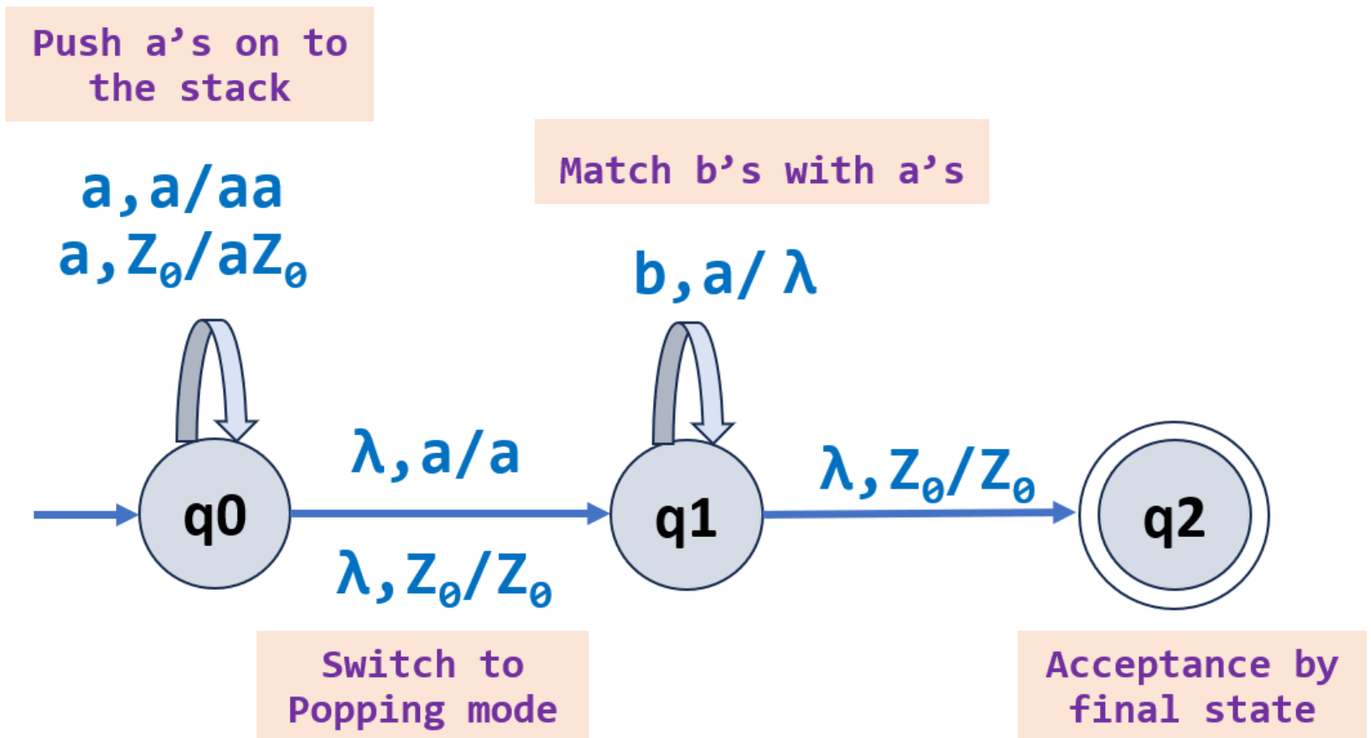


Note: The language L = { ww^R | w∈{0,1}* } contains even palindromes.

L={ε, aa, bb, abba, baab, aaaa, bbbb, …}

Example:

2) Consider the N-PDA recognizing L = { $a^n b^n$ | n≥0}



Note: The language L = { $a^n b^n$ | n≥0 } has even length strings containing equal number of a's followed by equal number of b's.

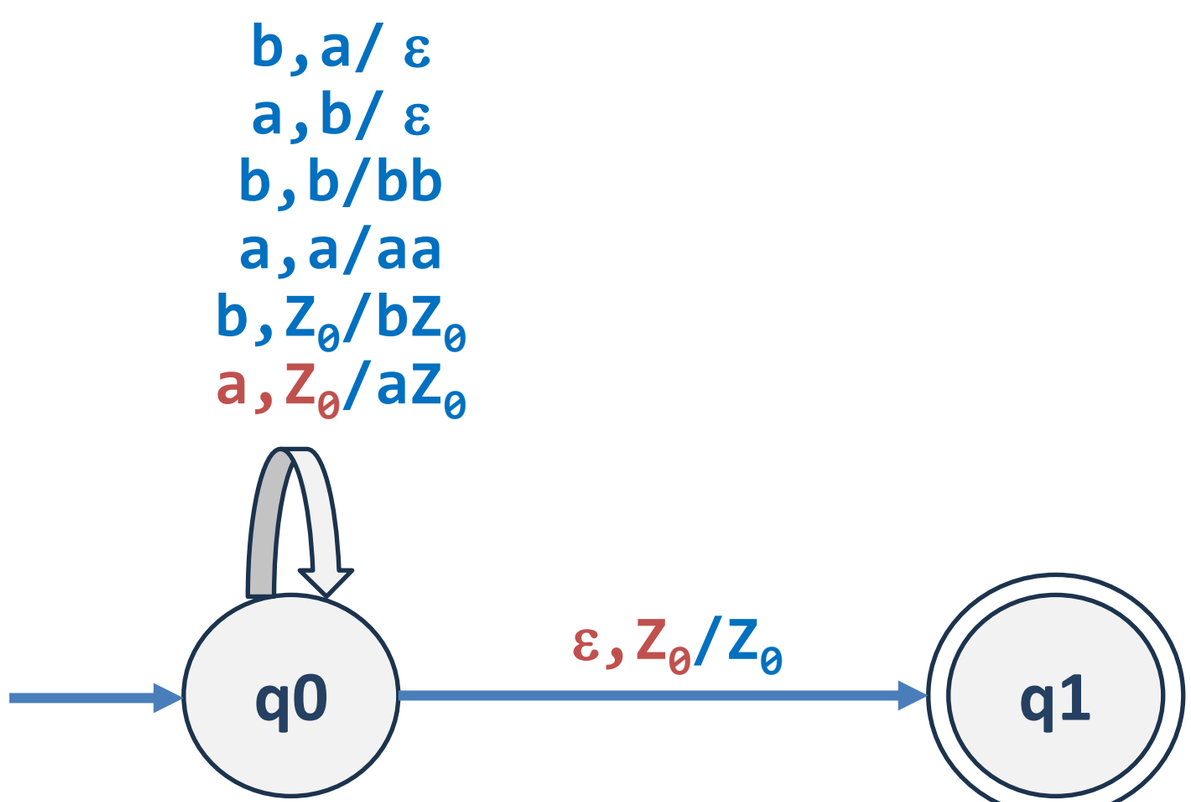
L={ λ, ab, aabb, aaabbb, aaaabbbb, … }

1. Construct a PDA for the language L={ w | $n_a(w)=n_b(w)$) and w ∈{a, b}* }

Solution: L={ε, ab, ba, abba, aabb, baab, abab, … }

1. Construct a PDA for the language L={ w | $n_a(w)=n_b(w)$) and w ∈{a, b}* }

Solution: L={ε, ab, ba, abba, aabb, baab, abab, … }

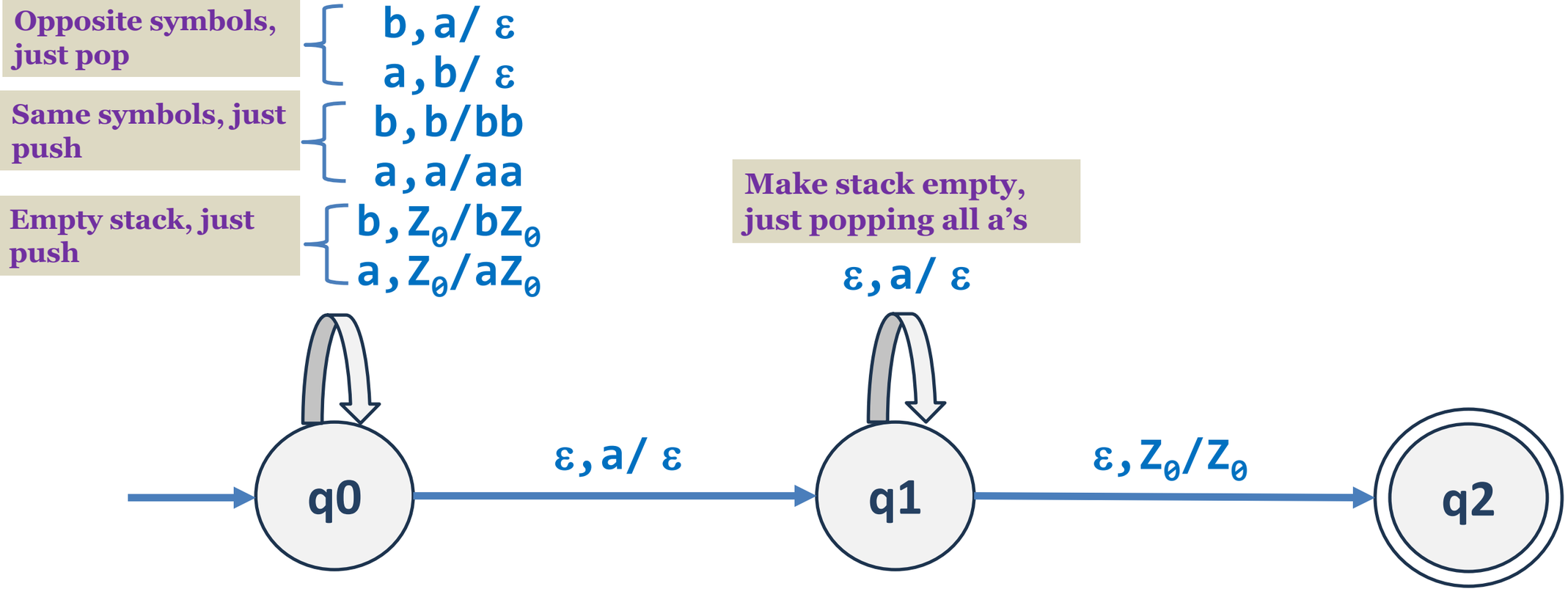


2. Construct a PDA for language L={ w | $n_a(w) > n_b(w)$) and w ∈{a, b*} }

Solution: L={ a, aa, aaa, … , aab, aba, baa, … }

2. Construct a PDA for language L={ w | $n_a(w) > n_b(w)$) and w ∈{a, b*} }

Solution: L={ a, aa, aaa, … , aab, aba, baa, … }

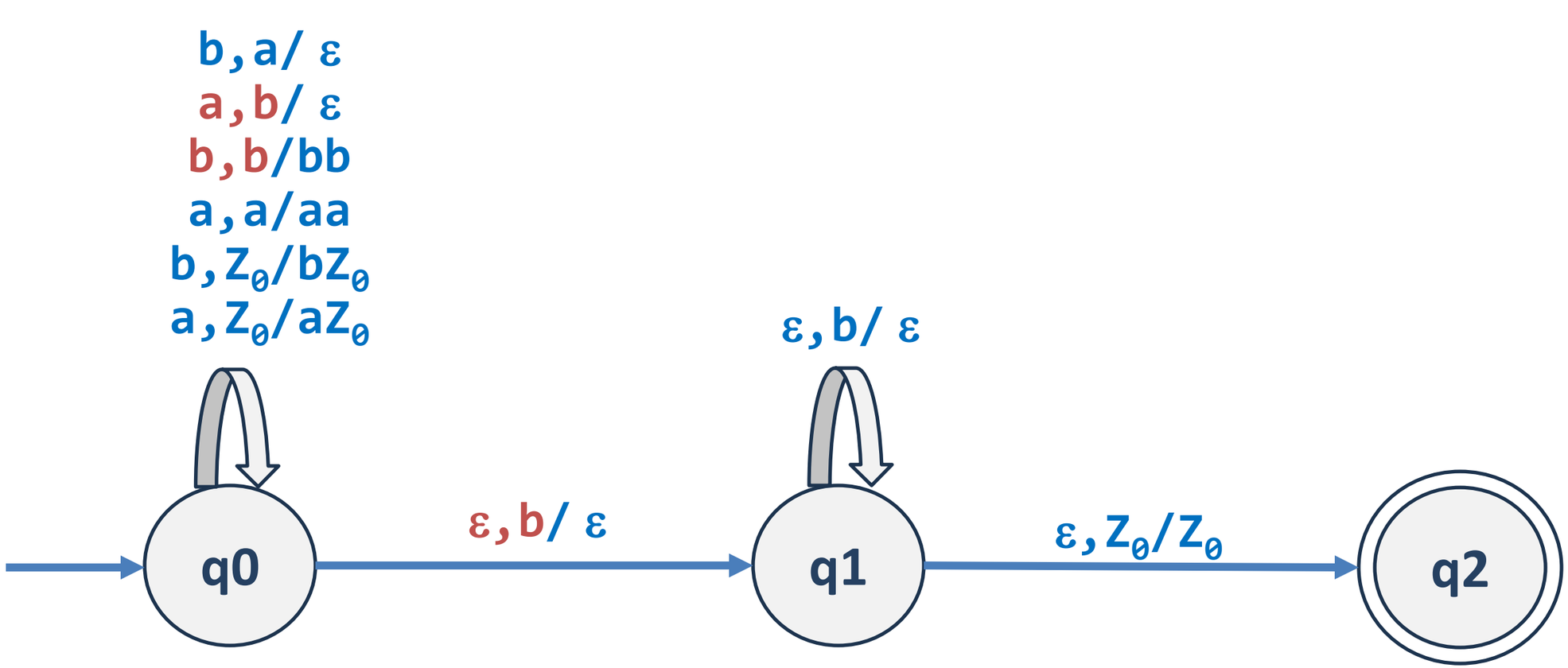


3. Construct a PDA for language L={ w | $n_a(w) < n_b(w)$) and w ∈ {a, b*} }

Solution:

3. Construct a PDA for language L={ w | $n_a(w) < n_b(w)$) and w ∈{a, b*} }

Solution:

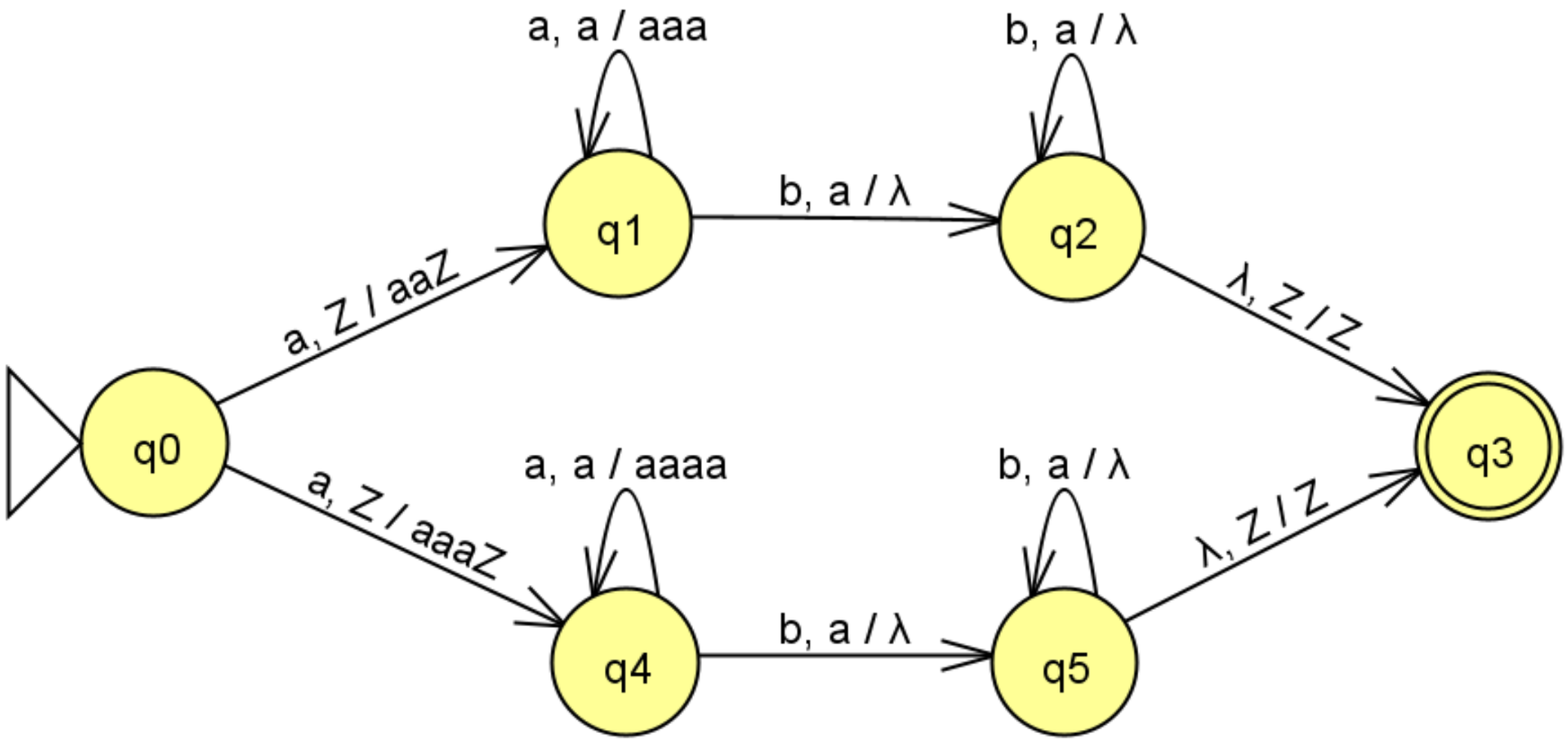


4. Construct PDA for language L = { $a^n b^{2n} \cup a^n b^{3n} \mid n \geq 1$ }

Solution:

4. Construct PDA for language L = { $a^n b^{2n} \cup a^n b^{3n} \mid n \geq 1$ }

Solution:



4a. Construct PDA for language L = { $a^n b^{2n} \cup a^n b^{3n} \mid n \geq 0$ }

Solution:

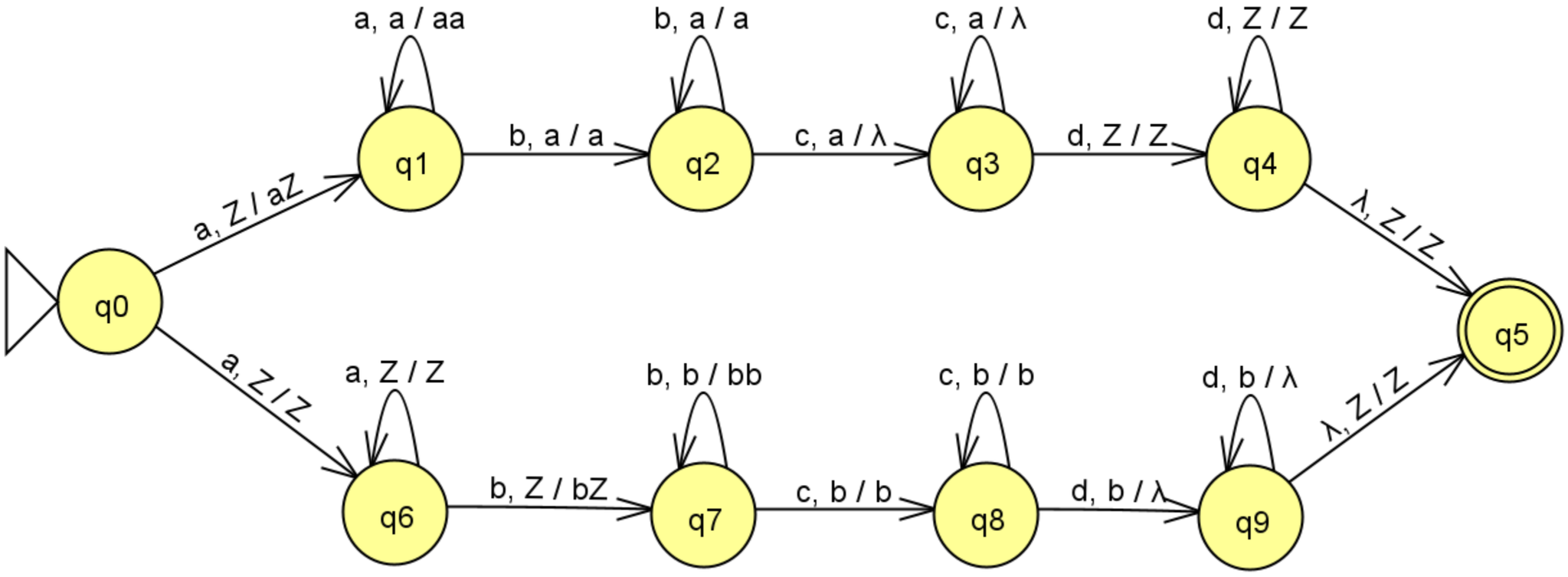
PES UNIVERSITY ONLINE

5. Construct PDA for the language . $L = \{a^i b^j c^k d^l \mid i=k \text{ or } j=l \text{ and } i,j,k,l \geq 1\}$

Solution:

5. Construct PDA for the language . $L = \{a^i b^j c^k d^l \mid i=k \text{ or } j=l \text{ and } i,j,k,l \geq 1\}$

Solution:

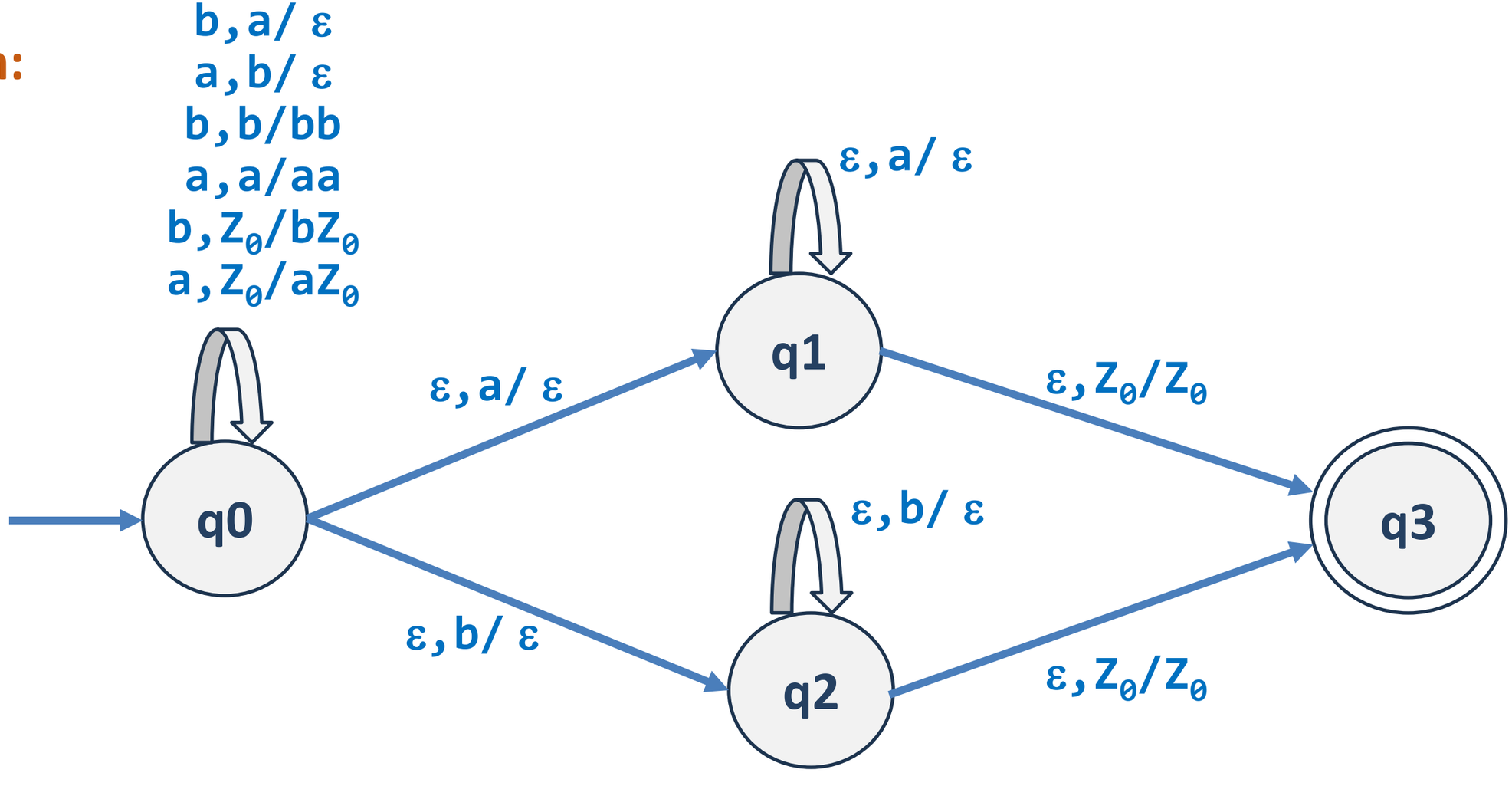


6. Construct PDA for the language . $L = \{ w \mid w \in \{a, b\}^* \text{ and } n_a(w) \neq n_b(w) \}$

Solution:

6. Construct PDA for the language . L = { w | w ∈ {a, b}* and $n_a(w) \neq n_b(w)$ }

Solution:



Exercise:

1) Construct PDA for language $L = \{ a^n w^R w b^n \mid w \in \{a, b\}^* \text{ and } n \geq 0 \}$

2) Construct PDA for language $L = \{a^m b^n c^p d^q \mid m+n=p+q ; m,n,p,q>=1\}$

THANK YOU

Prakash C O and **Preet Kanwal**

Department of Computer Science & Engineering

PES UNIVERSITY

Additional Info:

1) How to convert a final state acceptance PDA into an empty stack acceptance PDA?
2) How to convert an empty stack acceptance PDA into a final state acceptance PDA?
3) Construct a PDA that accepts { wcw^R | w is any string of a's and b's } by empty stack.